

Verilog FPGA Program 0

Basic

아이힐

Version 1.2

목차

1	개요	2
2	register 는 무엇인가?	3
2.1	reg 회로 분석	4
2.2	simulation	21
3	reg 와 wire	34
3.1	실습 - 1 (1-clock 안에 이루어지는 경우)	35
3.2	실습 - 2 (2-clock 안에 이루어지는 경우)	38
3.3	delay 오류로 인한 결과	40
4	signed, unsigned 덧셈, 뺄셈	44
4.1	unsigned 덧셈	44
4.2	unsigned 뺄셈	46
4.3	signed 덧셈, 뺄셈	47
4.4	unsigned number 곱하기	52
4.4.1	상수 곱하기	52
4.4.2	dsp (dsp slice)	53
4.4.3	변수 곱하기	54
4.4.4	10bits * 10bits 연산 구현	55
4.4.5	10bits * 10bits 연산 비교	57
5	SM (State Machine) 구현	61
5.1	SM 구성	61
5.2	상태 정의	62
5.3	상태 플래그 생성	62
5.4	코드 구현	62
5.5	상태 전이 구현	64
5.6	simulation 결과 확인	65
6	for 루프 구현	67
6.1	C 로 구현	67
6.2	verilog 로 구현	68
7	그 외의 내용	72
8	Revision History	76

1. 개요

본서는 Verilog를 입문하시는 분들을 위한 내용으로 구성되었습니다. verilog는 다른 고급언어(C, C++, C#, JAVA, PYTHON 등)와 다른 세계의 언어입니다. verilog는 HW를 설계하는 언어입니다. 고급 언어들은 설계된 HW 상위에서 구현되지만, verilog는 HW 자체를 설계하는 언어입니다.

본서는 보드에서 실습을 진행하지 않고, simulation으로 결과를 확인합니다. 본서에서 설명하는 모든 예제는 vivado 2018.3 버전을 사용하였습니다. 그러나 다른 버전에서도 문제없이 동작할 것으로 생각됩니다.

본서는 기본적인 내용들 위주로 설명되어 있습니다. 저의 다른 강의 내용들을 학습하시기 전에 기본적인 내용을 학습한다는 생각으로 가볍게 보시길 바랍니다.

전체적인 내용은 다음과 같습니다.

2장은 verilog에서 가장 중요한 register에 대한 내용입니다. register를 설명하고, schematic을 확인하고 simulation을 통해 register에 대해서 확실하게 이해할 수 있도록 설명합니다.

3장은 register와 wire의 차이점에 대해서 설명합니다. 어떤 때에는 reg를 사용하고, 어떤 때에는 wire를 사용하는지 설명합니다.

4장은 verilog에서 사용하는 모듈과 상위 고급언어에서 사용하는 함수에 대하여 설명합니다.

5장은 연산에 관한 내용입니다. verilog에서는 데이터 형이 없습니다. 사용자가 정해서 거기에 맞게 연산을 구현해야 합니다. 특히 signed, unsigned 의 덧셈, 뺄셈을 알아봅니다. 이를 통하여 bit 연산에 대한 개념을 이해할 수 있습니다. 또한 곱셈을 구현하는 2가지 방법에 대해서도 설명합니다.

6장은 verilog 알고리즘 구현에 핵심적인 SM (State Machine)에 대해서 설명합니다. SM을 사용해서 알고리즘을 구현할 때의 장점과 어떻게 SM을 구현하는지 설명합니다.

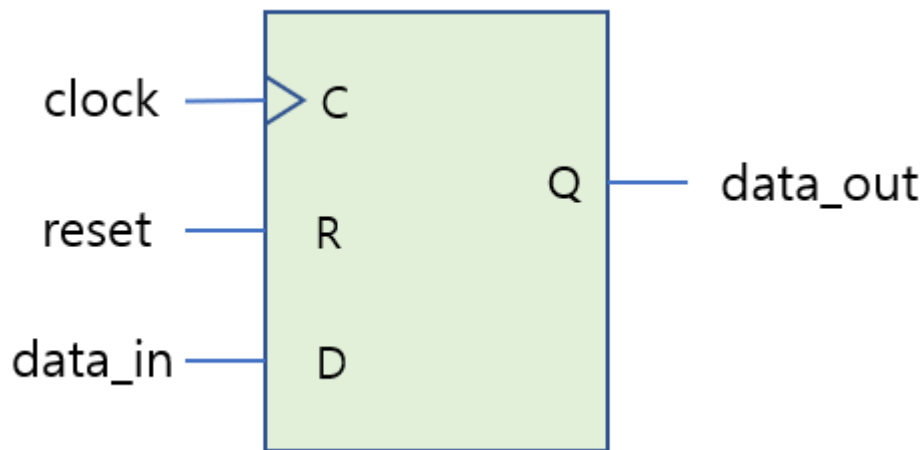
7장은 그 외의 내용으로 verilog 입문시에 자주 범하게 되는 오류들에 대해서 설명합니다.

본서를 구매하시는 분들에게는 본서에서 설명된 소스를 제공하여 드립니다. 소스는 자료실에서 다운받으실 수 있고 자료실 링크는 세션 0 - 수업자료 1 의 첨부파일에서 다운받으시면 됩니다.

2. register 는 무엇인가?

reg는 verilog 에서 가장 기본이 되는 개념으로 모든 verilog 프로그램은 reg로 구성되어 있다고 해도 과언이 아닙니다. 그만큼 verilog를 배우는데 있어서 reg를 이해하는 것은 매우 중요합니다. (reg는 register를 의미합니다)

reg는 D Flip-Flop으로 구성되어 있습니다. 아래 그림은 D Flip-Flop을 보여줍니다.



D Flip-Flop의 동작은 아래와 같습니다.

- ✓ reset 이 1 (High) 이면, clock 이나 data_in 값에 관계없이 Q (data_out) 값은 0 가 됩니다.
- ✓ reset 이 0 (Low) 이면, clock 이 rising edge (0 에서 1로 바뀌는 순간)에 data_in(D) 값이 data_out(Q) 값으로 전달됩니다. 즉 clock이 rising edge 일 때 Q 값은 D 값으로 변합니다.
- ✓ data_out 값은 위 2가지의 경우가 아니면 값이 계속 유지됩니다.

여기에서 몇가지 개념에 대해서 설명하도록 하겠습니다.

1) Active Low, Active High

어떤 신호가 Active Low라고 하는 것은, Low 값이 들어올 때 동작한다는 의미입니다. 예를 들어 wen (write enable) 신호가 Active Low로 동작한다는 것은, wen에 Low (0) 을 주었을 때 write 상태가 되고, 반대로 High (1)을 주었을 때에는 write를 할 수 없는 상태가 된다는 것을 의미합니다.

어떤 신호가 Active High라고 하는 것은, High 값이 들어올 때 동작한다는 의미입니다. 예를 들어 wen (write enable) 신호가 Active High로 동작한다는 것은, wen에 High (1) 을 주었을 때 write 상태가 되고, 반대로 Low (0)을 주었을 때에는 write를 할 수 없는 상태가 된다는 것을 의미합니다.

위에서 설명한 D Flip-Flop의 reset은 Active High 입니다. reset 신호가 High(1)이 되었을 때, Active (Q 값이 reset 됨, 0 가 됨)되기 때문입니다.

2.1 reg 회로 분석

verilog에서 reg를 생성하는 keyword는 reg 입니다.

```
36 reg          sig1 ;
37 reg [3:0]    sig2 ;
```

- ✓ 36 라인은 1bit sig1 신호를 reg 로 생성합니다.
- ✓ 37 라인은 4bits sig2 신호를 reg 로 생성합니다.

reg를 구현하는 코드는 always 문을 사용합니다.

```
40 reg          sig1 ;
41 always @(posedge clock or negedge reset)
42 begin
43     if(~reset)    sig1 <= 1'b0;
44     else          sig1 <= in1 ;
45 end
46
47 reg [3:0]    sig2 ;
48 always @(posedge clock or negedge reset)
49 begin
50     if(~reset)    sig2 <= 4'b0;
51     else          sig2 <= in2 ;
52 end
```

- ✓ 라인 40 : 1bit sig1을 reg로 생성합니다.
- ✓ 라인 41 - 45 : sig1을 always 문을 사용하여 구현합니다.
- ✓ 라인 41 : “posedge clock” 은 clock의 rising edge (positive edge)에서 데이터가 전달됨을 의미합니다.
“negedge reset” 은 reset 신호가 Active Low 신호임을 의미합니다. 즉 reset 값이 0 (Low) 일 때, 출력 값(sig1)이 0 가 된다는 의미입니다.
- ✓ 라인 43 : reset 신호가 Low (0) 일 때 sig1 이 0가 됩니다.
- ✓ 라인 44 : reset 신호가 High (1) 일 때에는, clock이 positive edge 일 때 sig1 값이 in1 값으로 변경됩니다.
- ✓ 라인 47 : 4bit sig2을 reg로 생성합니다.
- ✓ 라인 48 - 52 : sig2을 always 문을 사용하여 구현합니다.
- ✓ 라인 48 : “posedge clock” 은 clock의 rising edge (positive edge)에서 데이터가 전달됨을 의미합니다.
“negedge reset” 은 reset 신호가 Active Low 신호임을 의미합니다. 즉 reset 값이 0 (Low) 일 때, 출력 값(sig2)이 0000 가 된다는 의미입니다.
- ✓ 라인 50 : reset 신호가 Low (0) 일 때 sig2 이 0000 가 됩니다.
- ✓ 라인 51 : reset 신호가 High (1) 일 때에는, clock이 positive edge 일 때 sig2 값이 in2 값으로 변경됩니다.

always 문에 대해서 몇가지를 설명합니다.

- ✓ 대입문 (왼쪽의 값이 오른쪽 값으로 변경됨)은 “<=”을 사용합니다.
- ✓ 대입문의 왼쪽값의 bits 수와 오른쪽의 bits 수는 같아야 합니다.

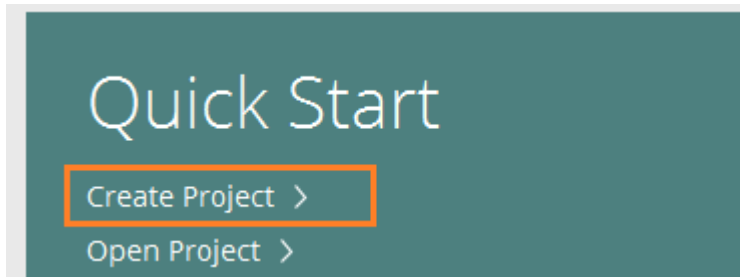
상수 표기 방법에 대하여 중요한 몇가지만 설명합니다.

- ✓ 상수는 “자리수” + “” + “진수” + “값” 으로 표현합니다.
- ✓ 진수는 b(2진수), d(10진수), h(16진수) 3가지가 있습니다.
- ✓ 모든 값이 0 일 때에는 0 하나만 사용해도 됩니다.
- ✓ 값들 사이에 자리수를 구분하기 위하여 “_”를 추가하여 사용할 수 있습니다.
- ✓ 아래는 몇가지 표현 방법을 나타냅니다.

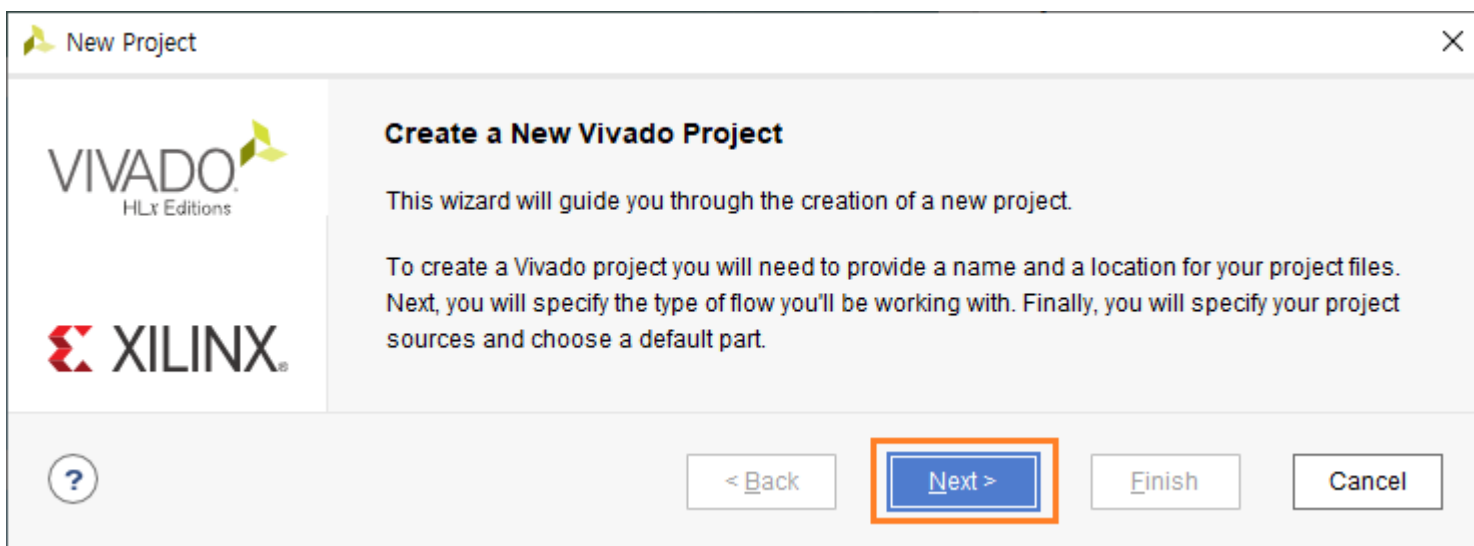
- 4bits 15를 표현하는 방법 : 4'b1111, 4'd15, 4'hf
- 8bits 250(0xfa)을 표현하는 방법 : 8'b11111010, 8'b1111_1010, 8'd250, 8'hfa
- 8bis 0을 표현하는 방법 : 8'b0, 8'b0000_0000, 8'd0, 8'h0, 8'h00
- 8bits 12을 표현하는 방법 : 8'd12, 8'hc, 8'h0c, 8'b0000_1100, 8'b1100

reg의 회로 구성이 어떻게 되는지 알아보겠습니다. vivado 툴에서 reg을 생성하고 변환된 회로를 살펴보겠습니다.

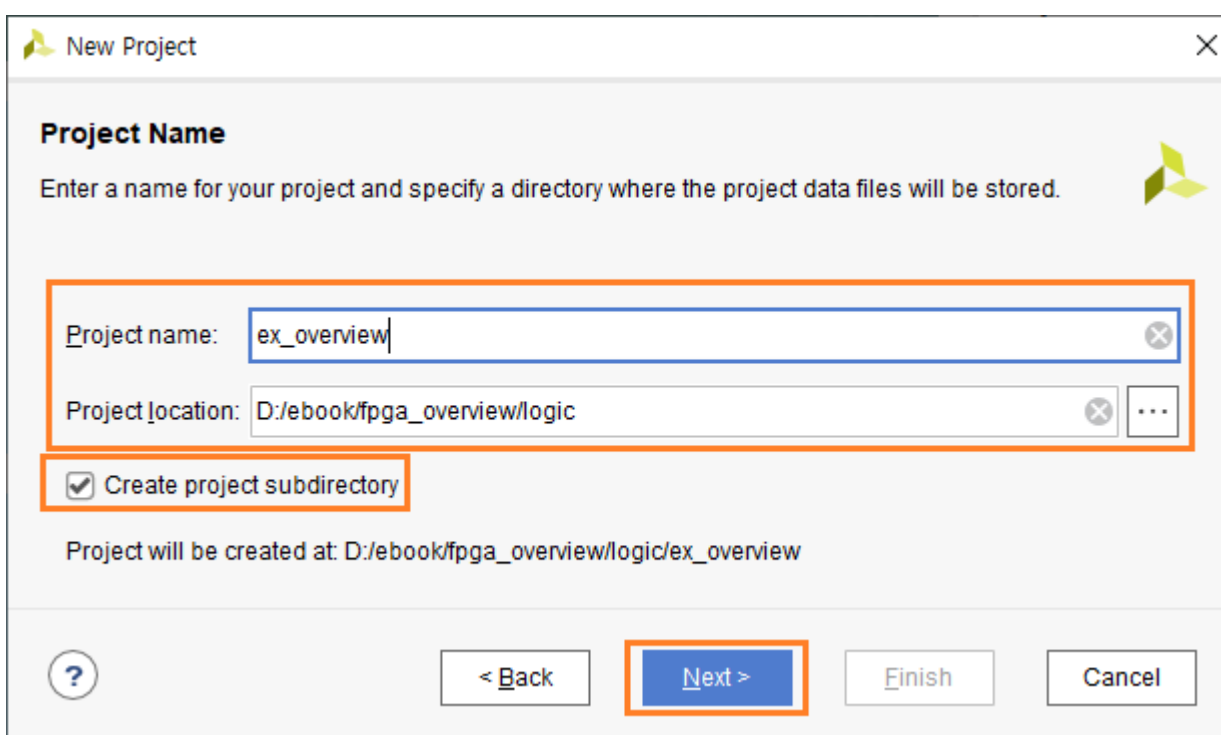
vivado를 실행하고(본서에서는 vivado 2018.3을 사용합니다. 버전이 달라도 크게 다르지 않습니다) “Create Project”를 클릭해서 프로젝트를 생성합니다.



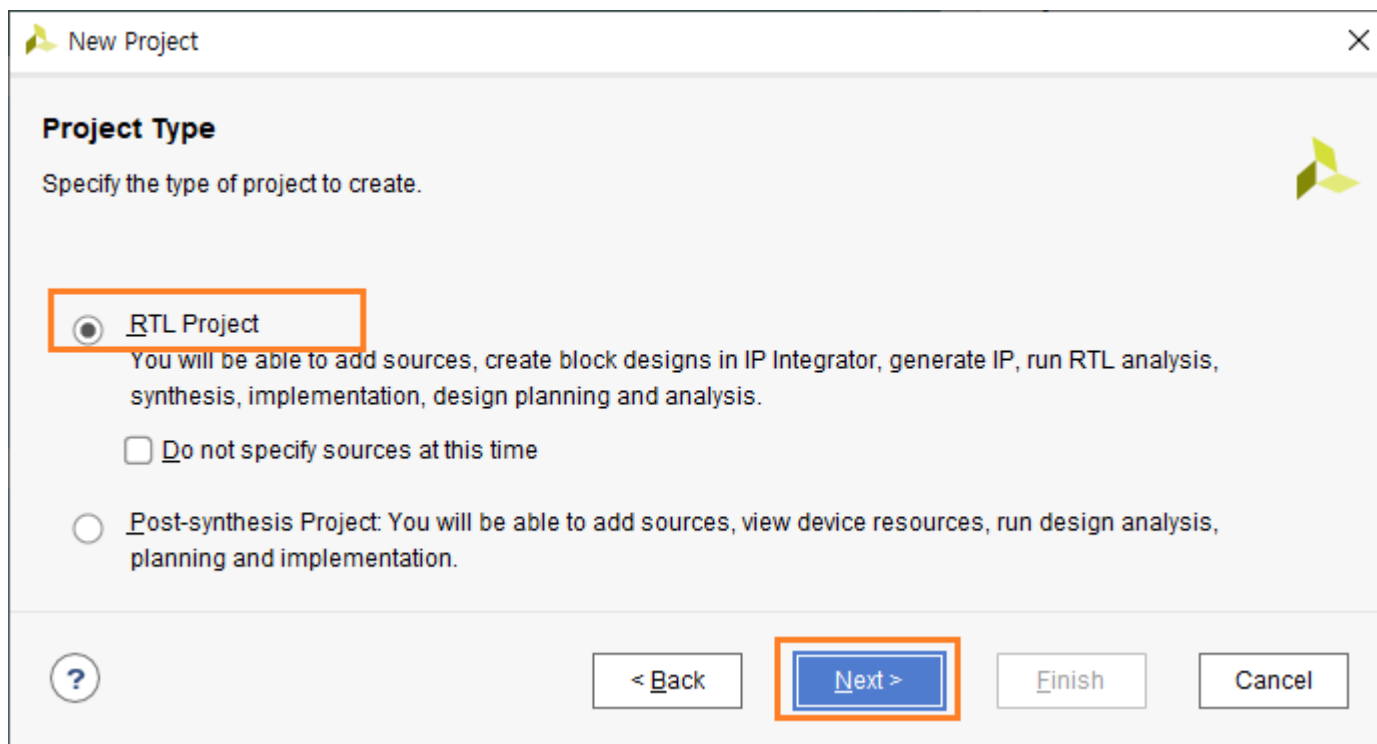
Next를 클릭합니다.



Project name, Project location 을 설정하고 Next를 클릭합니다.



Project Type을 RTL Project로 선택하고 Next를 클릭합니다.



The 'New Project' dialog box is shown with the 'Project Type' tab selected. The 'RTL Project' radio button is selected and highlighted with an orange box. Below it, a description states: 'You will be able to add sources, create block designs in IP Integrator, generate IP, run RTL analysis, synthesis, implementation, design planning and analysis.' There are two other options: 'Do not specify sources at this time' (unchecked) and 'Post-synthesis Project' (unchecked). At the bottom, the 'Next >' button is highlighted with an orange box, along with '< Back', 'Finish', and 'Cancel' buttons.

New Project

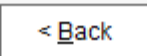
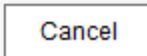
Project Type

Specify the type of project to create.

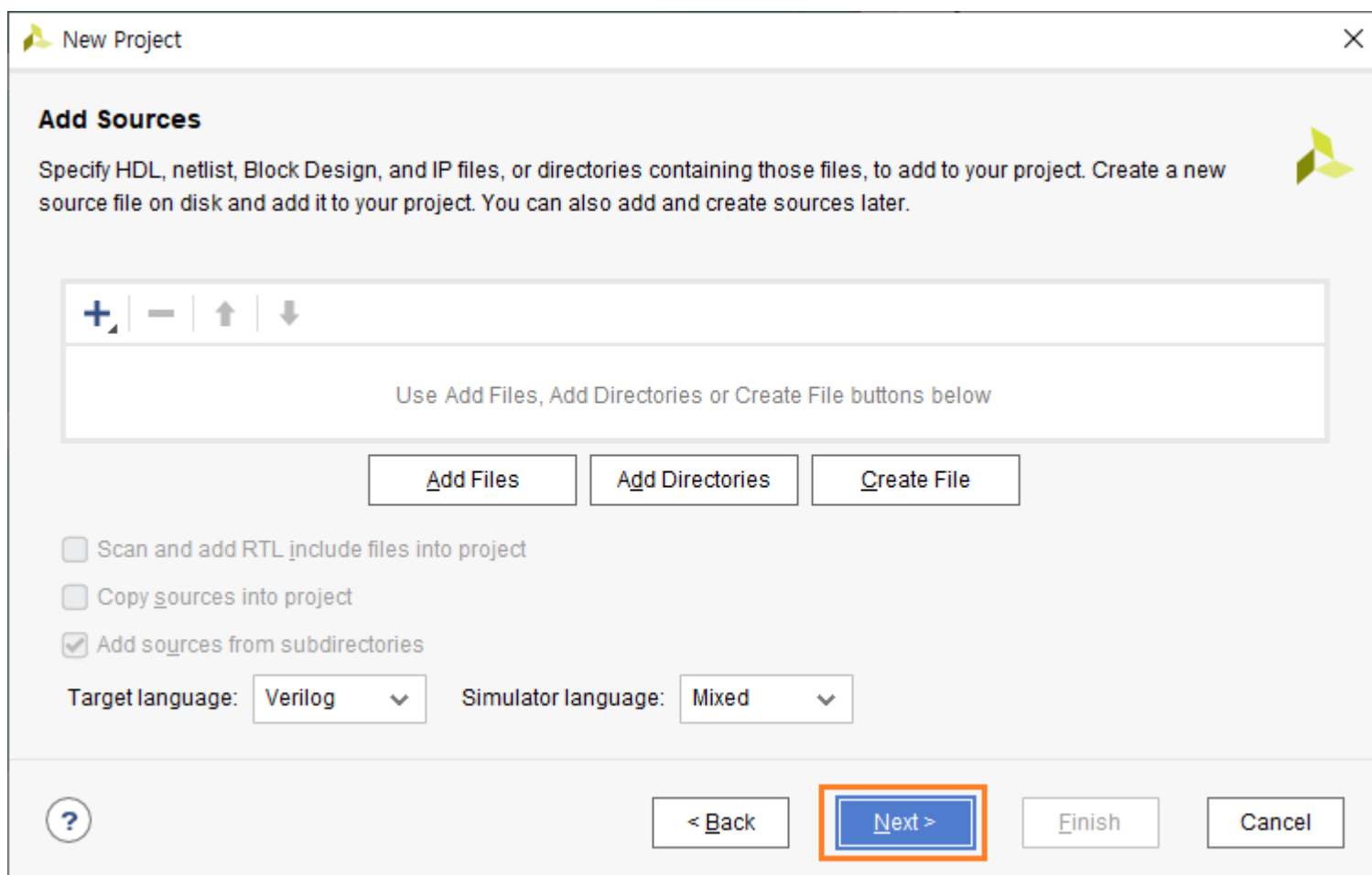
☒ **RTL Project**
You will be able to add sources, create block designs in IP Integrator, generate IP, run RTL analysis, synthesis, implementation, design planning and analysis.

☐ Do not specify sources at this time

☐ Post-synthesis Project: You will be able to add sources, view device resources, run design analysis, planning and implementation.

Add Sources 윈도우에서 Next를 클릭합니다. 소스 코드는 프로젝트 생성 후 추가합니다.



The 'New Project' dialog box is shown with the 'Add Sources' tab selected. The 'Add Files', 'Add Directories', and 'Create File' buttons are visible. The 'Add sources from subdirectories' checkbox is checked. The 'Target language' is set to 'Verilog' and the 'Simulator language' is set to 'Mixed'. At the bottom, the 'Next >' button is highlighted with an orange box, along with '< Back', 'Finish', and 'Cancel' buttons.

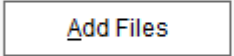
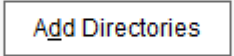
New Project

Add Sources

Specify HDL, netlist, Block Design, and IP files, or directories containing those files, to add to your project. Create a new source file on disk and add it to your project. You can also add and create sources later.

Use Add Files, Add Directories or Create File buttons below

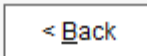
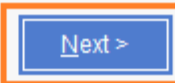
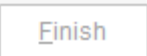
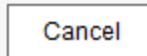
  

☐ Scan and add RTL include files into project

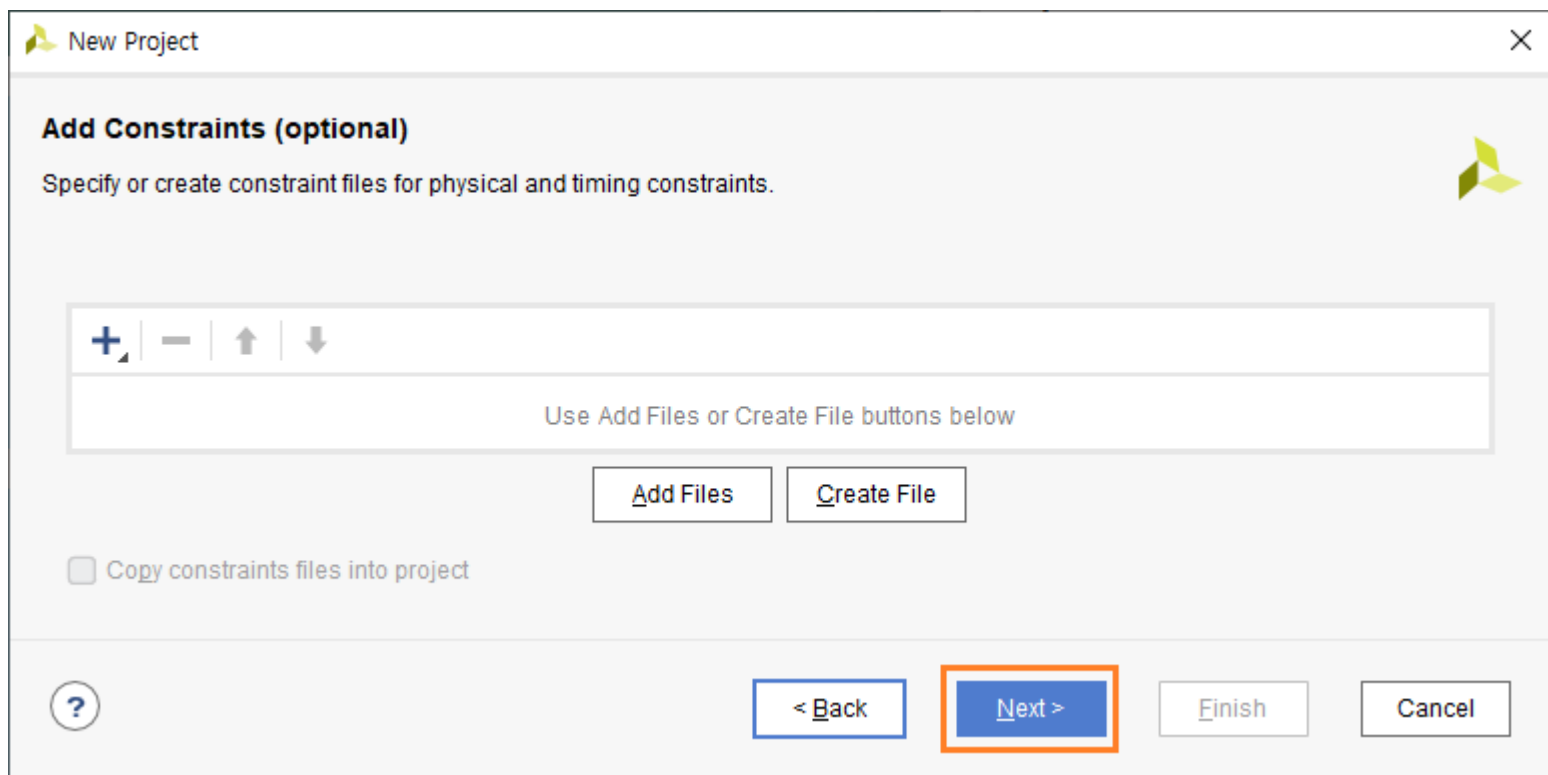
☐ Copy sources into project

☒ Add sources from subdirectories

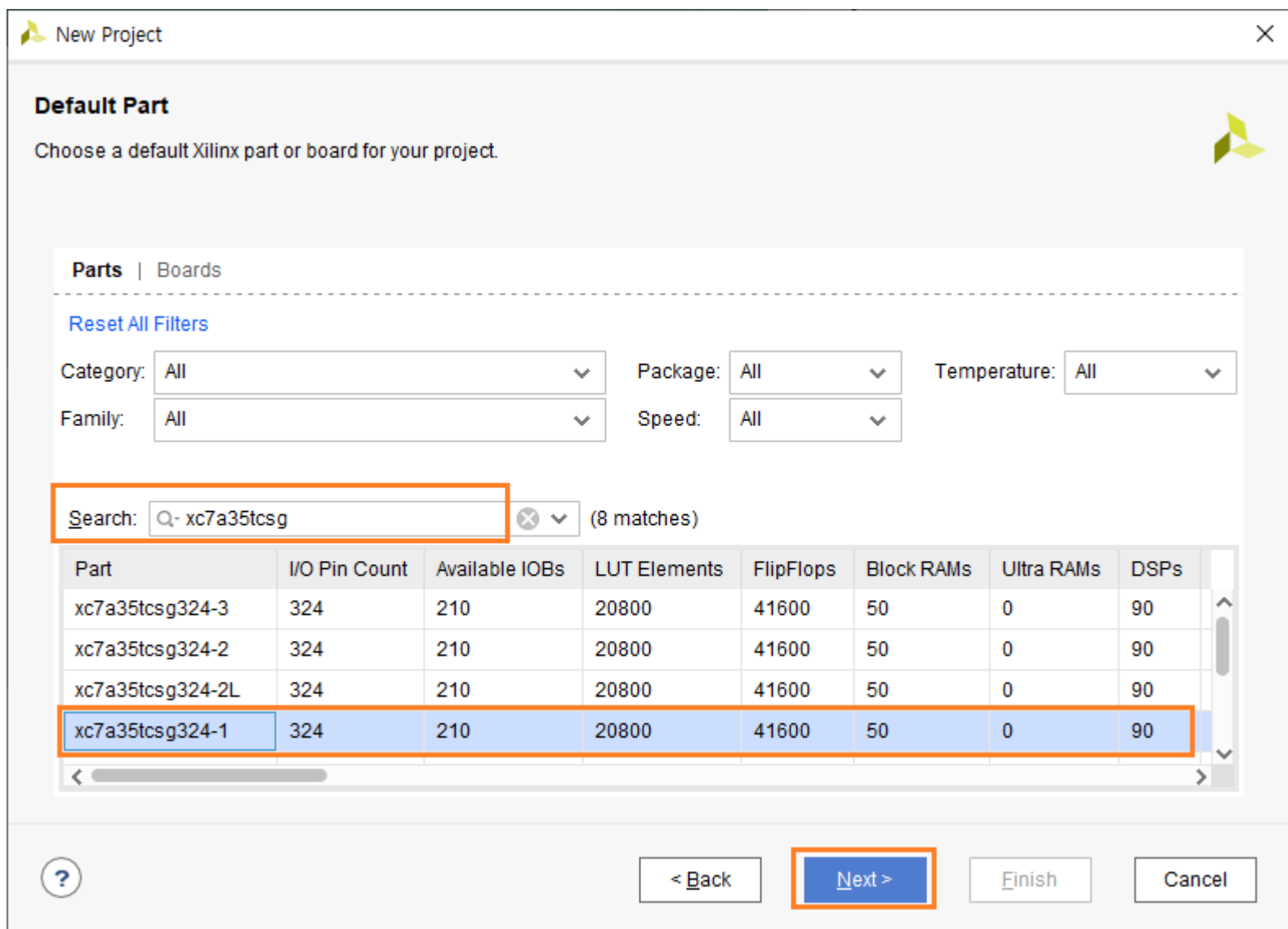
Target language: Verilog Simulator language: Mixed

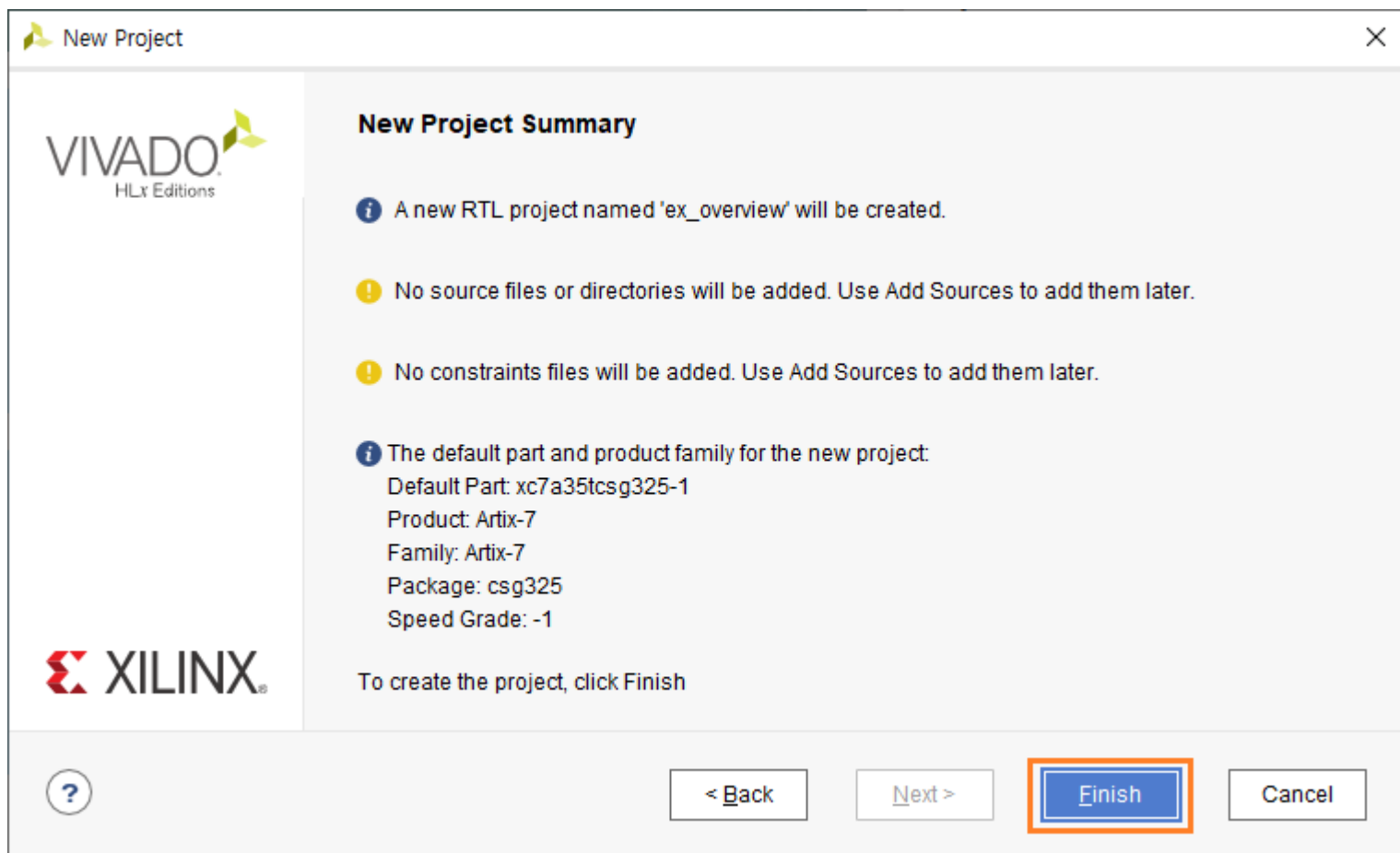
Add Constraints 윈도우에서 Next를 클릭합니다. 프로젝트 생성 후 Constraints 파일을 추가합니다.



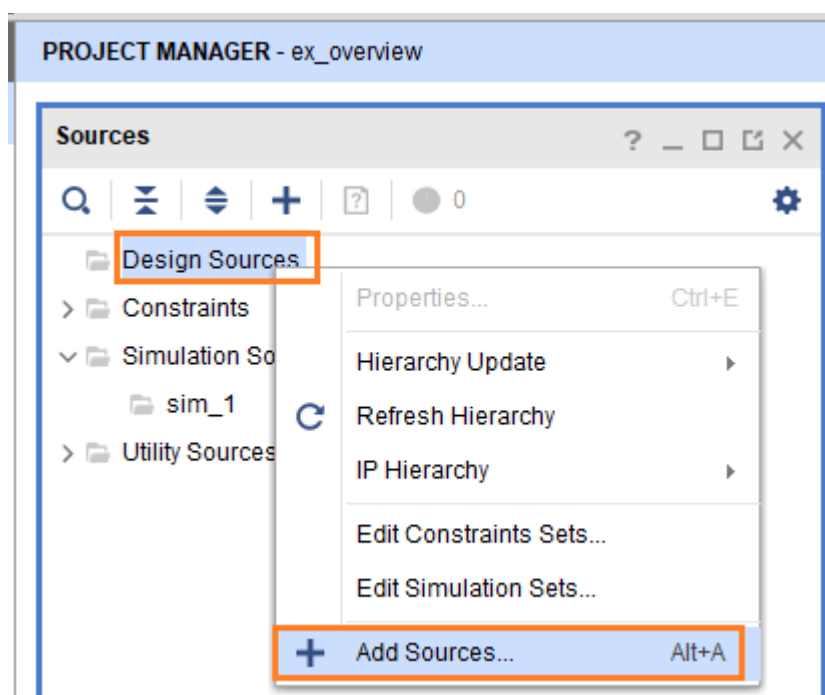
Part를 선택합니다. 본 강의에서는 Arty A7 35T 보드를 사용합니다. Search 에 xc7a35tcsg 입력 후 xc7a35tcsg324-1을 선택하고 Next를 클릭합니다.



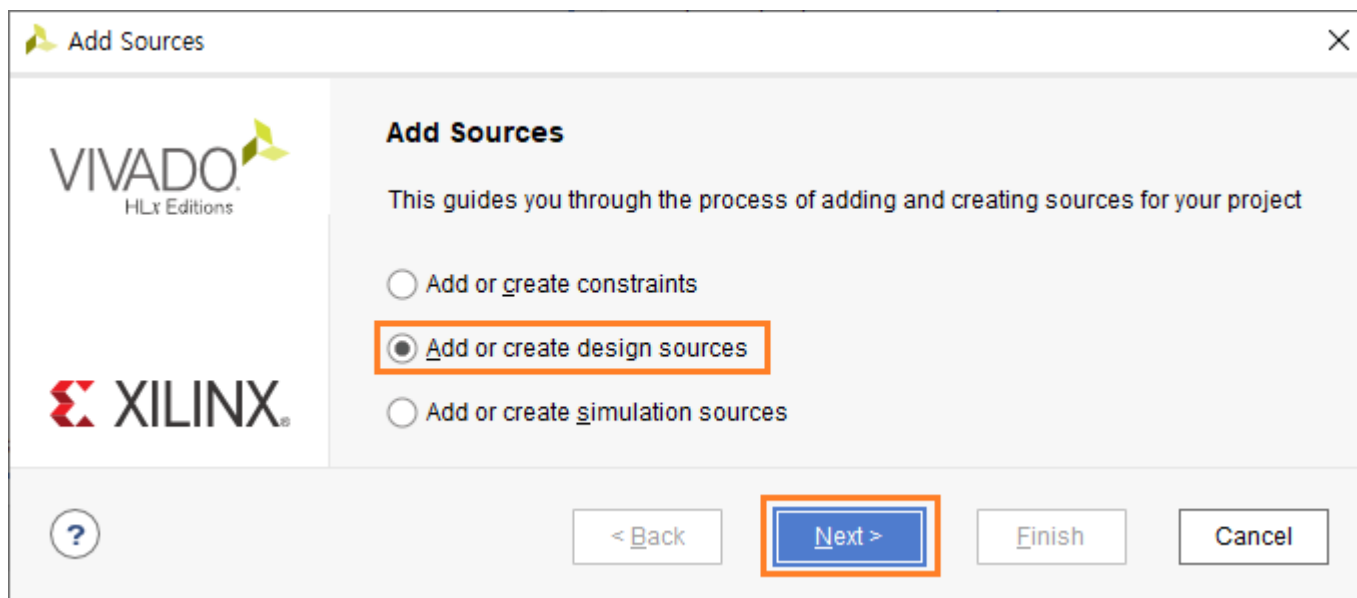
Finish를 클릭하여 프로젝트를 생성합니다.



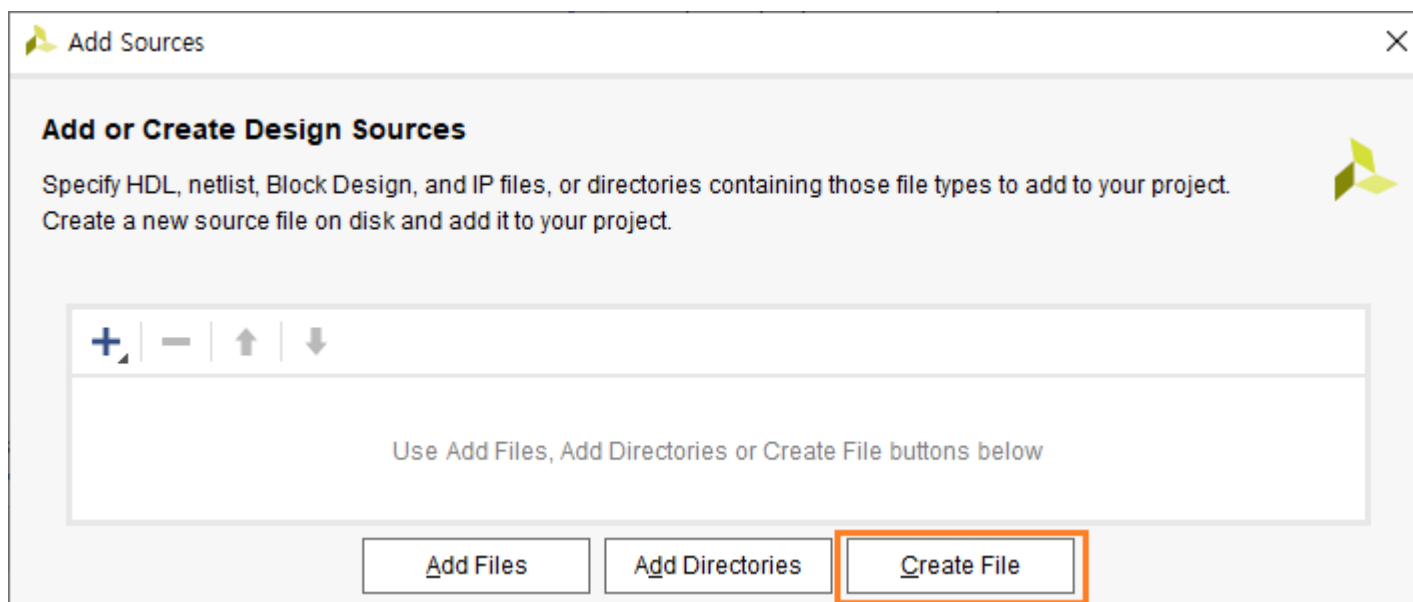
코드를 추가합니다. Sources - Design Sources - 우클릭 - Add Sources... 클릭합니다.



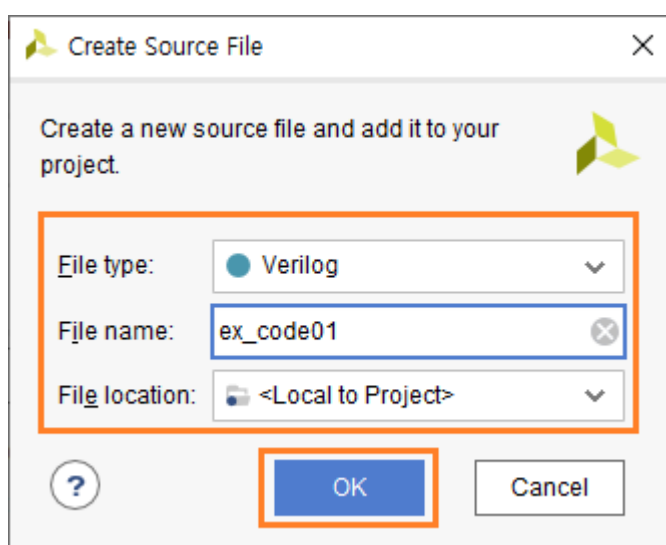
“Add or create design sources”를 선택하고 Next를 클릭합니다.



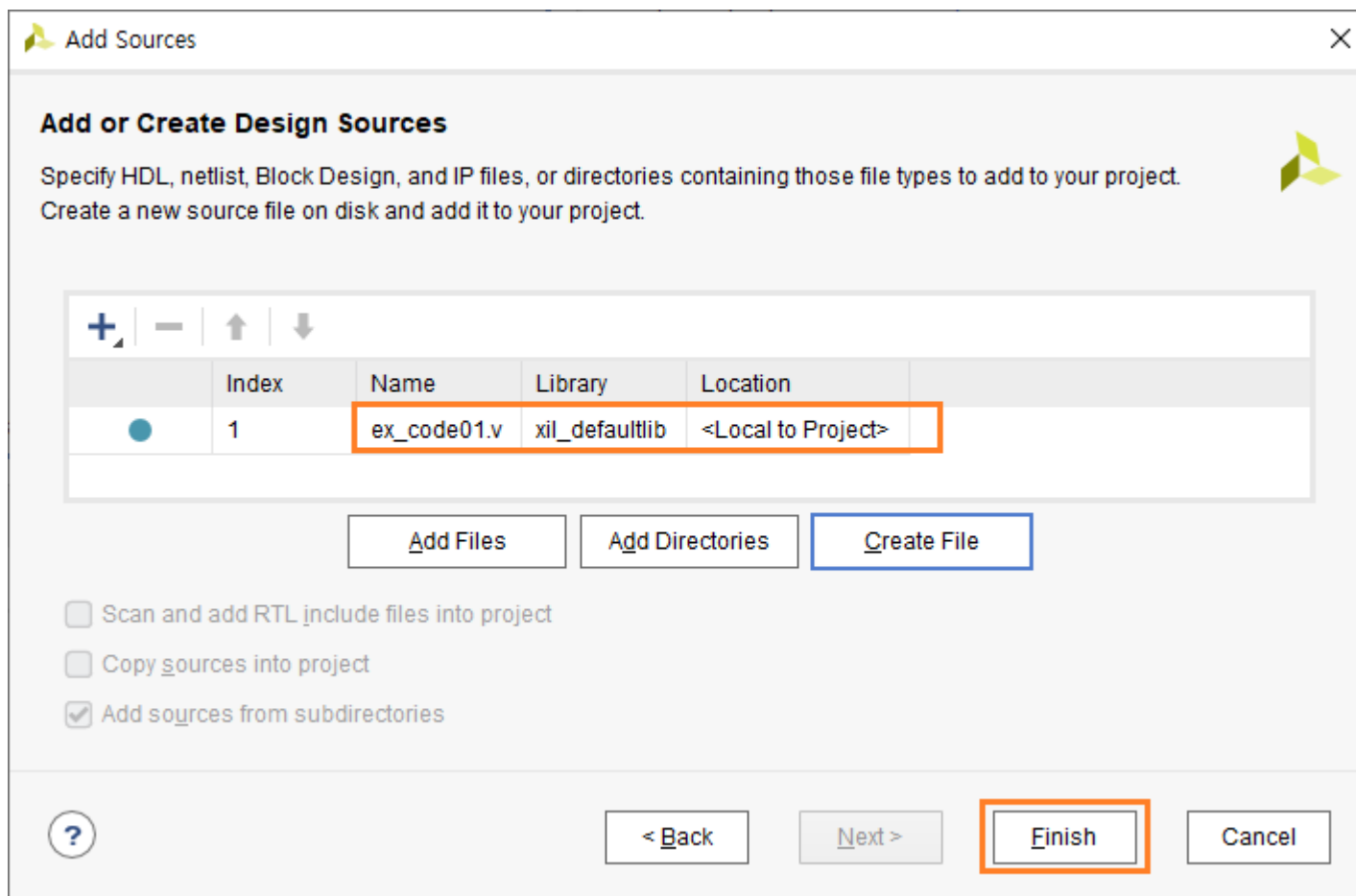
Add Souces 윈도우에서 “Create File”을 클릭해서 새로운 파일을 생성합니다.



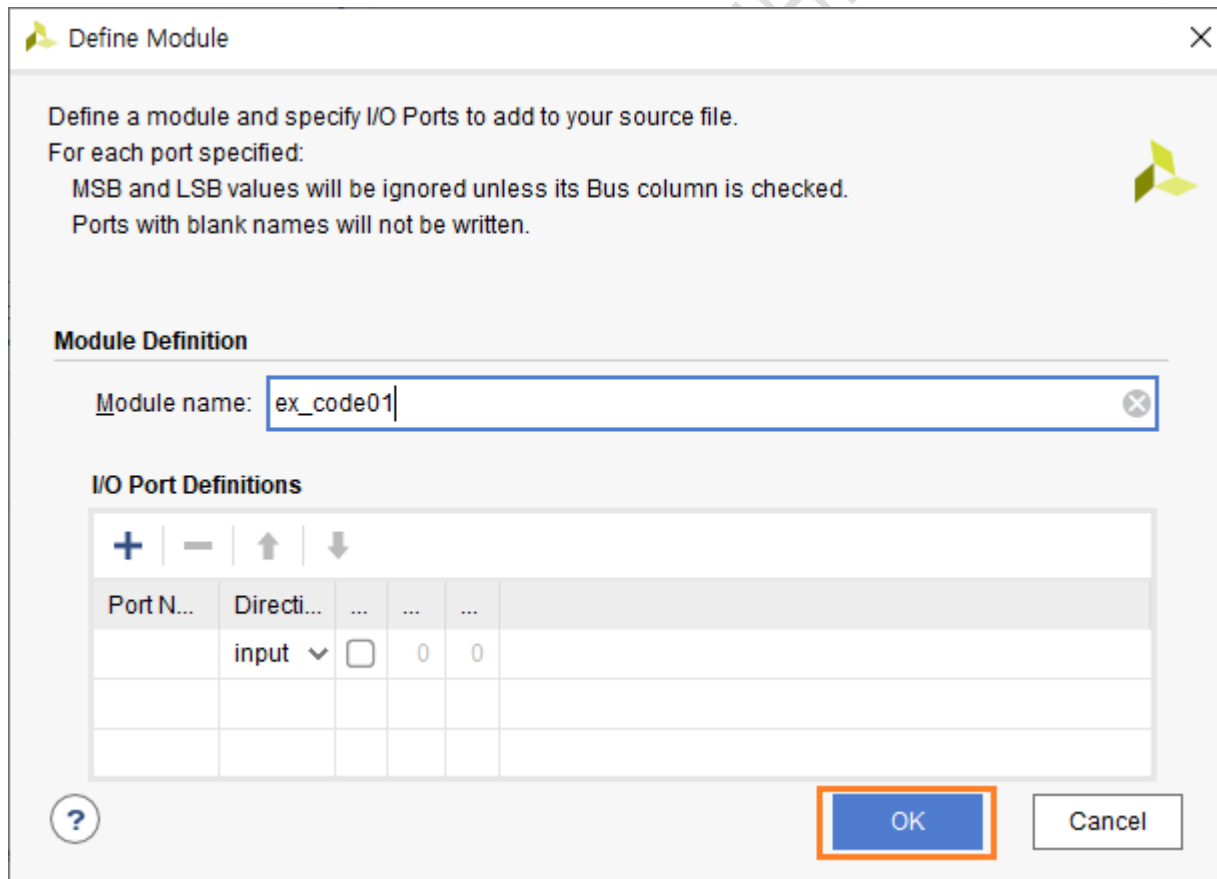
File name에 “ex_code01”로 입력하고 OK를 클릭합니다.



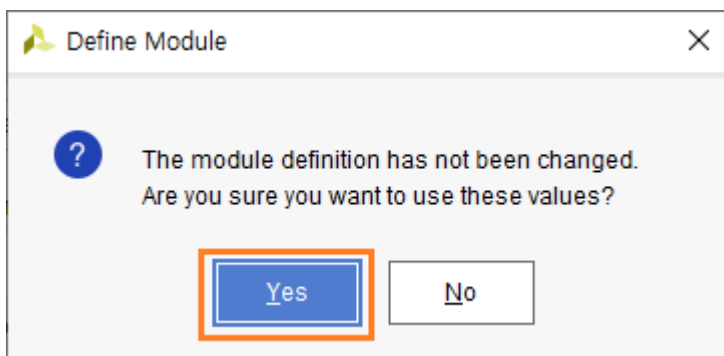
ex_code01.v 파일이 생성됩니다. Finish 클릭합니다.



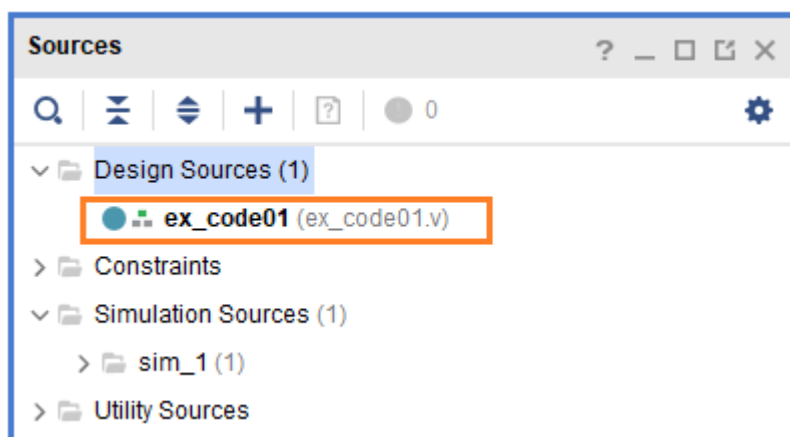
Define Module 윈도우에서는 OK를 클릭합니다.



모듈은 사용자가 직접 코드로 구현합니다. Yes를 클릭합니다.



Design Sources 아래에 “ex_code01” 모듈이 추가되었습니다. ex_code01 왼쪽의 아이콘()은 ex_code01이 Top Module로 지정되었음을 나타냅니다.



ex_cod01을 더블클릭해서 코드를 추가하도록 하겠습니다. 본 강의에서는 코드용 에디터로 UltraEdit를 사용합니다. UltraEdit의 환경설정에서 탭 크기를 8로 설정하면 본 강의에서 제공되는 코드의 탭 정렬을 맞출 수 있습니다.

아래와 같이 코드를 추가합니다.

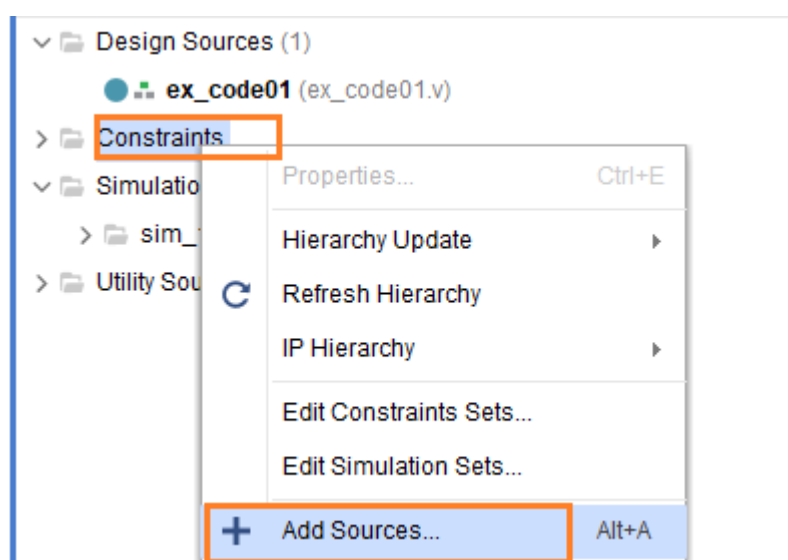
```

23 module ex_code01(
24     reset      ,
25     clock      ,
26     in1        ,
27     in2        ,
28     out1       ,
29     out2       ,
30 );
31 input      reset      ;
32 input      clock      ;
33 input      in1        ;
34 input [3:0] in2        ;
35 output     out1       ;
36 output [3:0] out2      ;
37
38 reg        out1 ;
39 always @(posedge clock or negedge reset)
40 begin
41     if(~reset)    out1 <= 1'b0;
42     else          out1 <= in1 ;
43 end
44
45 reg [3:0] out2 ;
46 always @(posedge clock or negedge reset)
47 begin
48     if(~reset)    out2 <= 4'b0;
49     else          out2 <= in2 ;
50 end
51
52 endmodule

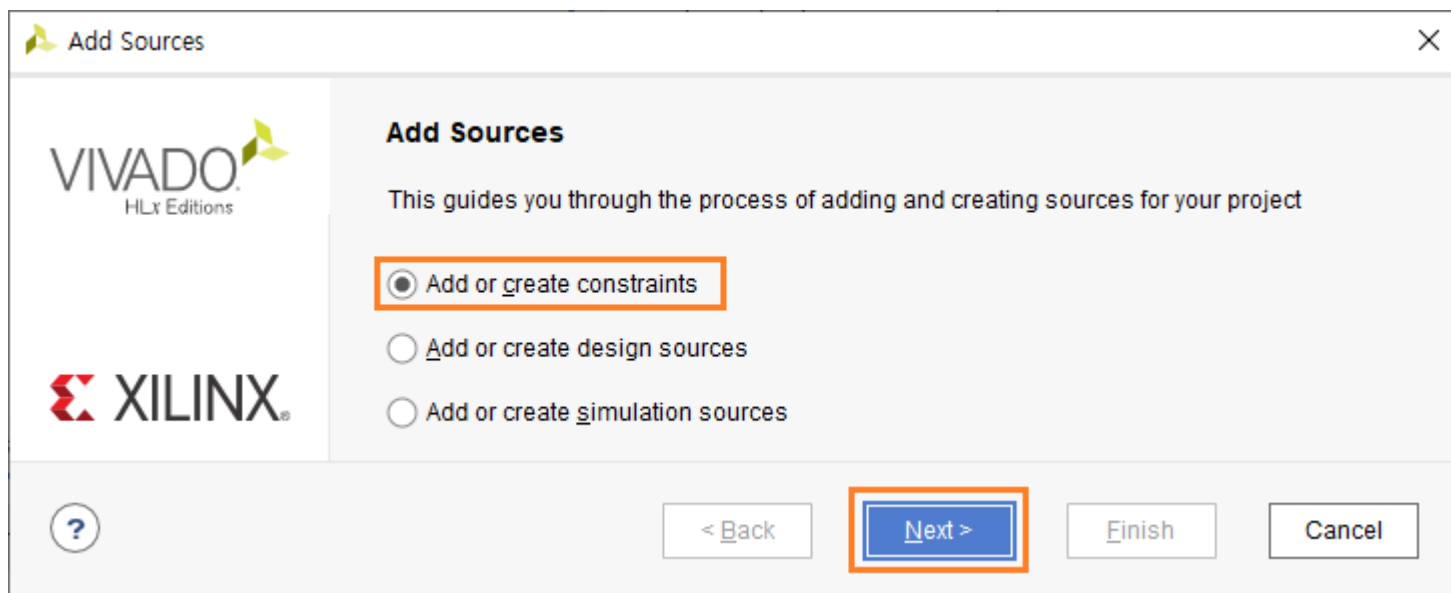
```

- ✓ 라인 23 - 30 : module 정의
- ✓ 라인 31 -36 : in/out port 정의
- ✓ 라인 38 - 43 : 1bit reg, out1 생성, 구현
- ✓ 라인 45 - 50 : 4bits reg, out2 생성, 구현

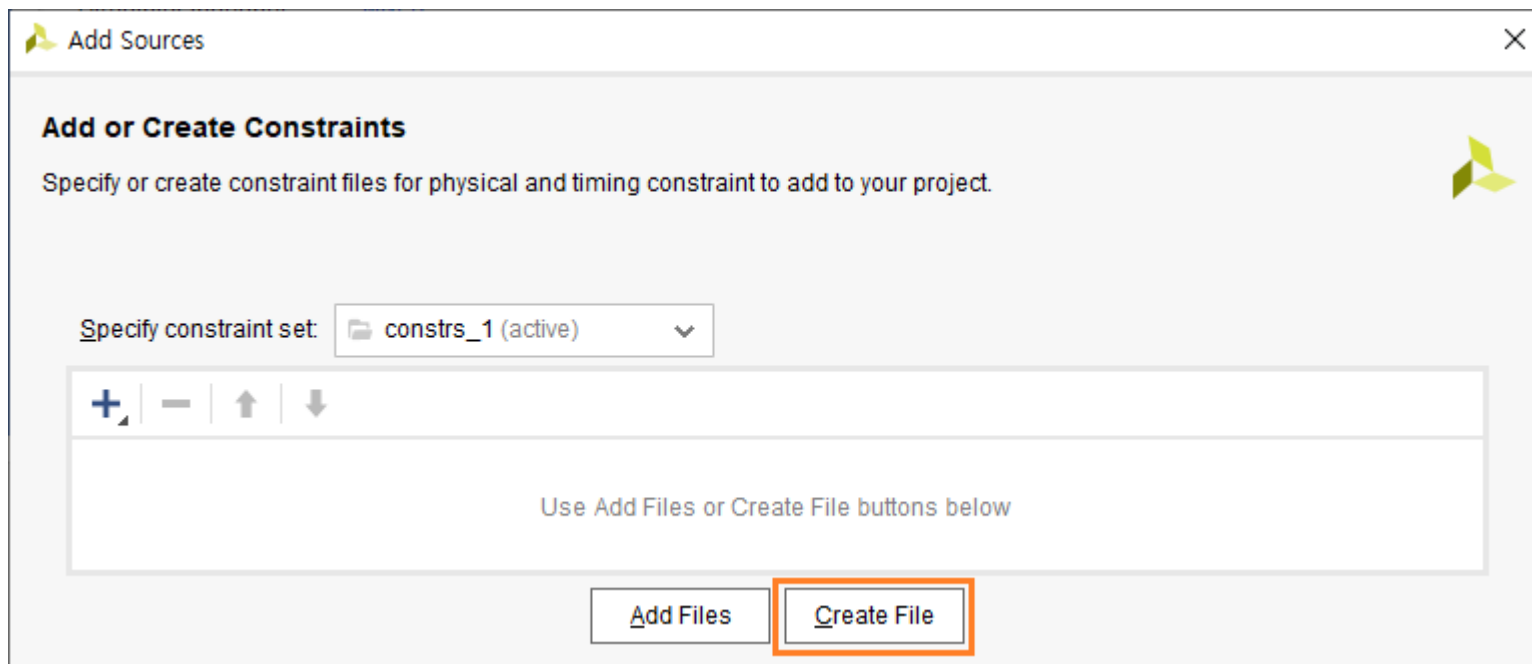
생성된 코드의 회로(Schematic)을 생성하기 위해서는 Constraints(xdc) 파일을 생성해야 합니다. Constraints - 우클릭 - Add Sources... 클릭합니다.



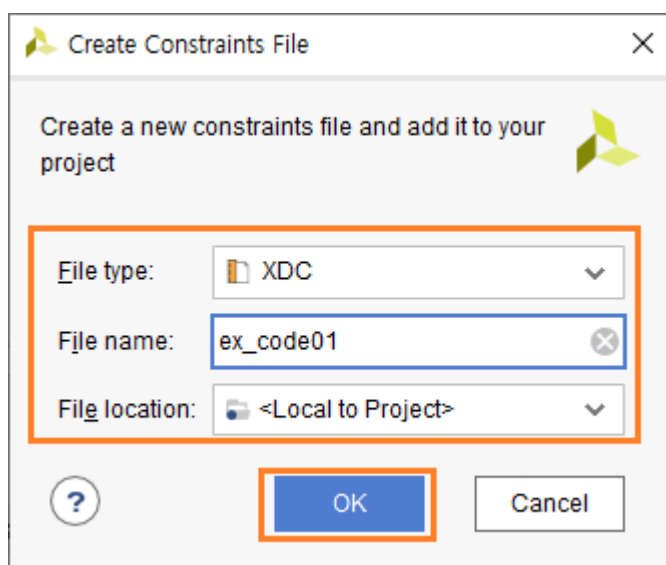
“Add or create constraints”을 선택하고 Next를 클릭합니다.



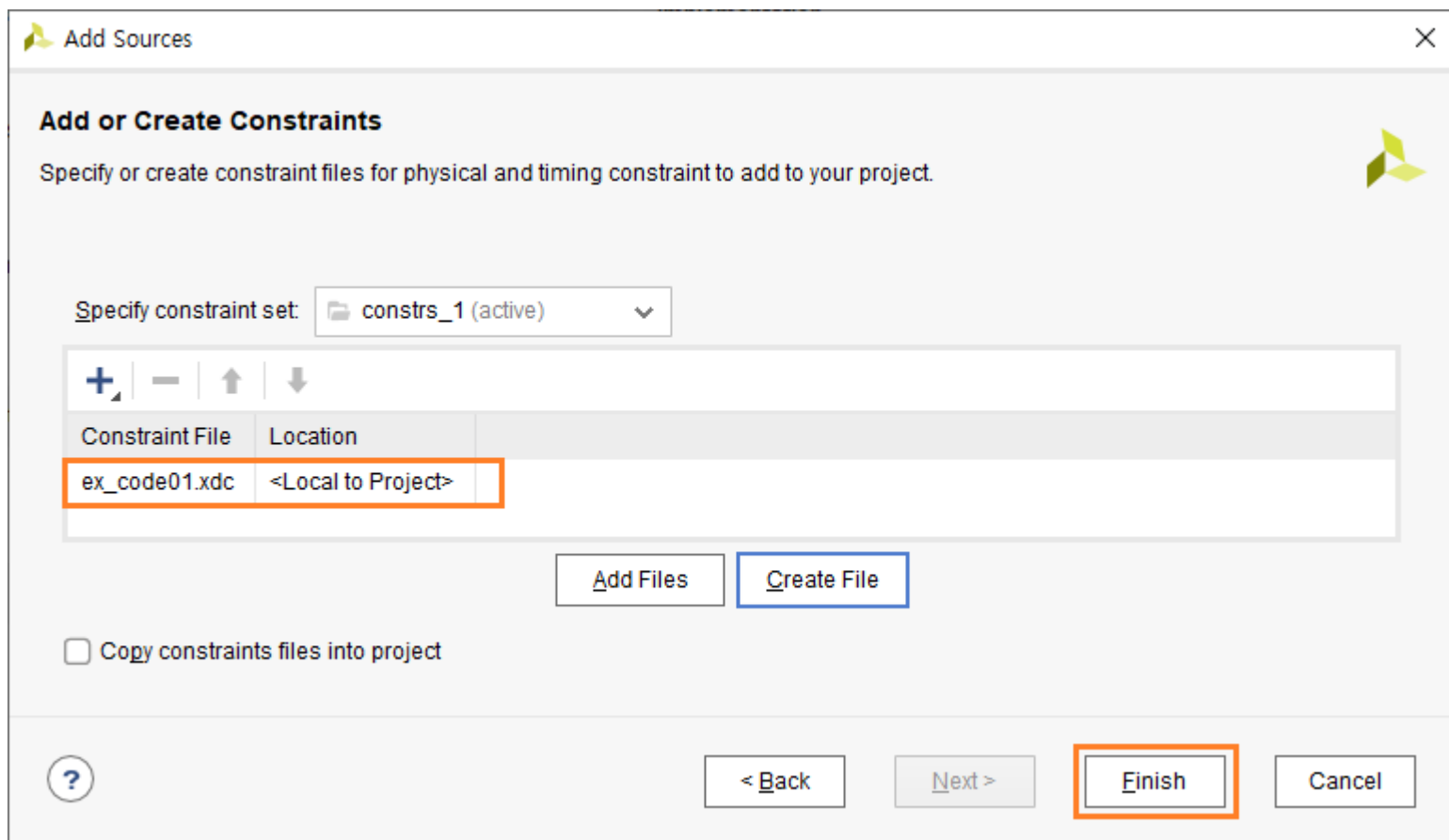
Add Sources 윈도우에서 Create File을 클릭합니다.



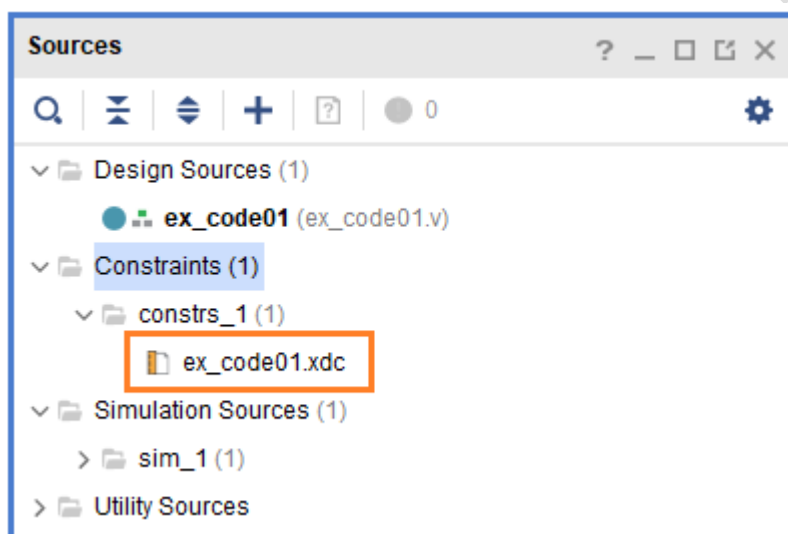
File name : ex_code01을 입력하고 OK를 클릭합니다.



Finish를 클릭합니다.



ex_code01.xdc 파일이 생성되었습니다.



ex_code01.xdc 파일을 아래와 같이 수정합니다.

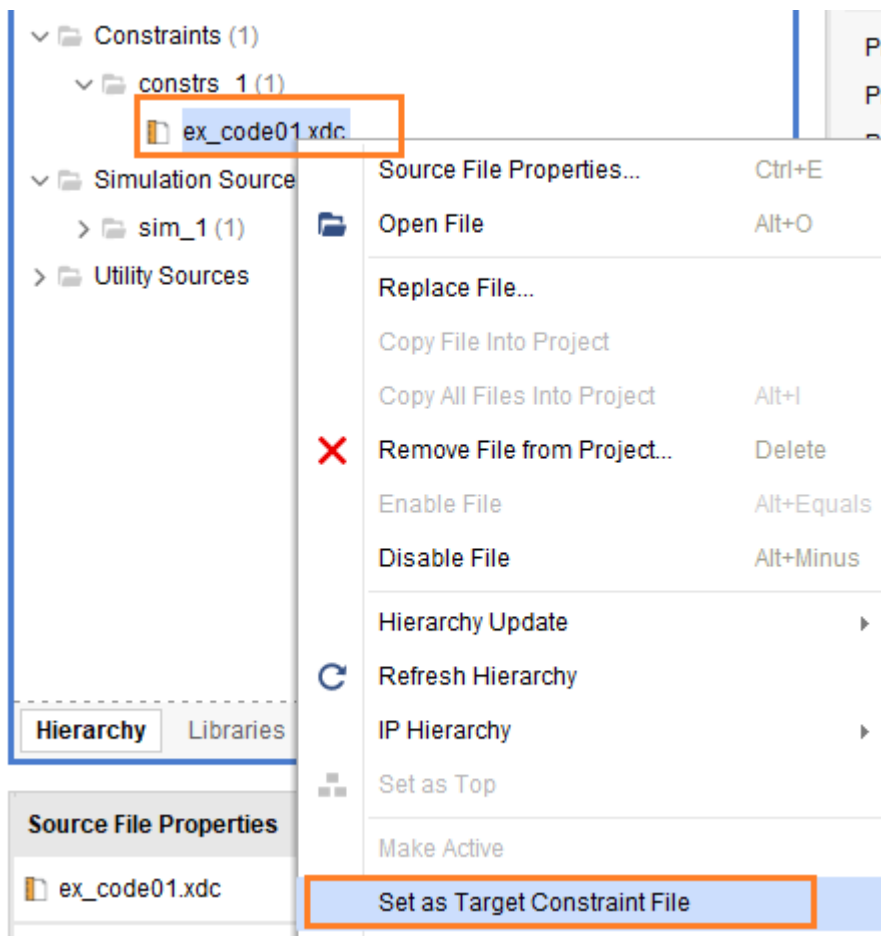
```

1 ## clock
2 set_property -dict { PACKAGE_PIN E3      IOSTANDARD LVCMOS33 } [get_ports { clock }];
3 create_clock -add -name sys_clk_pin -period 10.00 -waveform {0 5} [get_ports { clock }];
4
5 ## reset
6 set_property -dict { PACKAGE_PIN C2      IOSTANDARD LVCMOS33 } [get_ports { reset }];
7
8 ## signal
9 set_property -dict { PACKAGE_PIN E1      IOSTANDARD LVCMOS33 } [get_ports { in1      }];
10 set_property -dict { PACKAGE_PIN G6      IOSTANDARD LVCMOS33 } [get_ports { in2[0]   }];
11 set_property -dict { PACKAGE_PIN F6      IOSTANDARD LVCMOS33 } [get_ports { in2[1]   }];
12 set_property -dict { PACKAGE_PIN G4      IOSTANDARD LVCMOS33 } [get_ports { in2[2]   }];
13 set_property -dict { PACKAGE_PIN G3      IOSTANDARD LVCMOS33 } [get_ports { in2[3]   }];
14 set_property -dict { PACKAGE_PIN J4      IOSTANDARD LVCMOS33 } [get_ports { out1      }];
15 set_property -dict { PACKAGE_PIN H4      IOSTANDARD LVCMOS33 } [get_ports { out2[0]   }];
16 set_property -dict { PACKAGE_PIN J3      IOSTANDARD LVCMOS33 } [get_ports { out2[1]   }];
17 set_property -dict { PACKAGE_PIN J2      IOSTANDARD LVCMOS33 } [get_ports { out2[2]   }];
18 set_property -dict { PACKAGE_PIN K2      IOSTANDARD LVCMOS33 } [get_ports { out2[3]   }];
19
20
21 #####
22 # MISC
23 #####
24 set_operating_conditions -ambient_temp 80.0
25
26 set_property CONFIG_MODE SPIx1 [current_design]
27 set_property CFGBVS VCCO [current_design]
28 set_property CONFIG_VOLTAGE 3.3 [current_design]
29
30 set_property BITSTREAM.CONFIG.SPI_BUSWIDTH 1 [current_design]
31 set_property BITSTREAM.CONFIG.PERSIST NO [current_design]
32 set_property BITSTREAM.CONFIG.SPI_FALL_EDGE NO [current_design]
33 set_property BITSTREAM.CONFIG.CONFIGRATE 22 [current_design]

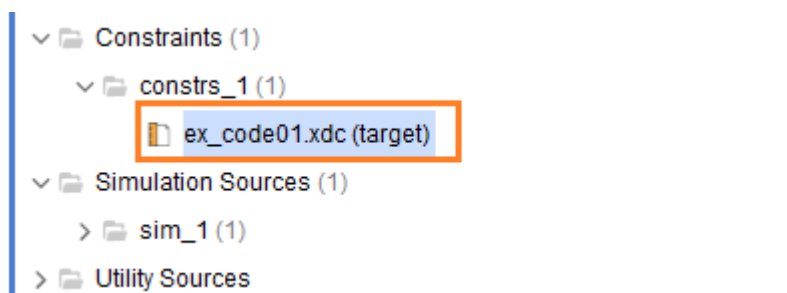
```

- ✓ 라인 1-3 : clock 설정
- ✓ 라인 5-6 : reset 설정
- ✓ 라인 8-18 : in/out port 설정, test를 위한 것으로 임의의 핀으로 할당함.
- ✓ 라인 21 - 33 : bitstream 생성 옵션

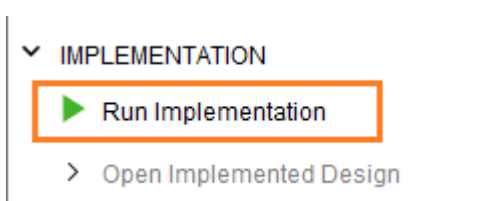
추가된 xdc 파일을 Target Constraints file로 지정합니다. (xdc 파일이 여러개 일 때에 어떤 xdc 파일을 사용할지를 지정합니다) ex_cod01.xdc - 우클릭 - Set as Target Constraints File 클릭합니다.



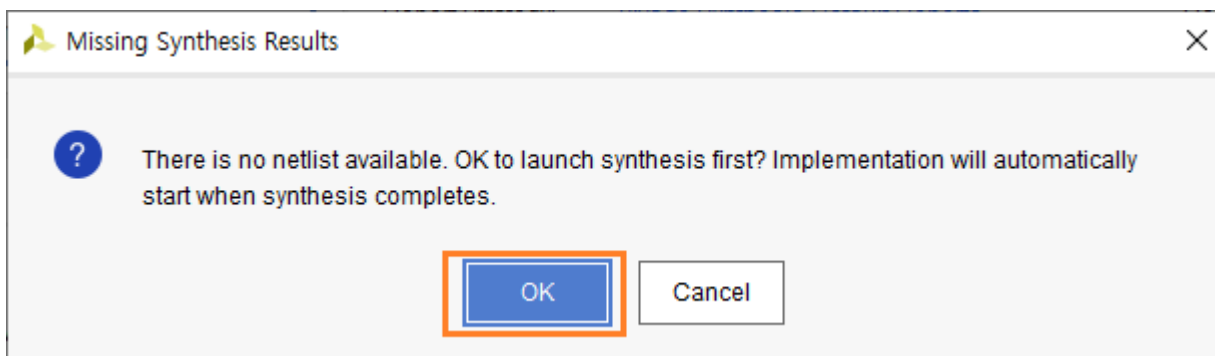
ex_cod01.xdc 파일 우측에 “target”이 추가되었습니다.



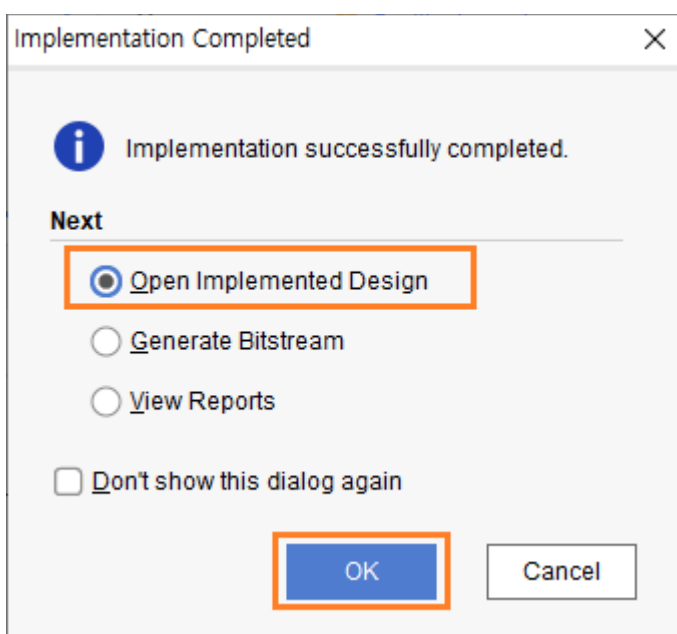
이제 모든 준비가 완료되었습니다. Schematic 을 생성하기 위하여 Synthesis - Implementation을 진행합니다. Flow Navigator 탭에서 IMPLEMENTATION - Run Implementation 을 클릭합니다.



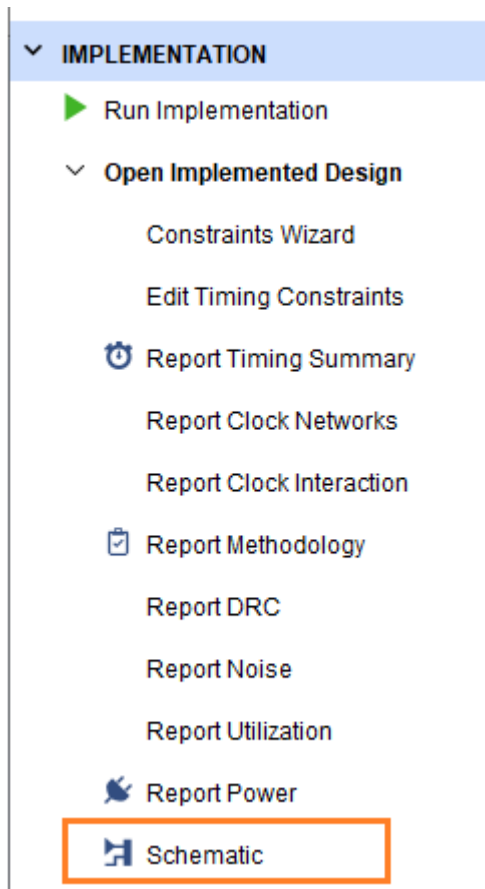
OK를 클릭하면, Synthesis - Implementation을 진행합니다.



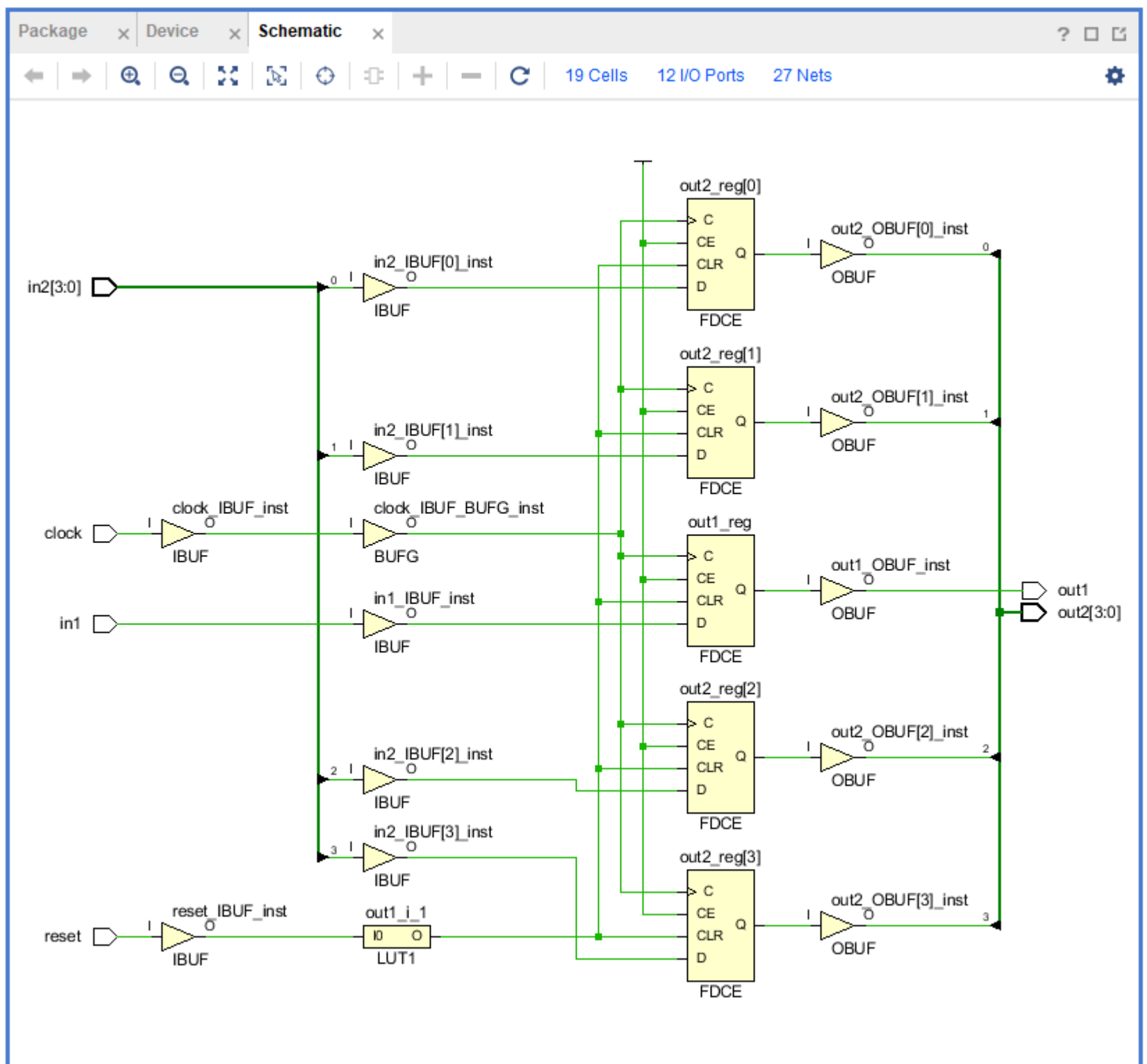
Synthesis, Implementation 완료 되면, Open Implemented Design을 선택하고 OK를 클릭합니다.



Schematic을 확인하기 위하여 Flow Navigator 탭에서 IMPLEMENTATION - Open Implemented Design - Schematic 을 클릭합니다.



아래 그림은 schematic을 보여줍니다.



in1, in2의 input port에는 IBUF가 사용되고, out1, out2 output port에는 OBUF가 사용되었습니다. clock 에는 BUFG 가 사용되었습니다. BUFG는 Global clock에 사용됩니다. reset 핀은 IBUF와 LUT1이 사용되었습니다. LUT1은 inverting을 구현합니다. 그리고 총 5개의 D-Flip/Flop이 사용되었습니다. D-Flip/Flop의 CE(Chip Enable, Active High)핀은 내부적으로 H로 연결되었고, clock(C)과 data(D), output(Q)가 각각 연결되었습니다.

2.2 simulation

vivado에서는 총 5가지의 simulation을 지원합니다.

- ✓ Behavioral Simulation
- ✓ Post-Synthesis Functional Simulation
- ✓ Post-Synthesis Timing Simulation
- ✓ Post-Implementation Functional Simulation
- ✓ Post-Implementation Timing Simulation

크게 보면 2가지 (Function simulation, Timing simulation)로 나누어 볼 수 있습니다.

1) Behavioral Simulation : Function simulation

verilog 코드가 로직적으로 오류 없이 동작하는지를 확인하기 위하여 사용됩니다. 즉 소자의 타이밍, 딜레이 정보(Transition delay, setup-time, hold-time)를 무시하고, 이상적인 조건(Transition delay=0, setup-time=0, hold-time=0)에서 simulation을 진행합니다. 대부분 Behavioral simulation을 많이 사용합니다.

2) Post-Synthesis Functional Simulation : Function simulation

Synthesis (verilog 코드를 netlist 변환)후에 진행되는 Function simulation 입니다. Behavioral sim. 과 차이점은 synthesis 과정에서 추가되는 netlist의 신호들을 확인해 볼 수 있습니다. 그러나 타이밍 정보가 없기 때문에 Behavioral sim. 과 크게 다르지 않습니다.

3) Post-Synthesis Timing Simulation : Timing simulation

Synthesis 후에 진행되는, 타이밍, 딜레이 정보가 추가된 simulation 입니다. 각 소자의 transition delay가 적용되어 simulation이 진행됩니다. Timing violation이 발생할 때, 해당 부분의 simulation을 통해서 원인을 찾고 코드를 수정할 수 있습니다.

4) Post-Implementation Functional Simulation : Function simulation

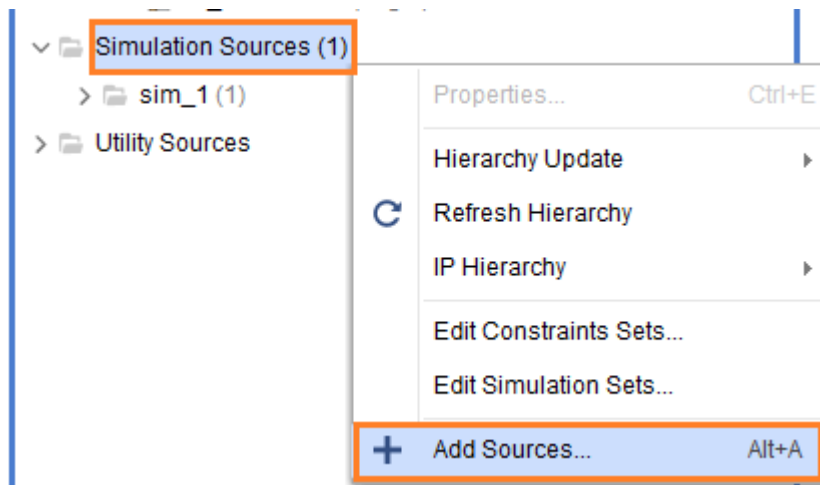
Implementation (verilog 코드를 netlist 변환하고 P&R까지 진행, P&R : Place & Route)후에 진행되는 Function simulation 입니다. 타이밍 정보가 없기 때문에 Post-Synthesis Function sim. 과 거의 동일합니다.

5) Post-Implementation Timing Simulation : Timing simulation

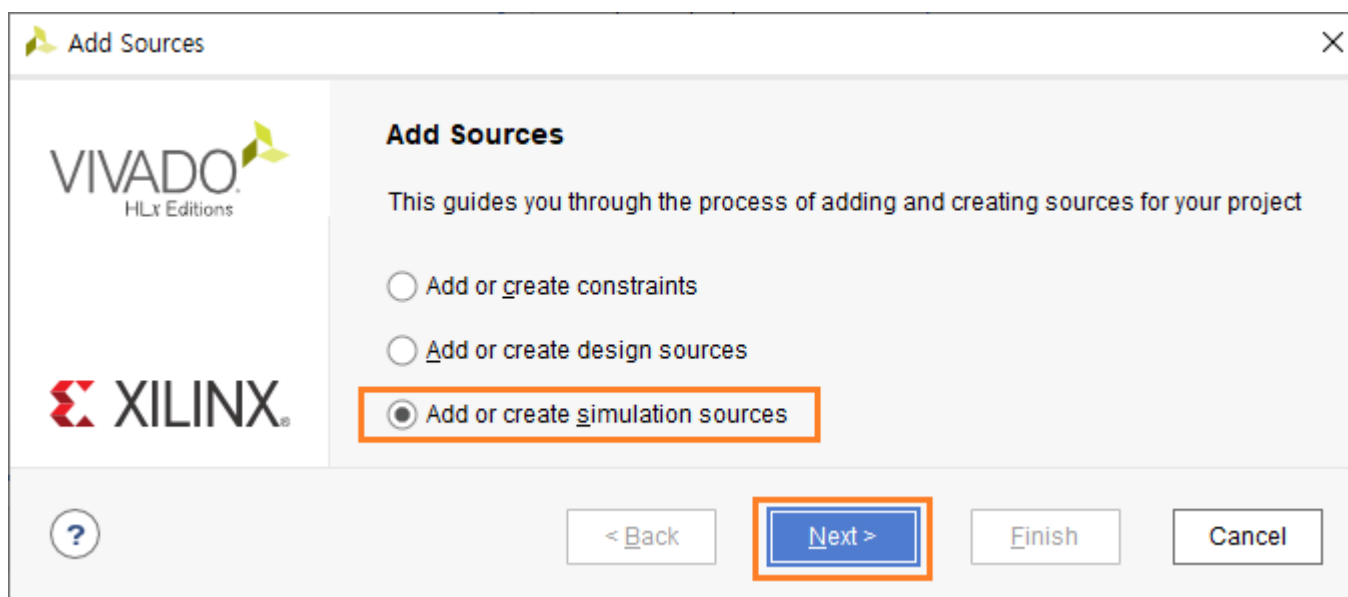
Synthesis, Implementation 후에 진행되는, 타이밍, 딜레이 정보가 추가된 simulation 입니다. 각 소자의 transition delay 와 Routing으로 인한 Path delay까지 적용되어 simulation이 진행됩니다. Timing violation이 발생할 때, 해당 부분의 simulation을 통해서 원인을 찾고 코드를 수정할 수 있습니다. ASIC 진행시 필수로 진행해야 합니다.

5가지의 simulation을 통하여 그 차이점을 확인해 보도록 하겠습니다.

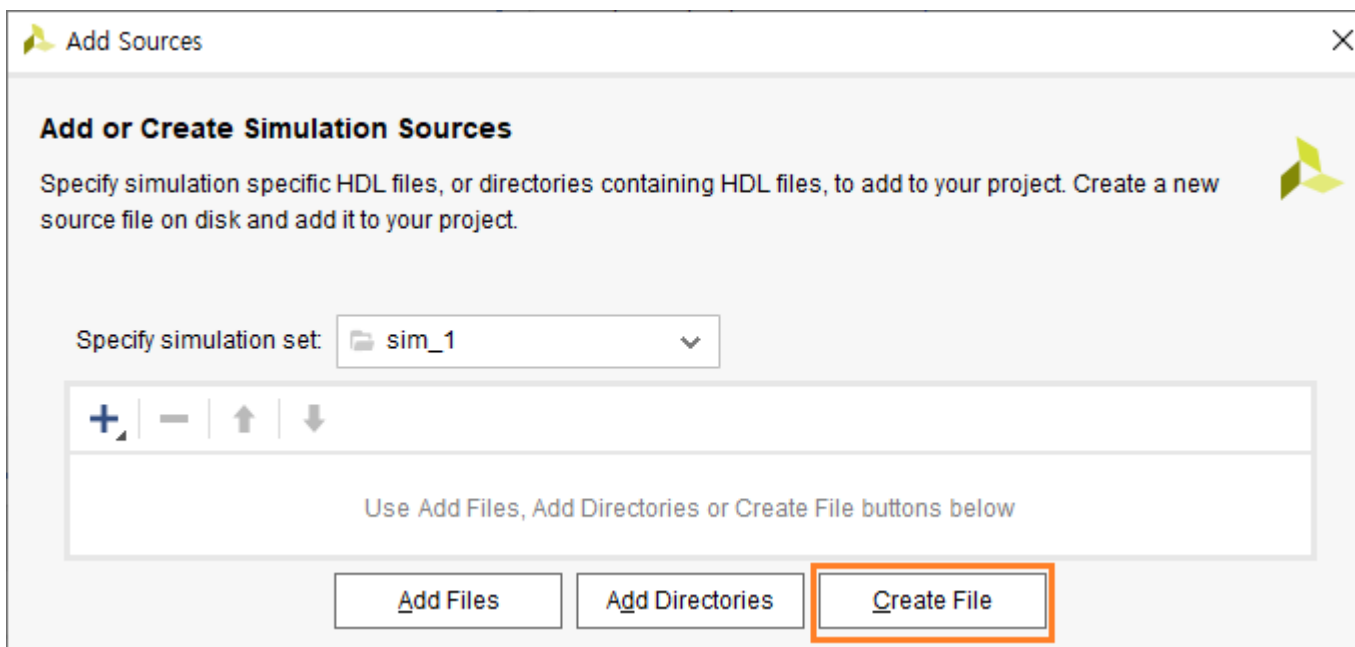
Test bench를 추가합니다. Sources 탭에서 Simulation sources - 우클릭 - Add Sources...를 클릭합니다.



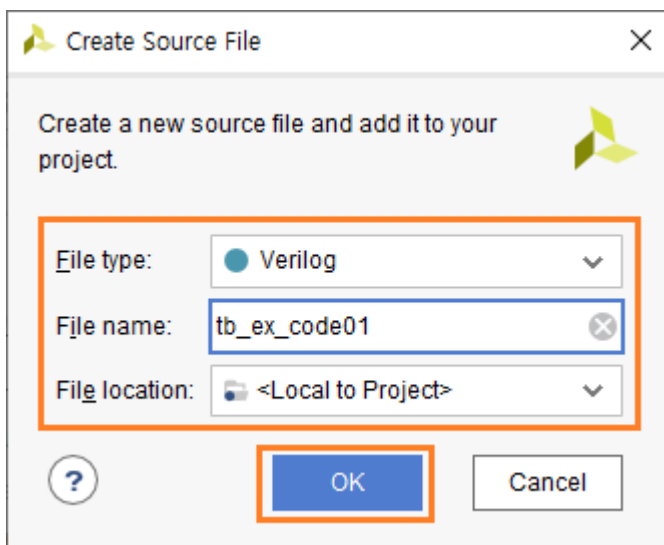
Add or create simulation sources 를 선택하고 Next를 클릭합니다.



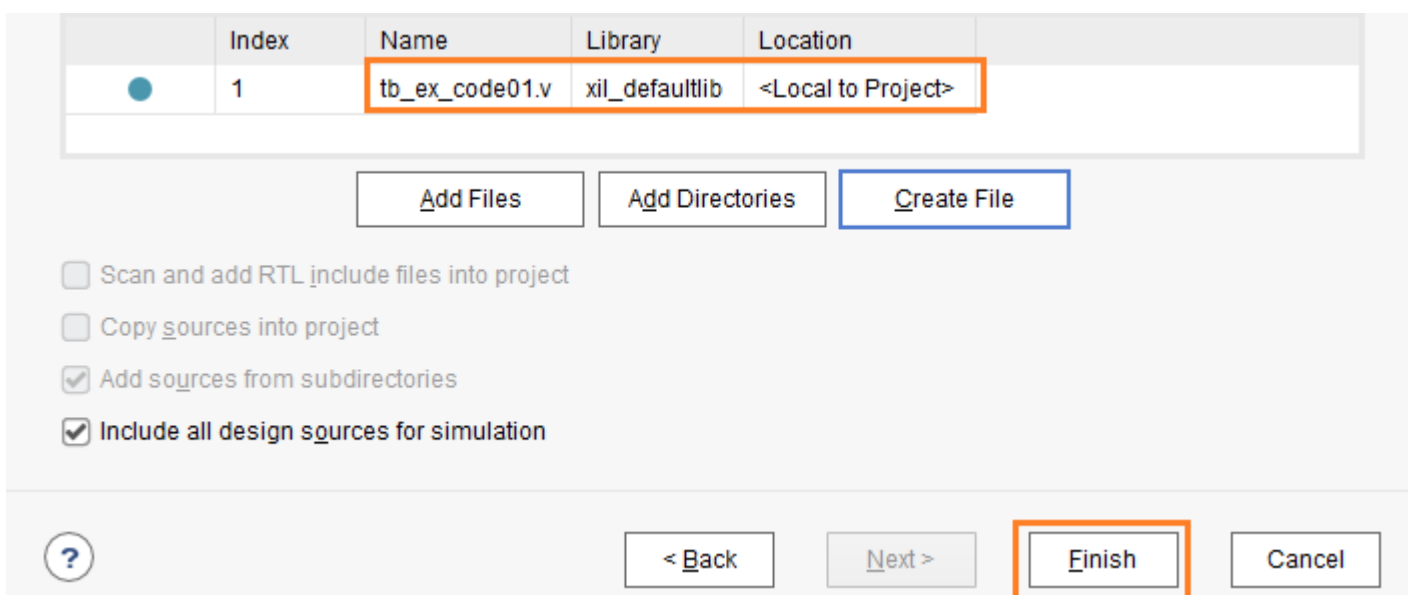
Add Sources 윈도우에서 Create File을 클릭합니다.



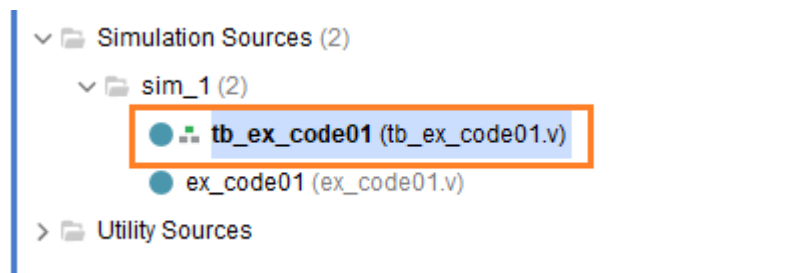
File name에 tb_ex_code01을 입력하고 OK를 클릭합니다.



Finish를 클릭해서 tb_ex_code01.v를 생성합니다.



Define Module 윈도우에서 OK를 클릭하면, tb_ex_code01.v 가 생성됩니다. tb_ex_code01.v - 우클릭 - Set as Top를 클릭해서 Top Module로 지정합니다.



tb_ex_code01 모듈을 더블클릭해서 tb_ex_code01.v 파일에 아래 내용을 추가합니다.

```

1  `timescale 1ns / 1ps

23 module tb_ex_code01();
24
25     reg            reset, clock;
26     reg            in1;
27     reg [3:0]      in2;
28     initial        begin
29         reset = 1'b0;
30         clock = 1'b0;
31         in1   = 1'b0;
32         in2   = 4'b0;
33
34         #1205 reset = 1'b1;
35
36         #233  in1 = 1'b1;
37         #133  in2 = 4'd10;
38
39         #533  reset = 1'b0;
40         #102  reset = 1'b1;
41
42         #202  in1 = 1'b0;
43         in2 = 4'd0;
44
45         #303  in1 = 1'b1;
46         in2 = 4'd15;
47     end
48
49     always #5      clock = ~clock;          // 100 Mhz
50
51     wire          out1;
52     wire [3:0]     out2;
53     ex_code01 ex_code01(
54         .reset      (reset),
55         .clock       (clock),
56         .in1         (in1),
57         .in2         (in2),
58         .out1        (out1),
59         .out2        (out2)
60     );
61
62 endmodule

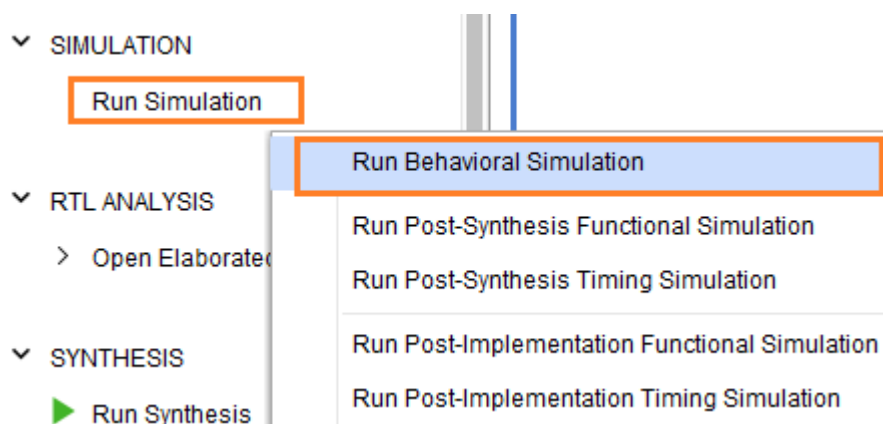
```

- ✓ 라인 1 : timescale 설정, simulation 단위는 1ps, time 단위는 1ns (#1205 : 1205ns를 의미함)
- ✓ 라인 23 : tb_ex_cod01 모듈 define
- ✓ 라인 25-27 : ex_code01 모듈의 input port로 입력될 신호를 생성

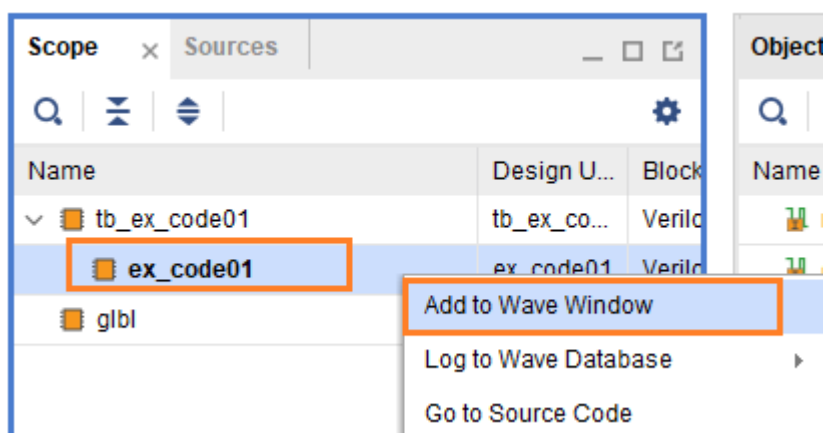
- ✓ 29 - 32 : time 0 에서 초기값
- ✓ 라인 34 : 1205ns 후에 reset = 1로 변경
- ✓ 라인 36 - 46 : test를 위한 신호 생성
- ✓ 라인 49 : clock 생성, 100Mhz
- ✓ 라인 51 - 60 : ex_code01 모듈 추가

1) Behavioral sim.

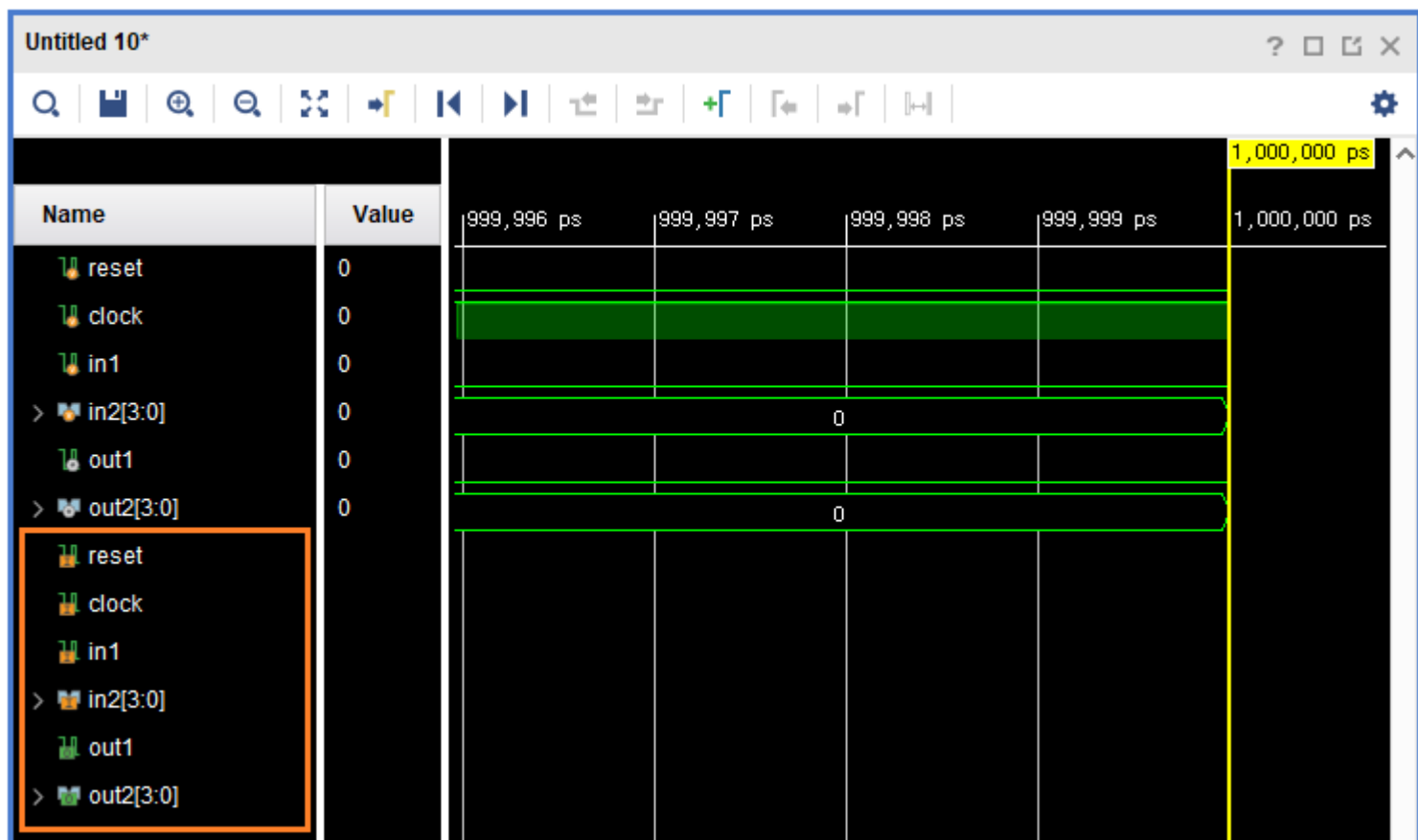
SIMULATION - Run Simulation 클릭 - Run Behavioral Simulation 클릭합니다.



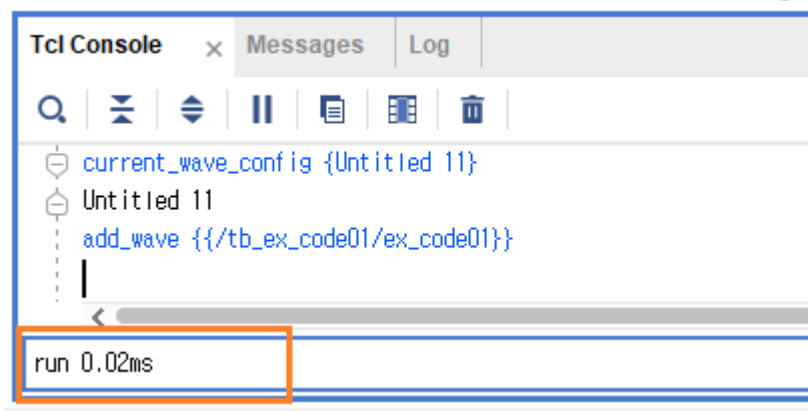
Wave 윈도우의 Scope 탭에서 ex_code01 - 우클릭 - Add to Wave Window 클릭합니다. Wave 윈도우에 ex_code01 모듈의 모든 신호들이 추가됩니다.



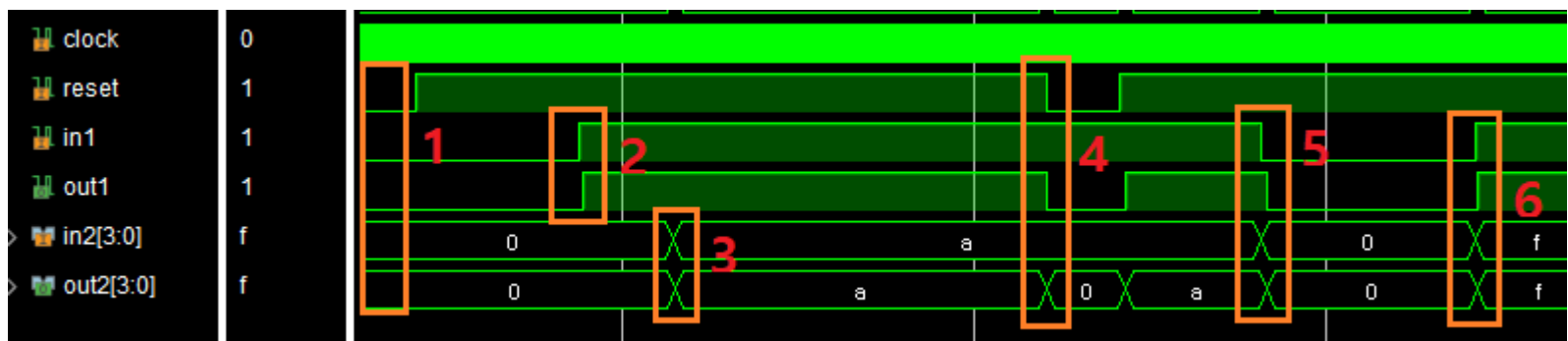
reset ~ out2[3:0] 신호가 추가되었습니다.



하단의 Tcl Console 창에 “run 0.02ms”을 입력하고 엔터를 클릭하면 0.02ms 동안 simulation이 진행됩니다.

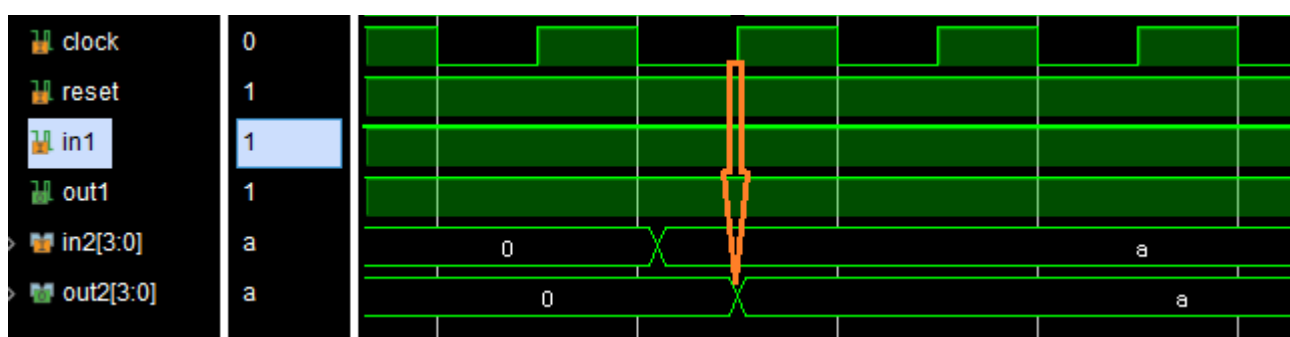
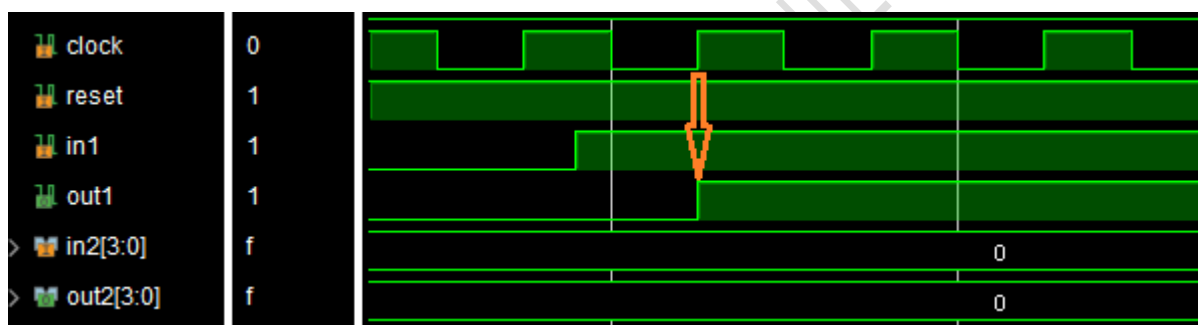


아래 그림은 전체적인 파형을 보여줍니다.

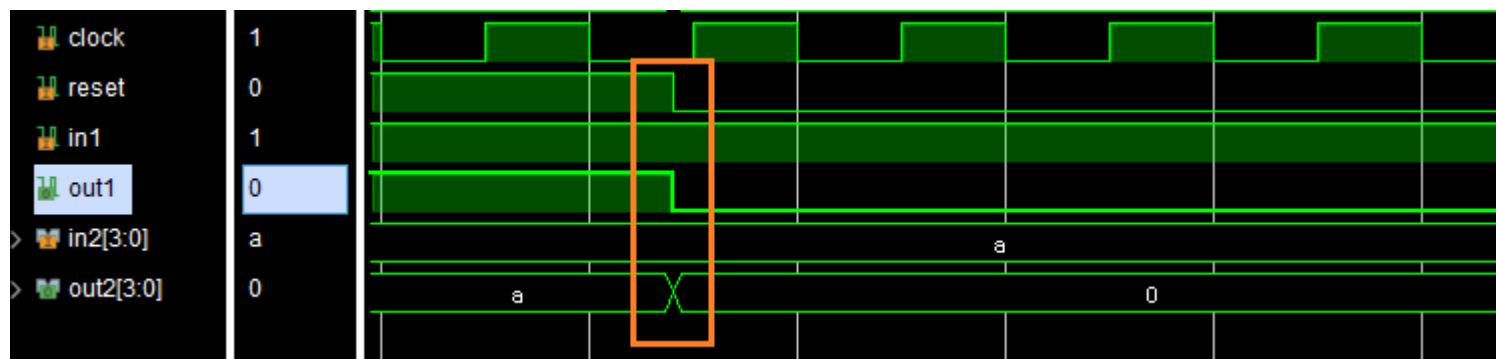


- ✓ 구간-1 : reset=0 일 때, out1 = 0, out2 = 0 가 됩니다.
- ✓ 구간-2 : reset=1 일 때, clock의 positive edge에서 out1 = in1 이 됩니다.
- ✓ 구간-3 : reset=1 일 때, clock의 positive edge에서 out2 = in2 가 됩니다.
- ✓ 구간-4 : reset=0 일 때, out1 = 0, out2 = 0 가 됩니다.
- ✓ 구간-5, 6 : reset=1 일 때, clock의 positive edge에서 out1 = in1, out2 = in2 가 됩니다.

구간-2, 3을 확대해서 보겠습니다. in1, in2가 변경될 때, 바로 out1, out2의 값이 변경되지 않고, 그 다음 clock의 rising edge 에서 변경됨을 알 수 있습니다.



아래 그림은 구간-4를 확대한 그림입니다. reset이 Low가 되면, clock에 상관없이 out1, out2의 값이 바로 0로 되는 것을 알 수 있습니다.

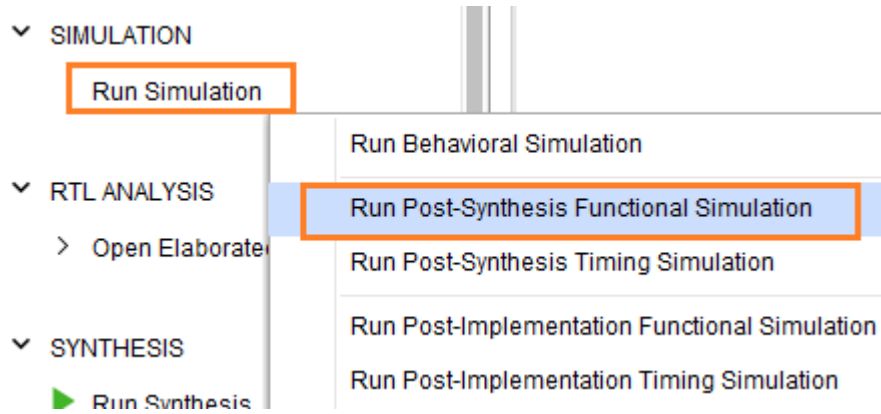


Behavioral simulation은 타이밍, 딜레이 정보가 포함되지 않았기 때문에 clock에 정확하게 동기 되어 신호들이 변경됩니다. 또한 코드상에 있는 신호들만 볼 수 있습니다.

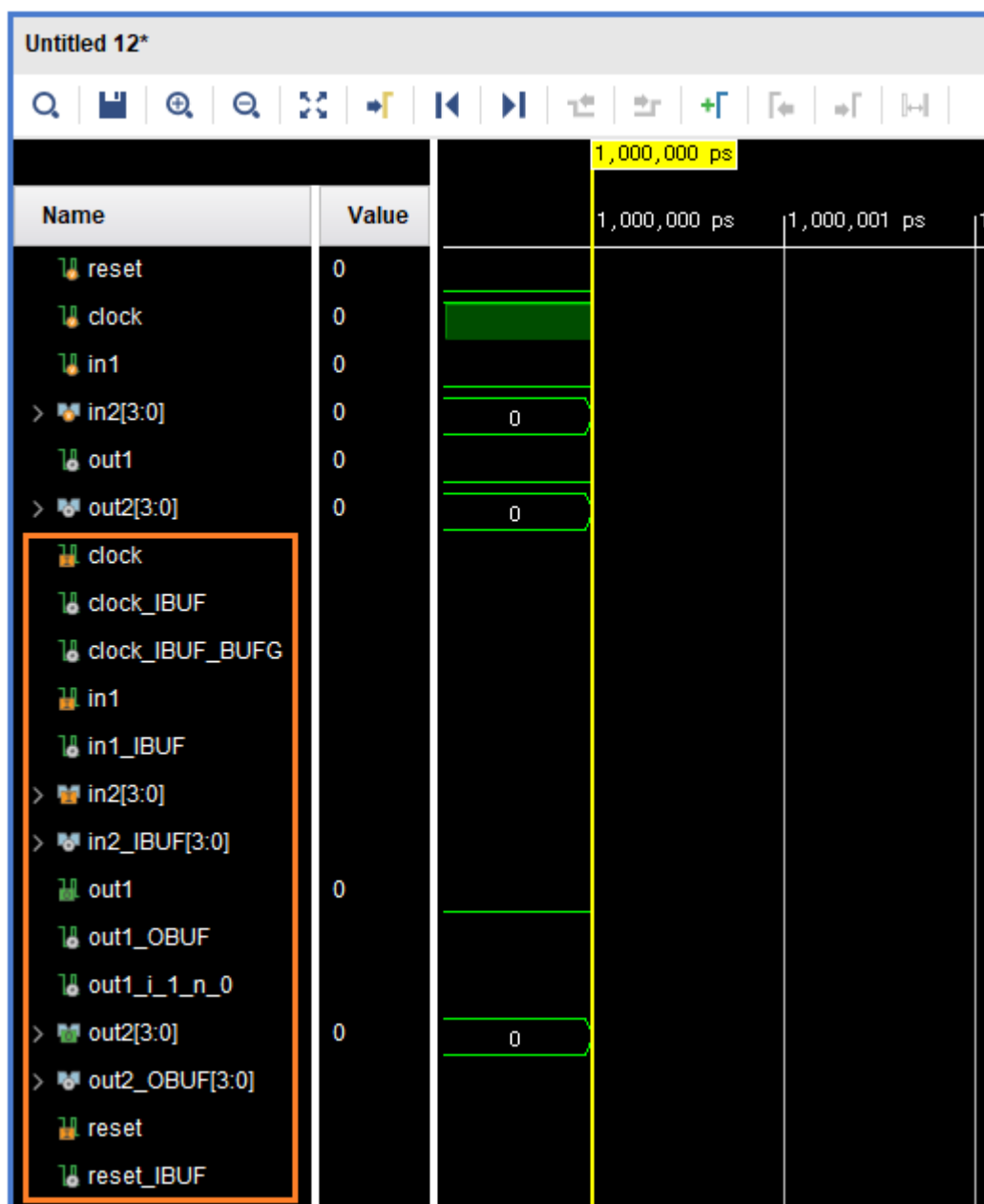
- Synchronous, Asynchronous Signal : Synchronous 신호는 Clock에 동기가 되어 변하는 신호를 말합니다. 위에서 reset=1 일 때, out1, out2신호는 clock의 rising edge 에서 변하기 때문에 Synchronous 신호라고 할 수 있습니다. 그러나 reset=0가 되면 out1, out2신호는 clock에 무관하게 바로 “0”가 됩니다. reset=0일 때에는 out1, out2는 Asynchronous 신호라고 말할 수 있지만, 이는 reset 신호의 특성이라고 보면 됩니다. 따라서 out1, out2는 Synchronous 신호로 보는게 좋습니다.
- 대표적인 Asynchronous 신호는 UART 신호입니다. UART신호는 데이터만 전송되고 Clock은 전송되지 않습니다. SPI, I2C는 Synchronous 신호입니다. 모두 data와 clock이 동시에 전송되기 때문입니다.

2) Post-Synthesis Functional Simulation

SIMULATION - Run Simulation 클릭 - Run Post-Synthesis Functional Simulation 클릭합니다.

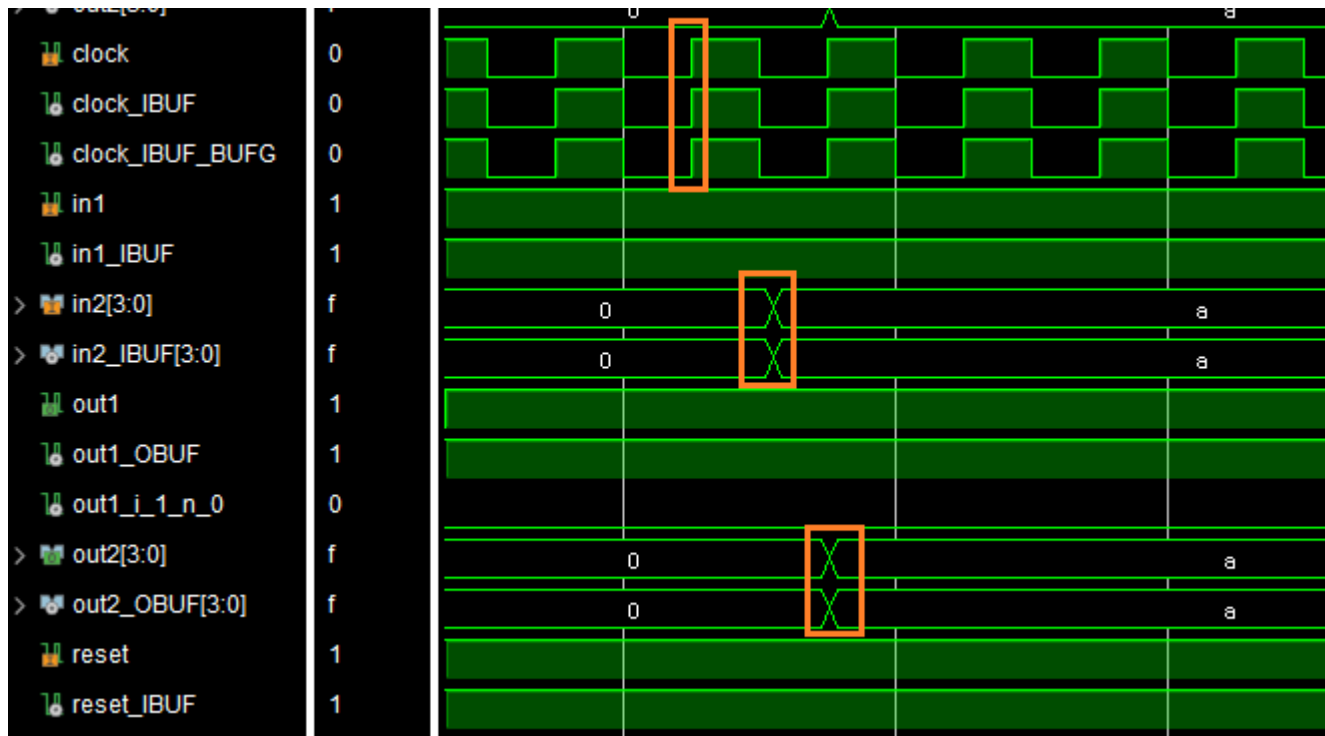


ex_code01 - 우클릭 - Add to Wave Window 클릭합니다. wave 윈도우에 많은 신호들이 추가되었습니다. ex_code01 모듈에 없던 신호들도 보입니다. 사실 Synthesis를 진행하면 verilog 코드들은 netlist로 변환됩니다. 추가된 신호들은 netlist로 변환되면서 추가된 신호들입니다. 이 신호들은 schematic 에서 보았던 신호들과 동일합니다.



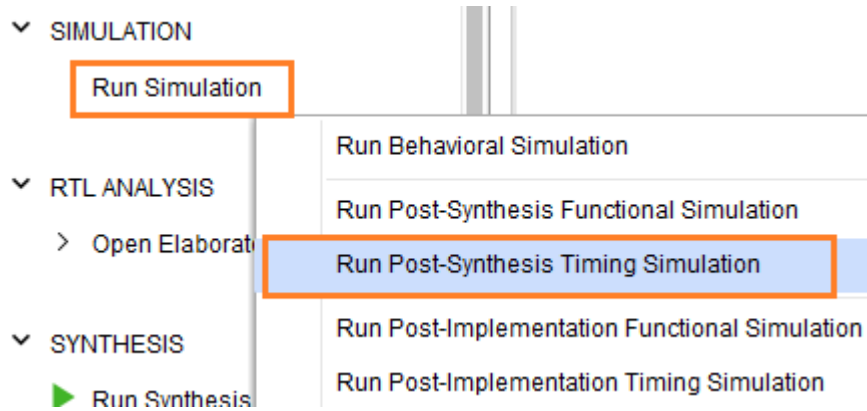
하단의 Tcl Console 창에 “run 0.02ms”을 입력하고 엔터키를 눌러서 0.02ms 동안 simulation을 진행합니다.

아래 그림은 파형을 보여줍니다. 일부분을 확대하였는데, clock, clock_IBUF, clock_IBUF_BUFG, 3개의 신호가 모두 delay 없이 동일한 파형입니다. in2, in2_IBUF와 out2, out2_OBUF 도 delay 없이 동일한 파형임을 알 수 있습니다. 따라서 post-synthesis functional sim, post-implemented functional sim은 behavioral sim 과 거의 동일합니다. 특별히 사용할 일이 거의 없습니다. behavioral sim 으로도 충분하기 때문입니다.



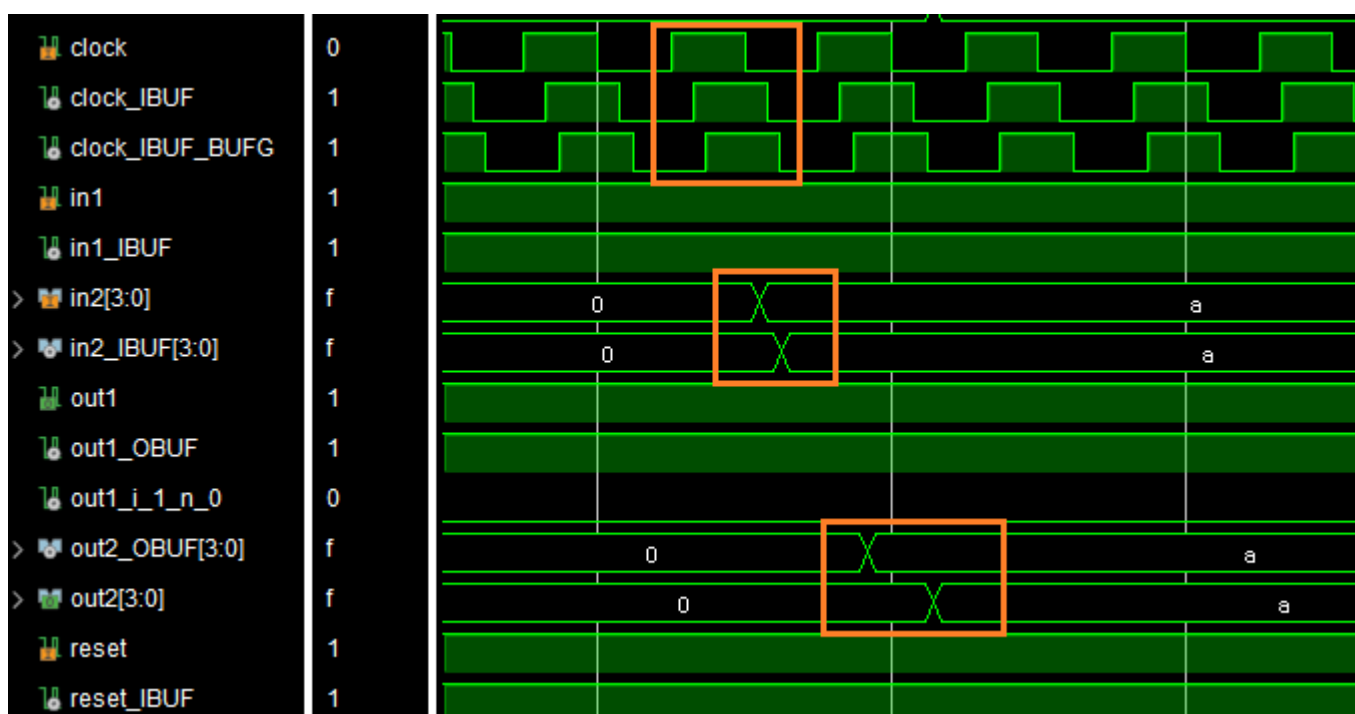
3) Post-Synthesis Timing Simulation

SIMULATION - Run Simulation 클릭 - Run Post-Synthesis Timing Simulation 클릭합니다.



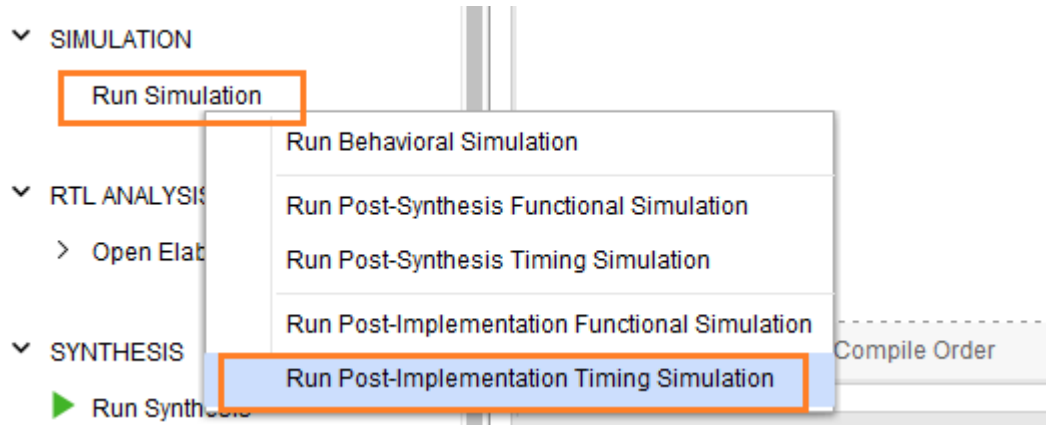
ex_code01 - 우클릭 - Add to Wave Window 클릭합니다. wave 윈도우에 많은 신호들이 추가되었습니다. 추가된 신호는 2) Post-Synthesis Functional Simulation 에서 추가된 신호와 동일합니다. 여기에서는 타이밍, 딜레이가 추가되어 simulation을 진행합니다.

하단의 Tcl Console 창에 “run 0.02ms”를 입력하고 엔터키를 눌러서 0.02ms 동안 simulation을 진행합니다. 아래 그림은 파형을 보여줍니다. 일부분을 확대하였는데, clock, clock_IBUF, clock_IBUF_BUFG, 3개의 신호가 다르게 나타납니다. 신호들이 조금씩 delay 되어 나타납니다. 이는 IBUF, BUFG를 거치면서 transition delay가 발생하기 때문입니다. 또한 in2, in2_IBUF와 out2_OBUF, out2 에도 delay가 추가되었습니다. post-synthesis timing sim. 은 각 로직의 transition delay 가 추가되어서 simulation을 진행합니다.



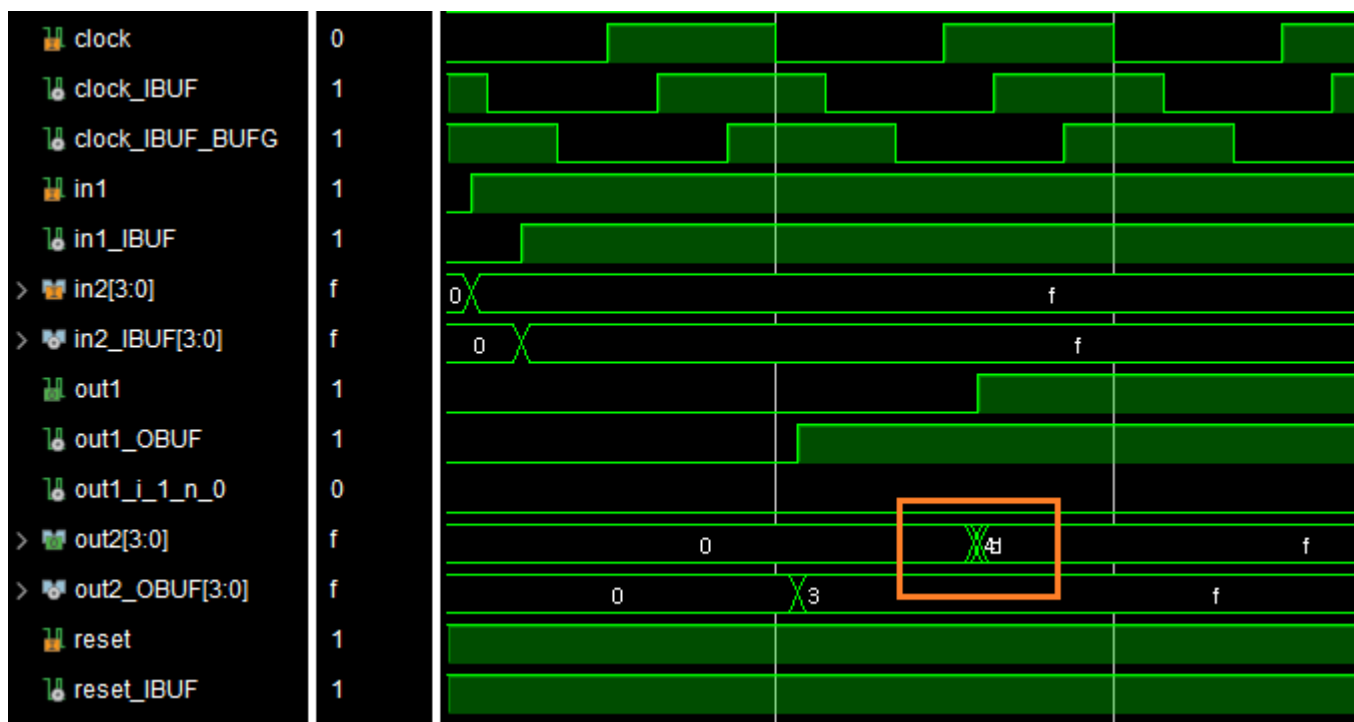
4) Post-Implementation Timing Simulation

SIMULATION - Run Simulation 클릭 - Run Post-Implementation Timing Simulation 클릭합니다.

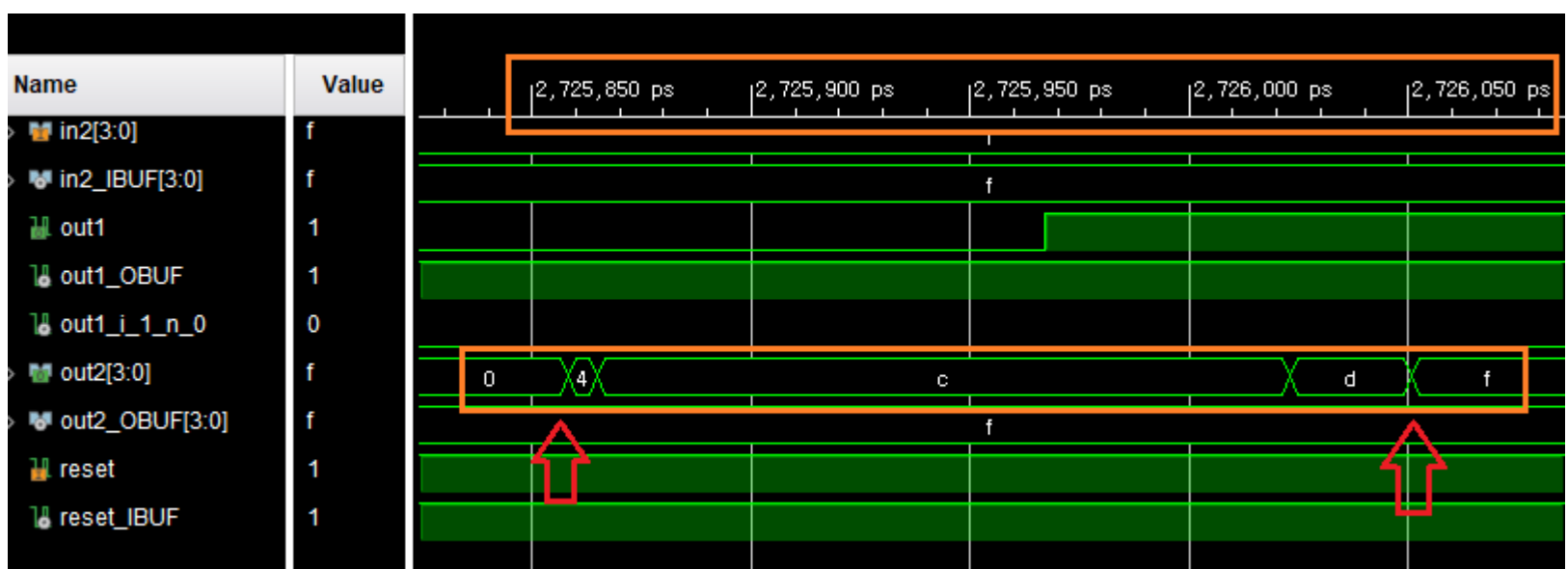


ex_code01 - 우클릭 - Add to Wave Window 클릭합니다. wave 윈도우에 많은 신호들이 추가되었습니다. 추가된 신호는 2) Post-Synthesis Functional Simulation 에서 추가된 신호와 동일합니다. 여기에서는 타이밍, 딜레이가 추가되어 simulation을 진행합니다. 특히 P & R (Place & Route) 에 의한 딜레이가 추가됩니다.

하단의 Tcl Console 창에 “run 0.02ms”을 입력하고 엔터키를 눌러서 0.02ms 동안 simulation을 진행합니다. 아래 그림은 파형을 보여줍니다. 일부분을 확대하였습니다. out2 값이 변하는 부분을 유의해서 살펴보겠습니다. 값이 바뀌는 부분에 glitch 노이즈 처럼 보이는 현상이 있습니다.



아래 그림은 이 부분을 좀더 확대해서 보여줍니다. out2 값이 0x0에서 0xf 로 변하는데 0x4 -> 0xc -> 0xd -> 0xf 로 변하고 있습니다. 이는 Routing 딜레이에 의한 것입니다. 즉 bit[2] 가 먼저 1로 바뀌고, 그 다음에 bit[3], bit[0], bit[1]의 순서로 값이 바뀌는 것을 알 수 있습니다. 그러나 delay 값을 계산해 보면 대략적으로 (2,726,050 ps - 2,725,850 ps = 200ps) 0.2ns 정도 입니다. 이는 디바이스마다 다르고, P&R에 따라서 달라집니다. Generate Bitstream을 진행했는데, Timing Error가 발생하면 Timing Simulation을 해서 어느 부분에 발생하는지를 확인하고 코드를 수정해야 합니다. 이 부분은 다음장에서 예를 들어 설명하도록 하겠습니다. FPGA개발을 완료하고 ASIC으로 구현하기 위해서는 반드시 Timing Simulation을 해야 합니다. Timing Simulation은 각 소자마다 delay가 적용되고, P&R delay도 적용되기 때문에 시간이 무척 많이 소요됩니다. 그러나 FPGA에서만 개발을 진행할 경우에는 Behavioral sim. 에서 주로 문제를 해결합니다. 이는 개발자의 성향에 따라 차이가 있습니다.



- Timing violation (타이밍 에러) : 위의 파형에서 in2의 값이 0x0에서 0x0F로 변합니다. 만약 out2의 값이 0x0에서 0x0F로 변하는 시간이 1-clock 이상의 시간이 발생하면 Timing violation이 발생합니다. 즉 어떤 연산의 결과값이 1-clock 이상의 시간이 소요되면, 이는 타이밍 에러를 발생시킵니다. 모든 디지털 로직은 1-clock안에 동작하는 것을 가정하기 때문입니다. Timing violation이 발생할 때, 이를 해결하는 것은 중간에 register를 하나 추가하는 것입니다. 즉 1-clock 안에 처리하는 것이 아니라, 나누어서 2-clock에 처리하도록 구성하는 것입니다. 이로 인해서 전체적으로 1-clock 딜레이가 추가됩니다. 다른 연관된 부분들도 모두 1-clock 딜레이를 추가합니다. verilog 코드를 보면 없어도 되는 신호들인데, _1d, _2d 신호들이 있는 것을 보게 됩니다. 이는 타이밍을 맞추기 위해서 delay가 추가된 것입니다. (물론 P&R 조건을 바꾸어서 다시 P&R을 진행해서 P&R delay를 줄이는 방법도 있지만, 저는 그렇게 추천하지는 않습니다.)

3. reg 와 wire

이번 장에서는 register 와 wire 에 대해서 설명합니다. register와 wire의 차이점이 무엇이고 언제 register를 사용하고 언제 wire를 사용해야 하는지를 설명하도록 하겠습니다.

register와 wire을 이해하기 위해서는 delay의 개념을 이해해야 합니다. register는 앞장에서 설명 드렸듯이 Flip/Flop으로 구성되어 있고, clock에 동기 되어 동작합니다. wire는 and, or, not 을 이용한 모든 연산(더하기, 빼기, 곱하기, 몫 등)에 이용됩니다. 연산을 수행하는 데에는 시간이 필요합니다. 앞장의 Timing Simulation에서 설명했듯이, Propagation Delay, P&R Delay 등의 시간이 필요합니다.

다음은 연산에 소요되는 시간과 사용되는 main clock과의 관계를 나타냅니다.

1) 어떤 연산이 1-clock 안에 이루어 지는 경우

- ✓ wire를 이용하여 연산처리
- ✓ reg를 이용하여 연산결과를 1-clock delay
- ✓ 총 delay : 1-clock

2) 어떤 연산이 1-clock으로는 부족하고 2-clock 이 필요한 경우

- ✓ 연산을 2부분으로 나누어서 처리
- ✓ wire를 이용하여 연산처리 - 1
- ✓ reg를 이용하여 연산결과-1을 1-clock delay
- ✓ wire를 이용하여 연산처리 - 2 (1-clock delay 신호들을 연산 처리함)
- ✓ reg를 이용하여 연산결과-2을 1-clock delay
- ✓ 총 delay : 2-clock

3.1 실습 - 1 (1-clock 안에 이루어지는 경우)

이번장에서는 1-clock 안에 연산이 이루어지는 경우에 reg, wire의 활용에 대해서 설명합니다. 이번장에서는 단순한 adder를 가지고 설명하도록 하겠습니다.

수식 : $out = in1 + in2$;

모듈 이름은 ex_code02a 입니다.

```

23 module ex_code02a(
24     reset      ,
25     clock      ,
26     in1        ,
27     in2        ,
28     out        ,
29 );
30 input      reset      ;
31 input      clock      ;
32 input [9:0] in1       ;
33 input [9:0] in2       ;
34 output [10:0] out     ;
35
36 wire [10:0] add = {1'b0, in1} + {1'b0, in2};
37
38 reg [10:0] out ;
39 always @(posedge clock or negedge reset)
40 begin
41     if(~reset) out <= 11'b0;
42     else out <= add ;
43 end
44
45 endmodule

```

- ✓ 라인 23 - 29 : ex_code02a 모듈 define
- ✓ 라인 30 - 34 : in/out port define
- ✓ 라인 36 : wire 연산 구현, in1+in2의 결과를 add 에 저장, 10bits 2개를 더하면 11bits 결과가 됨에 유의
- ✓ 라인 38 - 43 : 연산 결과를 reg out 에 저장함.
- ✓ 결과적으로 in1, in2에 비해서 out 결과값은 1-clock delay 후에 나타남.

위의 코드를 변경하여 wire를 사용하지 않고 reg만 사용해서 구현합니다. 모듈 이름은 ex_code02b 입니다.

```
23 module ex_code02b(  
24     reset      ,  
25     clock      ,  
26     in1        ,  
27     in2        ,  
28     out        ,  
29 );  
30 input         reset      ;  
31 input         clock      ;  
32 input [9:0]   in1        ;  
33 input [9:0]   in2        ;  
34 output [10:0] out        ;  
35  
36 reg [10:0] out ;  
37 always @(posedge clock or negedge reset)  
38 begin  
39     if(~reset)      out <= 11'b0;  
40     else            out <= {1'b0, in1} + {1'b0, in2} ;  
41 end  
42  
43 endmodule
```

ex_code02a, ex_code02b는 동일한 코드입니다. 둘 다 in1, in2 데이터의 1-clock 후에 결과가 out로 나타납니다. 두 코드의 차이점은 중간 과정에서 wire를 사용하였느냐, 사용하지 않았느냐의 차이밖에 없습니다. wire는 임시 변수와 같은 역할을 하는 것입니다. 두 코드는 완전 동일한 코드입니다.

다음 코드를 보겠습니다.

$out = ((in_a + in_b) > 1023) ? 1023 : (in_a + in_b)$ // $(in_a + in_b)$ 의 값이 1023보다 크면 1023, 아니면 $in_a + in_b$

모듈 이름은 ex_code03a 입니다.

```

23 module ex_code03a(
24     reset      ,
25     clock      ,
26     in1        ,
27     in2        ,
28     out        ,
29 );
30 input         reset      ;
31 input         clock      ;
32 input [9:0]    in1       ;
33 input [9:0]    in2       ;
34 output [9:0]   out       ;
35
36 wire [10:0]    add = {1'b0, in1} + {1'b0, in2};
37
38 reg [9:0]      out ;
39 always @(posedge clock or negedge reset)
40 begin
41     if(~reset)    out <= 10'b0;
42     else          out <= add[1] ? 10'd1023 : add[9:0] ;
43 end
44
45 endmodule

```

wire를 사용하지 않고 구현합니다. 모듈 이름은 ex_code03b 입니다.

```

23 module ex_code03b(
24     reset      ,
25     clock      ,
26     in1        ,
27     in2        ,
28     out        ,
29 );
30 input         reset      ;
31 input         clock      ;
32 input [9:0]    in1       ;
33 input [9:0]    in2       ;
34 output [9:0]   out       ;
35
36 reg [9:0]      out ;
37 always @(posedge clock or negedge reset)
38 begin
39     if(~reset) out <= 10'b0;
40     else      out <= (({1'b0, in1}+{1'b0, in2}) > 11'd1023) ? 10'd1023 : in1+in2 ;
41 end
42
43 endmodule

```

두 코드는 같은 듯 하지만 사실은 다릅니다. ex_code03a는 “11bits adder (36라인) + 1bit 비교기(42라인)”를 사용하였습니다. 그러나 ex_code03b는 “11bits adder + 11bits 비교기(40라인)”를 사용하였습니다. ex_code03a 가 더 효율적인 코드입니다. 타이밍상으로도 더 좋은 코드입니다. (즉 수행시간이 더 짧다는 말입니다)

3.2 실습 - 2 (2-clock 안에 이루어지는 경우)

이번장에서는 연산이 1-clock으로는 시간이 부족해서 2-clock으로 나누어서 연산을 구현하는 것에 대해서 설명합니다.

구현할 연산 : $out = in1 + in2 + in3$ // 1-clock 안에 3개의 adder를 구현할 수 없어서, 첫번째 clock에는 $in1+in2$ 를 구하고, 두번째 clock에 $(in1+in2)$ 결과 + $in3$ 을 구한다고 가정합니다.

모듈 이름은 ex_code04a 입니다.

```

23 module ex_code04a(
24     reset      ,
25     clock      ,
26     in1        ,
27     in2        ,
28     in3        ,
29     out        ,
30 );
31 input      reset      ;
32 input      clock      ;
33 input [9:0] in1       ;
34 input [9:0] in2       ;
35 input [9:0] in3       ;
36 output [11:0] out     ;
37
38 wire [10:0] add1 = {1'b0, in1} + {1'b0, in2};
39
40 reg [10:0] add1d;
41 always @(posedge clock or negedge reset)
42 begin
43     if(~reset)      add1d <= 11'b0;
44     else            add1d <= add1;
45 end
46
47 reg [9:0] in3d;
48 always @(posedge clock or negedge reset)
49 begin
50     if(~reset)      in3d <= 10'b0;
51     else            in3d <= in3 ;
52 end
53
54 wire [11:0] add2 = {1'b0, add1d} + {2'b0, in3d};
55 reg [11:0] out ;
56 always @(posedge clock or negedge reset)
57 begin
58     if(~reset)      out <= 12'b0;
59     else            out <= add2 ;
60 end
61
62
63 endmodule

```

- ✓ 라인 38 - 45 : $in1 + in2$ 를 연산 후 1-clock delay 합니다.
- ✓ 라인 47 - 52 : 매우 중요한 부분입니다. add2 연산에 사용하는 add1d 는 $in3$ 에 비해서 1-clock delay 된 신호입니다. 따라서 add2를 구할 때, 반드시 $in3$ 을 1-clock delay된 신호를 이용하여 결과를 만들어야 합니다.
- ✓ 라인 54 - 60 : add1d + $in3d$ 후에 1-clock delay를 해서 out 결과를 만들어 냅니다.

다음은 wire를 사용하지 않는 코드입니다. 모듈 이름은 ex_code04b 입니다.

```

23 module ex_code04b(
24     reset      ,
25     clock      ,
26     in1        ,
27     in2        ,
28     in3        ,
29     out        ,
30 );
31 input          reset      ;
32 input          clock      ;
33 input [9:0]    in1        ;
34 input [9:0]    in2        ;
35 input [9:0]    in3        ;
36 output [11:0]  out        ;
37
38 reg [10:0]     addld;
39 always @(posedge clock or negedge reset)
40 begin
41     if(~reset)      addld <= 11'b0;
42     else            addld <= {1'b0, in1} + {1'b0, in2};
43 end
44
45 reg [9:0]      in3d;
46 always @(posedge clock or negedge reset)
47 begin
48     if(~reset)      in3d <= 10'b0;
49     else            in3d <= in3 ;
50 end
51
52 reg [11:0]     out ;
53 always @(posedge clock or negedge reset)
54 begin
55     if(~reset)      out <= 12'b0;
56     else            out <= {1'b0, addld} + {2'b0, in3d} ;
57 end
58
59
60 endmodule

```

wire 가 제거된 코드입니다. ex_code04a 와 동일한 코드입니다.

wire는 중간 변수와 같은 역할을 합니다. 필요에 따라 코드를 구현하는 사용자의 스타일대로 사용할 수도 있고, 사용하지 않을 수도 있습니다. 중요한 것은 delay의 개념을 알고 각 신호의 delay를 맞추어 주는 것입니다.

3.3 delay 오류로 인한 오동작

delay를 맞추지 않았을 때, 발생하는 오류에 대해서 simulation으로 확인해 보도록 하겠습니다. ex_code04c_err 모듈은 in3d 를 사용하지 않은 코드입니다.

```

23 module ex_code04c_err(
24     reset      ,
25     clock      ,
26     in1        ,
27     in2        ,
28     in3        ,
29     out        ,
30 );
31 input      reset      ;
32 input      clock      ;
33 input [9:0] in1       ;
34 input [9:0] in2       ;
35 input [9:0] in3       ;
36 output [11:0] out     ;
37
38 reg [10:0] addld;
39 always @(posedge clock or negedge reset)
40 begin
41     if(~reset)      addld <= 11'b0;
42     else            addld <= {1'b0, in1} + {1'b0, in2};
43 end
44
45 /*
46 reg [9:0] in3d;
47 always @(posedge clock or negedge reset)
48 begin
49     if(~reset)      in3d <= 10'b0;
50     else            in3d <= in3 ;
51 end
52 */
53
54 reg [11:0] out ;
55 always @(posedge clock or negedge reset)
56 begin
57     if(~reset)      out <= 12'b0;
58     // else          out <= {1'b0, addld} + {2'b0, in3d} ;
59     else            out <= {1'b0, addld} + {2'b0, in3 } ;
60 end

```

Test Bench는 tb_ex_code04 입니다. ex_code04b 모듈과 ex_code04c_err 모듈에 0 ~ 1023까지 동일한 데이터를 in1, in2, in3에 입력해서 결과를 확인합니다. 우리가 기대하는 결과는 0, 3, 6, 9, ~ 입니다.

```

23 module tb_ex_code04();
24
25     reg            reset, clock;
26 initial          begin
27     reset = 1'b0;
28     clock = 1'b0;
29
30     #5000 reset = 1'b1;
31
32 end
33
34 always #5        clock = ~clock;          // 100 Mhz
35
36
37 reg      [9:0]    data;
38 always @(posedge clock or negedge reset)
39 begin
40     if(~reset)    data <= 10'b0;
41     else          data <= data + 1'b1;
42 end
43
44 wire     [11:0]   out_ok  ;
45 ex_code04b ex_code04b(
46     .reset      (reset    ),
47     .clock      (clock    ),
48     .in1        (data     ),
49     .in2        (data     ),
50     .in3        (data     ),
51     .out        (out_ok   )
52 );
53
54 wire     [11:0]   out_err  ;
55 ex_code04c_err ex_code04c_err(
56     .reset      (reset    ),
57     .clock      (clock    ),
58     .in1        (data     ),
59     .in2        (data     ),
60     .in3        (data     ),
61     .out        (out_err  )
62 );
63
64 endmodule

```

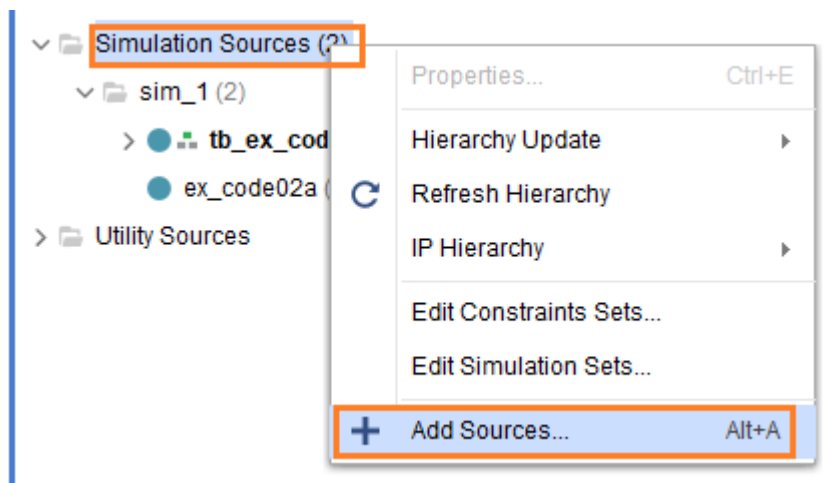
- ✓ 라인 37 - 42 : 0 ~ 1023까지 입력 데이터 생성
- ✓ 라인 44 - 52 : ex_code04b 모듈 추가
- ✓ 라인 54 - 62 : ex_code04c_err 모듈 추가

tb_ex_code04.v, ex_code04b.v, ex_code04c_err.v 파일을 Simulation Sources에 추가하고, tb_ex_code04 모듈을 Top 모듈로 지정하고, Run Simulation - Behavioral Simulation을 클릭합니다.

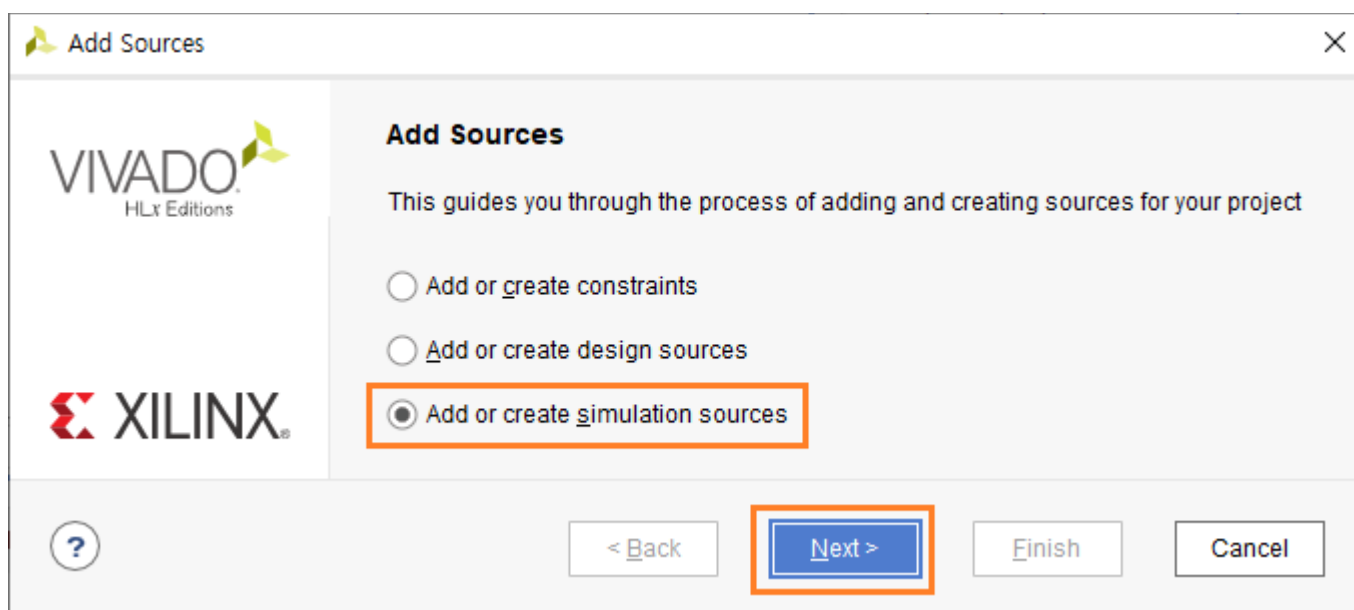
이 과정을 설명하도록 하겠습니다.

소스 코드(ex_code04b.v, ex_code04c_err.v)는 “ex_overview.srcs\sources_1\new” 폴더에 추가합니다. (프로젝트 이름은 ex_oeverview 라고 가정합니다) Test Bench 파일은 “ex_overview.srcs\sim_1\new” 폴더에 추가합니다.

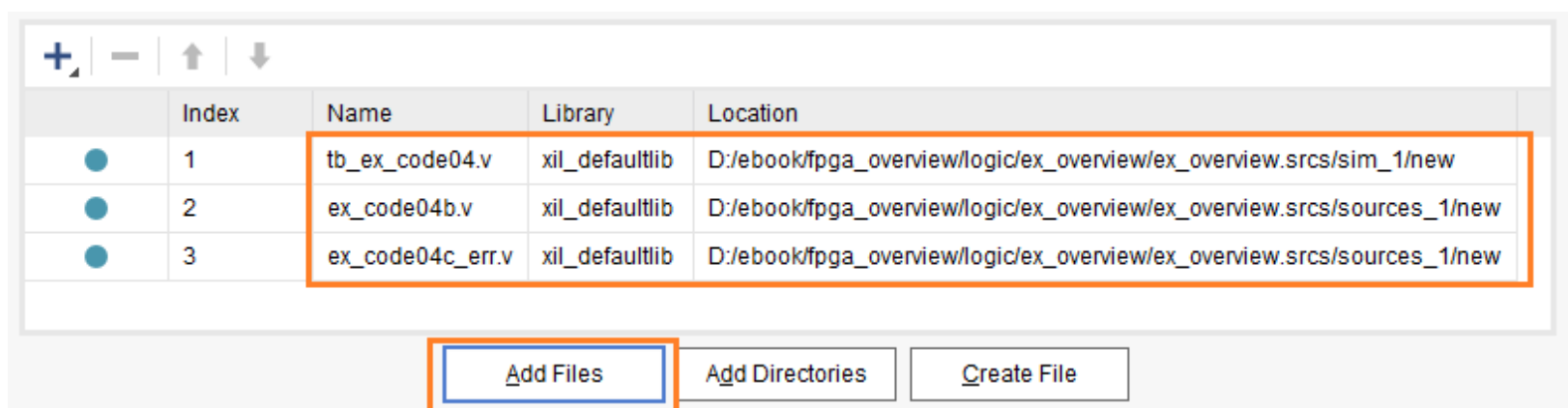
Simulation Sources - 우클릭 - Add Sources... 클릭합니다.



Add or Create simulation sources 선택하고 Next 클릭합니다.

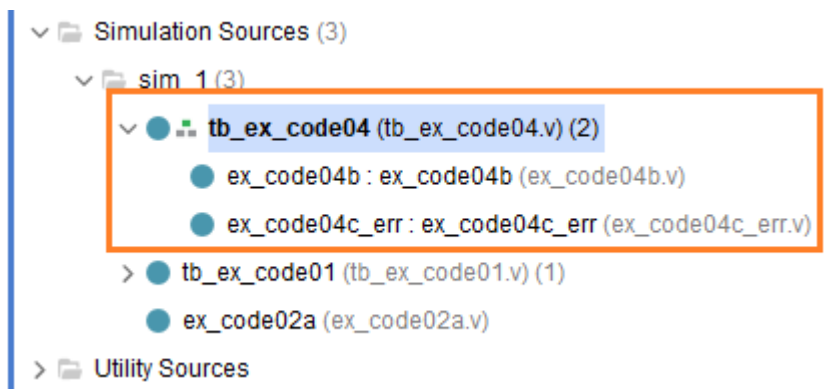


Add Sources 윈도우에서 Add Files 버튼을 클릭하여 3개의 파일을 선택합니다.



Finish 버튼을 클릭하면 3개의 파일이 추가됩니다.

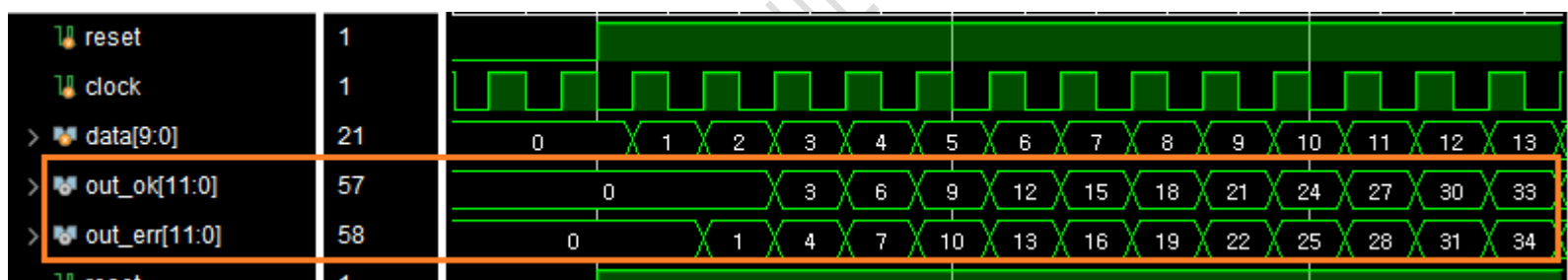
추가된 tb_ex_code04 선택 - 우클릭 - Set as Top 클릭하여, tb_ex_code04을 Top Module로 지정합니다.



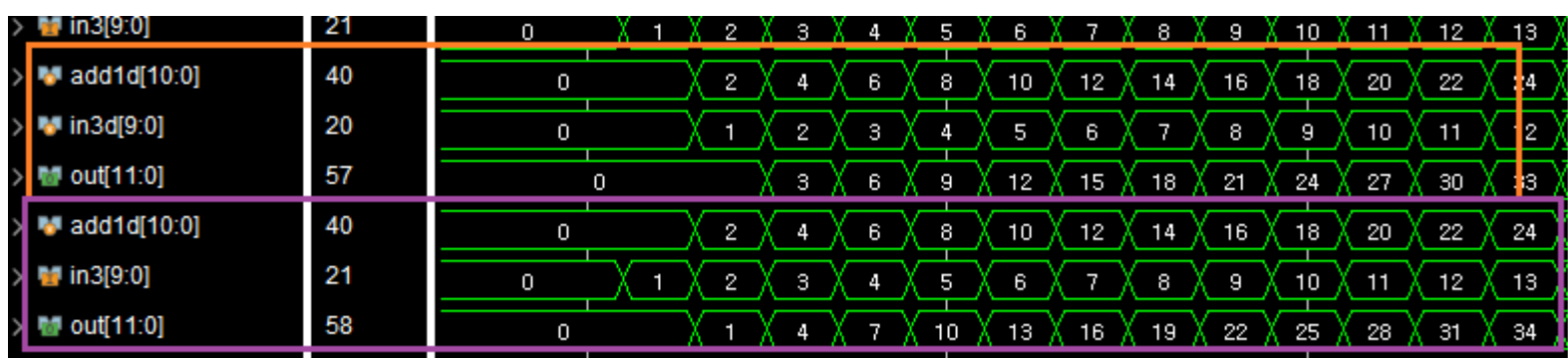
SIMULATION - Run Simulation - Run Behavioral Simulation 클릭해서 simulation을 진행합니다. ex_code04, ex_code04c_err 신호들을 wave 윈도우에 추가하고, Tcl Console 창에 “run 0.05m” 입력 후 엔터키를 눌러서 simulation을 진행합니다.

데이터를 Decimal로 보기 위하여, Wave 윈도우의 모든 신호를 선택하고 - 우클릭 - Radix - Unsigned Decimal 클릭합니다.

아래 그림은 simulation 결과를 보여줍니다. out_ok 값은 0, 3, 6, 9 ~ 로 원하는 값이 나오는데, out_err 값은 0, 1, 4, 7, 10 ~ 으로 나오고 있습니다.



아래 그림은 두 모듈의 차이를 보여줍니다. 첫번째 사각형은 ex_code04b 모듈의 파형입니다. add1d와 in3d가 더해져서 원하는 결과가 나오지만, 두번째 사각형은 ex_code04c_err 모듈이 파형으로 add1d와 in3이 더해져서 잘못된 결과가 나오고 있습니다.



특히 영상처리를 할 경우에는 이와 같이 매 pixel clock마다 데이터 값이 변하기 때문에 delay를 맞추어 주지 않으면 이상한 결과가 나오게 됩니다.

4. signed, unsigned 덧셈, 뺄셈

verilog에서는 연산을 하는 과정에서 자리수가 매우 중요합니다. 자리수를 반드시 사용하는 습관이 중요합니다. 자리수를 명시하지 않으면, 사용자의 의도와 다르게 처리될 수 있습니다. 이번 장에서는 signed, unsigned의 덧셈, 뺄셈에 대해서 알아보겠습니다.

4.1 unsigned 덧셈

2개의 10bits을 더하는 코드를 구현합니다.

```

23 module ex_code05a(
24     reset      ,
25     clock      ,
26     in1        ,
27     in2        ,
28     out1       ,
29     out2
30 );
31 input      reset      ;
32 input      clock      ;
33 input [9:0] in1       ;
34 input [9:0] in2       ;
35 output [9:0] out1     ;
36 output [10:0] out2    ;
37
38 wire [10:0] add = {1'b0, in1} + {1'b0, in2};
39
40 reg [9:0] out1 ;
41 always @(posedge clock or negedge reset)
42 begin
43     if(~reset)    out1 <= 10'b0;
44     else          out1 <= add[10] ? 10'd1023 : add[9:0] ;
45 end
46
47 reg [10:0] out2;
48 always @(posedge clock or negedge reset)
49 begin
50     if(~reset)    out2 <= 11'b0;
51     else          out2 <= add ;
52 end
53
54 endmodule

```

- ✓ 라인 38 : 두개의 10bits를 더하면 결과는 11bits가 됩니다. in1, in2를 11bits로 변환 후(MSB에 0를 추가) 더해 주었습니다.
- ✓ 라인 40 - 45 : out1은 10bits 결과입니다. 연산 결과가 10bits를 초과하면(carry 발생, 1023보다 크면), 1023이 되고, 1023보다 작거나 같으면 bit[10]은 버리고, bit[9:0]만 취하게 됩니다.
- ✓ 라인 47 - 52 : 연산 결과가 11bits이면 결과를 그대로 취하면 됩니다.

- ◆ 자리수가 다를 때에는 최종 결과값의 자리수에 맞추어 줍니다. 예를 들어 in1 : 5bits, in2 : 7bits 이면, 결과는 8bits 입니다.
 - ✓ `wire [7:0] out = {3'b0, in1} + {1'b0, in2}`
- ◆ 2의 이상의 수를 더할 때에는 모든 합의 최대값이 얼마인지를 확인해서 결과값의 비트 수를 정합니다. 예를 들면, 10bits 3개의 수를 더하면 결과는 12bits가 됩니다. 10bits 4개의 수를 더하면 결과는 12bits가 됩니다.
 - ✓ `wire [11:0] out = {2'b0, in1} + {2'b0, in2} + {2'b0, in3}; // in1, in2, in3 : 10bits`
 - ✓ `wire [11:0] out = {2'b0, in1} + {2'b0, in2} + {2'b0, in3} + {2'b0, in4}; // in1, in2, in3, in4 : 10bits`
 - ✓ `wire [10:0] out = {1'b0, in1} + {3'b0, in2} + {4'b0, in3}; // in1 : 10bits, in2 : 8bits, in3 : 7bits,`
`1023(10bits의 최대값) + 255(8bits의 최대값) + 127(7bits의 최대값) = 1405 (0x57D), 11bits`
- ◆ 4개 이상의 수를 더할 때에는 delay를 생각해야 합니다. fpga 내부에 multiplier가 있어서 대부분 1-clock안에 연산이 이루어지지만, 경우에 따라서는 1-clock이상의 시간이 소요되어 Timing violation이 발생할 수 있습니다.

IHL

4.2 unsigned 뺄셈

2개의 10bits 뺄셈을 구현하는 코드입니다.

```

23 module ex_code05b(
24     reset      ,
25     clock      ,
26     in1        ,
27     in2        ,
28     out1       ,
29     out2       ,
30 );
31 input         reset      ;
32 input         clock      ;
33 input [9:0]    in1       ;
34 input [9:0]    in2       ;
35 output [9:0]   out1      ;
36 output [10:0]  out2      ;
37
38 wire [10:0]    sub = {1'b0, in1} - {1'b0, in2};
39
40 reg [9:0]      out1 ;
41 always @(posedge clock or negedge reset)
42 begin
43     if(~reset)      out1 <= 10'b0;
44     else            out1 <= sub[10] ? 10'd0 : sub[9:0] ;
45 end
46
47 endmodule

```

- ✓ 라인 38 : 10bits in1 에서 10bits in2를 뺄 때, 연산결과를 11bits 로 만든 이유는, carry 가 발생하기 때문입니다. 즉 음수연산결과(in2 > in1)가 발생할 수 있기 때문입니다. 연산결과가 음수일 때에는 unsigned 연산이기 때문에 0로 만들어 줍니다. 이를 구현한 것이 45 라인 입니다.
- ✓ 라인 40 - 45 : 연산 결과를 out 에 저장합니다. 연산 결과가 음수일 때에는 0로 만들어 줍니다.

4.3 signed 덧셈, 뺄셈

부호를 가진 수(signed number)의 연산(덧셈, 뺄셈)을 이해하기 위해서는 양수, 음수를 어떻게 표현하는지를 알아야 합니다. 부호를 가진 수에서 최상위비트(MSB)는 부호를 의미합니다. 최상위 비트가 0이면 양수를 의미하고, 최상위 비트가 1이면 음수를 의미합니다. 크기(절대값)은 양수일 때에는 msb를 제외한 값의 크기가 절대값이고, 음수일 때에는 msb를 제외한 수를 반전시킨 후에 1을 더해 줍니다. 이러한 표현 방식을 2의 보수 표현 방식(2's complement)이라고 합니다. verilog에서는 부호를 가진 수를 2의 보수 표현 방식을 사용합니다. 결론을 미리 말씀드리면 verilog에서 좀 복잡해 보이는 2의 보수 표현방식을 사용하는 이유는 연산(덧셈, 뺄셈)을 간단하게 하기 위해서입니다.

8bits 기준으로 2의 보수 표현방식을 살펴봅니다. 양수 35, 음수 35를 어떻게 표현하는지 보겠습니다.

+35

- ✓ bit[7] : 0
- ✓ bit[6:0] : 35, 0x23
- ✓ +35 : 0010_0011

-35

- ✓ bit[7] : 1
- ✓ bit[6:0] : $\sim 0x23 + 1 = 0x5D$
- ✓ -35 : 1101_1101

1101_1101이 어떻게 -35가 되는지도 살펴보겠습니다. msb가 1이므로 음수입니다. 절대값은 $\sim \text{bit}[6:0] + 1$ 입니다. 즉 101_1101을 인버팅하면, 010_0010이 되고 여기에 1을 더하면, 010_0011 이 되어 $0x23=35$ 가 됩니다. 따라서 -35가 됩니다.

signed number의 비트확장에 대해서 알아보니다. 예를 들어 10bits의 signed number를 11bits의 signed number로 변환하기 위해서는 어떻게 해야 할까요? sign bit를 최상위 비트에 반복해서 사용하면 됩니다.

+35 : 8bits 인 경우, 0010_0011

- ✓ 9bits로 변경 : 0_0010_0011
- ✓ 10bits로 변경 : 00_0010_0011

-35 : 8bits의 경우, 1101_1101

✓ 9bits로 변경 : 1_1101_1101

✓ 10bits로 변경 : 11_1101_1101

양수, 음수에 무관하게 sign bit를 최상위비트에 반복해서 채워주면 bits를 확장할 수 있습니다.

verilog에서 signed number의 덧셈, 뺄셈은 이러한 bit 확장을 이용하면 매우 간단하게 구현할 수 있습니다. 아래는 10bits 의 signed number에 대한 덧셈, 뺄셈입니다.

```

23 module ex_code05c(
24     reset      ,
25     clock      ,
26     in1        ,
27     in2        ,
28     out1       ,
29     out2       ,
30 );
31 input      reset      ;
32 input      clock      ;
33 input [9:0] in1       ;
34 input [9:0] in2       ;
35 output [10:0] out1    ;
36 output [10:0] out2    ;
37
38 wire [10:0] add = {in1[9], in1} + {in2[9], in2};
39 wire [10:0] sub = {in1[9], in1} - {in2[9], in2};
40
41 reg [10:0] out1 ;
42 always @(posedge clock or negedge reset)
43 begin
44     if(~reset)    out1 <= 11'b0;
45     else          out1 <= add ;
46 end
47
48 reg [10:0] out2 ;
49 always @(posedge clock or negedge reset)
50 begin
51     if(~reset)    out2 <= 11'b0;
52     else          out2 <= sub ;
53 end
54
55 endmodule

```

✓ 라인 38-39 : signed number의 더하기, 빼기 연산입니다. 10bits을 11bits로 만들기 위해 sign bit를 msb에 추가 하였습니다.

✓ 라인 41 - 53 : 더하기, 빼기의 결과를 out1, out2로 출력합니다.

위 코드를 simulation으로 확인해 보도록 하겠습니다. Test bench는 tb_ex_code05c 입니다.

```

23 module tb_ex_code05c();
24
25     reg            reset, clock;
26     initial        begin
27         reset = 1'b0;
28         clock = 1'b0;
29
30         #5000    reset = 1'b1;
31
32     end
33
34     always #5      clock = ~clock;        // 100 Mhz
35
36     parameter      p57  = 10'h039;        // +57
37     parameter      p123 = 10'h07b;        // +123
38     parameter      m57  = 10'h3c7;        // -57
39     parameter      m123 = 10'h385;        // -123
40
41     reg [10:0] cnt;
42     always @(posedge clock or negedge reset)
43     begin
44         if(~reset)    cnt <= 11'b0;
45         else          cnt <= cnt + 1'b1;
46     end
47
48     reg [9:0] in1;
49     always @(posedge clock or negedge reset)
50     begin
51         if(~reset)    in1 <= 10'b0;
52         else          in1 <= (cnt==10'd10) ? p57 :
53                               (cnt==10'd20) ? p57 :
54                               (cnt==10'd30) ? m57 :
55                               (cnt==10'd40) ? m57 : in1 ;
56     end
57
58     reg [9:0] in2;
59     always @(posedge clock or negedge reset)
60     begin
61         if(~reset)    in2 <= 10'b0;
62         else          in2 <= (cnt==10'd10) ? p123 :
63                               (cnt==10'd20) ? m123 :
64                               (cnt==10'd30) ? p123 :
65                               (cnt==10'd40) ? m123 : in2 ;
66     end
67
68     wire [10:0] out1, out2 ;
69     ex_code05c ex_code05c(
70         .reset    (reset    ),
71         .clock    (clock    ),
72         .in1      (in1      ),
73         .in2      (in2      ),
74         .out1     (out1     ),
75         .out2     (out2     )
76 );

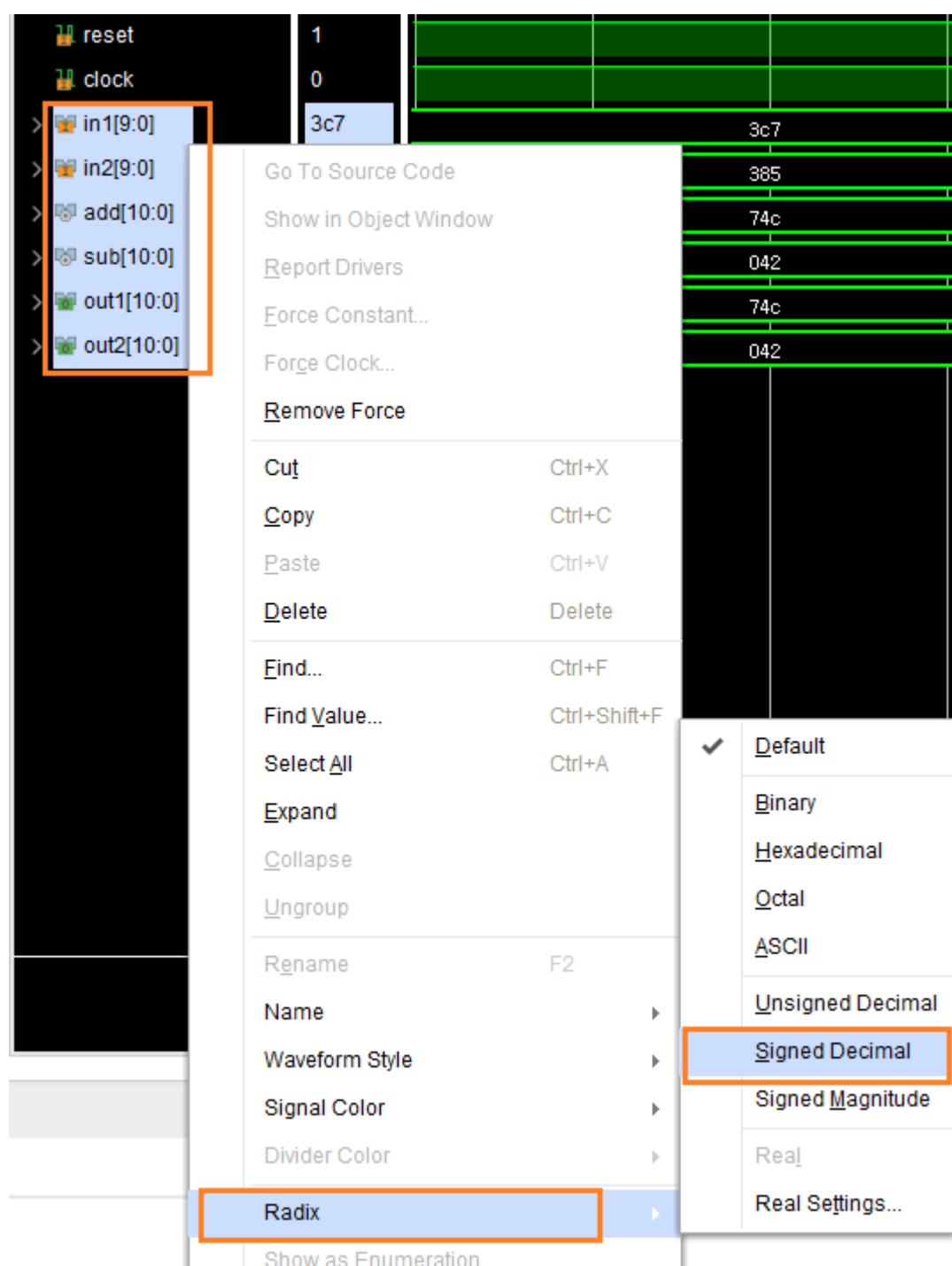
```

- ✓ 라인 25 - 34 : reset, clock 생성
- ✓ 라인 36 - 39 : in1, in2을 위한 파라미터 정의, +57, -57, +123, -123의 10bits signed value 정의
- ✓ 라인 41 - 46 : in1, in2 입력을 위한 counter 생성
- ✓ 라인 48 - 56 : in1에 순차적으로 +57, +57, -57, -57을 입력합니다.
- ✓ 라인 58 - 66 : in2에 순차적으로 +123, -123, +123, -123을 입력합니다.
- ✓ 라인 68 - 76 : ex_code05c 모듈 추가

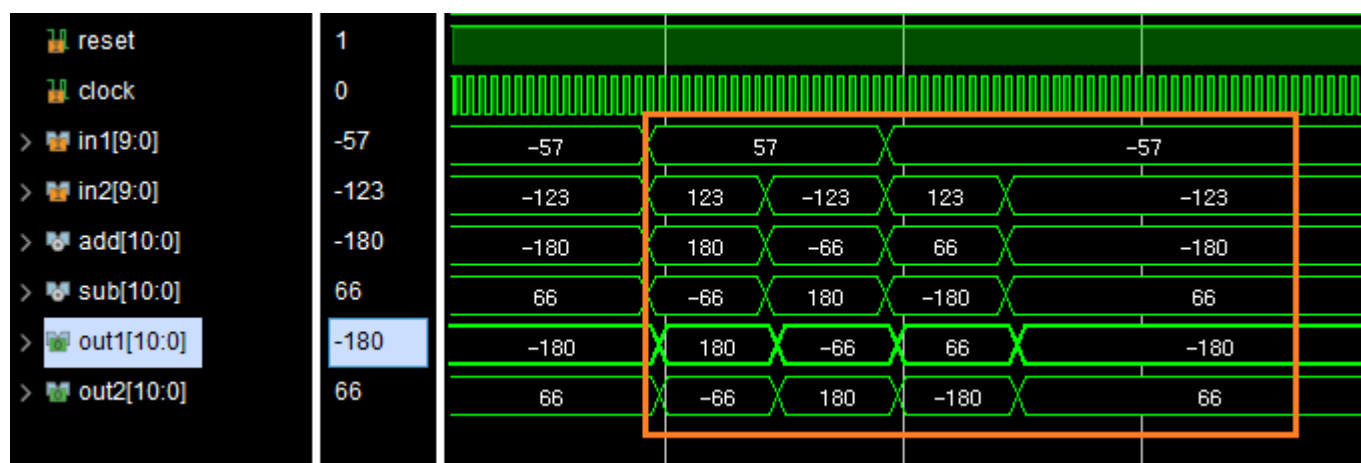
연산결과는 아래표와 같습니다.

	in1	in2	out1(in1+in2)	in1	in2	out2 (in1-in2)
1	+57	+123	+180	+57	+123	-66
2	+57	-123	-66	+57	-123	+180
3	-57	+123	+66	-57	+123	-180
4	-57	-123	-180	-57	-123	+66

- ① Simulation Sources - 우클릭 - Add Sources... 클릭합니다.
- ② Add or create simulation sources 선택 후, Next 클릭합니다.
- ③ Add Files클릭해서, “sim_1\new\tb_ex_code05c.v”, “sources_1\new\ex_code05c.v”를 추가합니다.
Finish를 클릭합니다.
- ④ tb_ex_code05c - 우클릭 - Set as Top 클릭합니다.
- ⑤ SIMULATION - Run Simulation - Run Behavioral Simulation 클릭합니다.
- ⑥ ex_code05c - 우클릭 - Add to Wave Window 클릭합니다.
- ⑦ Tcl Console -Command 창에 “run 0.01ms”입력하고 엔터를 누르면 simulation이 진행됩니다.
- ⑧ 파형 in1 ~ out2를 선택 - 우클릭 - Radix - Signed Decimal 클릭하면, signed 10진수로 값을 표현합니다.



아래는 결과를 보여줍니다. out1 : 180, -66, 66, -180, out2 : -66, 180, -180, 66 으로 원하는 값이 출력되고 있습니다.



4.4 unsigned number 곱하기

이 부분은 프로그래머의 스타일에 따라 다른 견해를 가질 수 있습니다. 본 강의의 내용은 나름 오랫동안 verilog 프로그램을 해온 저자의 경험과 견해를 알려 드립니다. 프로그래머에 따라서 다른 견해를 가질 수 있고, fpga, verilog 기술이 발달하면서 다른 방법들이 있을 수 있음을 알려드립니다.

4.4.1 상수 곱하기

상수값을 곱하는 방법은 크게 2가지가 있습니다.

첫번째 방법은 바로 곱하는 방법입니다.

두번째 방법은 더하기로 변환해서 구현하는 방법입니다. 2의 거듭제곱의 곱하기(예를 들어 x2, x4, x8, x16, 등)는 간단히 shift 연산자 (<<)를 사용하면 매우 빠르게 처리할 수 있는 점을 이용하여 상수값을 2의 거듭제곱의 수와 기타 수로 나누어서 처리하는 방법입니다. 예를 들어 어떤 값에 x35를 해야 하는 경우는 다음과 같습니다.

$$35 = 32 + 2 + 1 \quad : \quad in \times 35 = (in \ll 5) + (in \ll 1) + in ;$$

아래 코드는 2가지 방법을 사용하는 것을 보여줍니다.

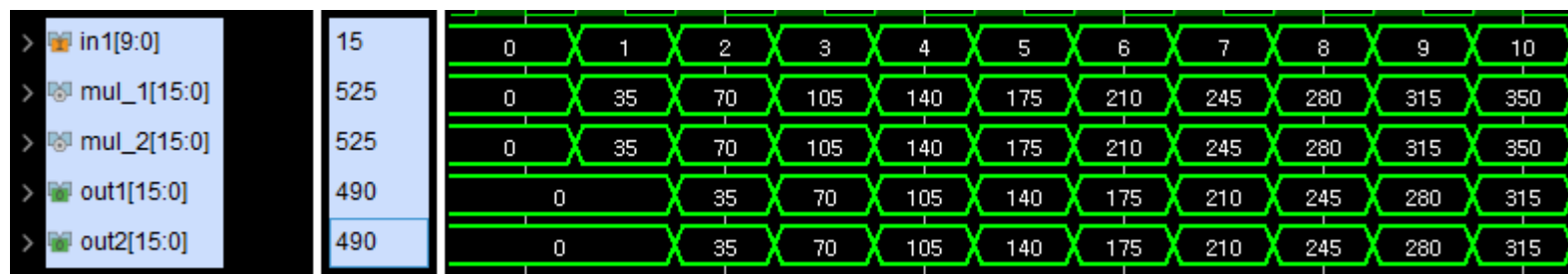
```

22 // in1[9:0] * 35 = out1[15:0]
23 module ex_code05e(
24     reset      ,
25     clock      ,
26     in1        ,
27     out1       ,
28     out2
29 );
30 input         reset      ;
31 input         clock      ;
32 input  [9:0]   in1       ;
33 output [15:0]  out1      ;
34 output [15:0]  out2      ;
35
36 wire  [15:0]   mul_1 = {1'b0, in1, 5'b0} + {5'b0, in1, 1'b0} + {6'b0, in1};
37 wire  [15:0]   mul_2 = in1 * 6'd35;
38
39 reg  [15:0]   out1 ;
40 always @(posedge clock or negedge reset)
41 begin
42     if(~reset)      out1 <= 16'b0;
43     else            out1 <= mul_1 ;
44 end
45
46 reg  [15:0]   out2 ;
47 always @(posedge clock or negedge reset)
48 begin
49     if(~reset)      out2 <= 16'b0;
50     else            out2 <= mul_1 ;
51 end
52
53 endmodule

```

- ✓ 라인 36 : 곱하기를 shift, adder로 변환해서 구현합니다.
- ✓ 라인 37 : fpga 내부에 dsp (digital signal processor)를 사용하여 연산을 처리합니다.

아래 그림은 simulation 결과를 보여줍니다. Test bench는 tb_ex_code05e.v 입니다. $in1 * 35$ 의 값이 out1, out2에 1-clock delay 되어서 나타납니다.



4.4.2 dsp (dsp slice)

fpga 내부에는 곱하기의 연산을 빠르게 처리할 수 있는 dsp가 포함되어 있습니다. 7-series fpga feature에는 아래와 같이 내용이 나와 있습니다. (7 Series FPGAs Data Sheet: Overview)

- DSP slices with 25 x 18 multiplier, 48-bit accumulator, and pre-adder for high-performance filtering, including optimized symmetric coefficient filtering.

dsp는 곱하기 연산을 1-clock안에 처리할 수 있도록 해 줍니다. 최대 25bits x 18bits 까지 처리할 수 있습니다. 만약 25bits x 18bis 이상이 필요할 경우에는 여러 개를 사용하여 처리됩니다. dsp는 무한정 사용할 수 있는 것이 아닙니다. 각 디바이스 마다 사용가능한 dsp의 갯수가 정해져 있습니다. xc7a35t 의 경우, 총 90까지 사용 가능합니다. 곱하기 연산이 많지 않으면 (지원하는 갯수보다 적으면) dsp를 사용하여 처리하면 됩니다. 그러나 지원하는 갯수를 초과하는 경우에는 곱하기 연산을 shift, adder로 구현해서 처리해야 합니다.

4.4.3 변수 곱하기

변수 곱하기를 처리하는 방법도 크게 2가지가 있습니다.

첫째는 dsp를 사용하여 처리하는 방법입니다.

두번째는 dsp를 사용하지 않고, 곱셈을 적당히 나누어서 처리하고 shift, adder를 이용하여 처리하는 방법입니다. 예를 들어 in1[9:0], in2[9:0]의 곱하기 연산을 아래와 같이 처리합니다.

- mul_1 = in1[9:6] * in2[9:6]; // 19:12
- mul_2 = in1[9:6] * in2[5:3]; // 15:9
- mul_3 = in1[9:6] * in2[2:0]; // 12:6
- mul_4 = in1[5:3] * in2[9:6]; // 15:9
- mul_5 = in1[5:3] * in2[5:3]; // 11:6
- mul_6 = in1[5:3] * in2[2:0]; // 8:3
- mul_7 = in1[2:0] * in2[9:6]; // 12:6
- mul_8 = in1[2:0] * in2[5:3]; // 8:3
- mul_9 = in1[2:0] * in2[2:0]; // 5:0
- out = (mul_1<<12) + (mul_2<<9) + (mul_3<<6) + (mul_4<<9) + (mul_5<<6) + (mul_6<<3) + (mul_7<<6)
+ (mul_8<<3) + mul_9

이 방법은 4bits * 4bits의 곱하기는 dsp를 사용하지 않고, LUT, FF으로 처리 가능하고, 1-clock안에 처리 가능하다는 전제로 처리되었습니다. 대부분의 fpga에서는 그렇게 동작합니다. 연산에서 오른쪽에 주석으로 된 곳은 결과값이 속할 bit를 나타냅니다. 9bits * 9bits = 20bits 입니다. 최종 결과의 20bits에서 해당 연산이 어느 곳(bit position)에 속하는지를 보여줍니다. 다음장에서 이를 구현하는 코드를 살펴보도록 하겠습니다. 이러한 코드를 많이 접하고 분석하면 연산에 대해서 많은 도움을 받을 수 있습니다.

4.4.4 10bits * 10bits 연산 구현

아래는 10bits * 10bits을 구현하는 코드를 보여줍니다.

```

1 module mul_10x10_3d( reset,
2                       clock,
3                       in1,
4                       in2,
5                       out
6 );
7   input               reset, clock;
8   input  [9:0]        in1;
9   input  [9:0]        in2;
10  output [19:0]        out;
11  reg    [19:0]        out;
12
13  wire   [7:0]          mul1 = in1[9:6] * in2[9:6];      // 19:12
14  wire   [6:0]          mul2 = in1[9:6] * in2[5:3];      // 15:9
15  wire   [6:0]          mul3 = in1[9:6] * in2[2:0];      // 12:6
16  wire   [6:0]          mul4 = in1[5:3] * in2[9:6];      // 15:9
17  wire   [5:0]          mul5 = in1[5:3] * in2[5:3];      // 11:6
18  wire   [5:0]          mul6 = in1[5:3] * in2[2:0];      // 8:3
19  wire   [6:0]          mul7 = in1[2:0] * in2[9:6];      // 12:6
20  wire   [5:0]          mul8 = in1[2:0] * in2[5:3];      // 8:3
21  wire   [5:0]          mul9 = in1[2:0] * in2[2:0];      // 5:0

```

- ✓ 라인 1 - 11 : module 정의, in/out port 정의, 모듈 이름의 뒤에 있는 “_3d”는 총 delay 수를 나타냅니다. 3d는 in1, in2에 비해서 최종 출력 out는 3-clock delay 되어 나타남을 의미합니다.
- ✓ 라인 13 - 21 : 비트를 나누어서 연산을 수행합니다. 주석문에 있는 연산 결과의 bit position은 나중에 adder를 구현할 때 중요하게 사용됩니다.

```

23 reg    [7:0]          mul1d;
24 reg    [6:0]          mul2d;
25 reg    [6:0]          mul3d;
26 reg    [6:0]          mul4d;
27 reg    [5:0]          mul5d;
28 reg    [5:0]          mul6d;
29 reg    [6:0]          mul7d;
30 reg    [5:0]          mul8d;
31 reg    [5:0]          mul9d;
32
33 wire   [19:0]          add1 = {mul1d, mul5d, mul9d} + {4'b0, mul2d, mul6d, 3'b0};
34 //wire [19:0]          add2 = {7'b0, mul3d, 6'b0} + {4'b0, mul4d, mul8d, 3'b0} + {7'b0, mul7d, 6'b0};
35 wire   [13:0]          add2 = {4'b0, mul3d, 3'b0} + {1'b0, mul4d, mul8d} + {4'b0, mul7d, 3'b0};
36
37 reg    [19:0]          add1d;
38 reg    [13:0]          add2d;
39 wire   [19:0]          mul = add1d + {3'b0, add2d, 3'b0};

```

- ✓ 라인 23 - 31 : 곱하기 연산후에 1-clock delay를 추가합니다.
- ✓ 라인 33-35 : 아주 중요한 부분입니다. adder를 한번에 수행하지 않고, timing delay를 고려하여 나누어서 수행합니다. add1은 20bits 2개를 더하는 연산입니다. mul1d, mul5d, mul9d의 bit position은 각각 [19:12], [11:6], [5:0] 이기 때문에 결합연산자“{ }”를 사용하여 구현하였습니다. 원래의 식은 {mul1d, 12'b0} + {8'b0, mul5d, 6'b0} + {14'b0, mul9d} 입니다. 이를 하나로 {mul1d, mul5d, mul9d}로 만들었습니다. 나머지 연산들도 동일한 방법으로 구현합니다.
- ✓ 라인 37 - 38 : add1, add2 연산후에 1-clock delay를 추가합니다.
- ✓ 라인 39 : 최종적으로 add1d + add2d를 구현합니다.


```
41 always @(posedge clock or negedge reset)
42 begin
43     if (~reset) begin
44         mul1d <= 0;
45         mul2d <= 0;
46         mul3d <= 0;
47         mul4d <= 0;
48         mul5d <= 0;
49         mul6d <= 0;
50         mul7d <= 0;
51         mul8d <= 0;
52         mul9d <= 0;
53         add1d <= 0;
54         add2d <= 0;
55         out <= 0;
56     end
57     else begin
58         mul1d <= mul1;
59         mul2d <= mul2;
60         mul3d <= mul3;
61         mul4d <= mul4;
62         mul5d <= mul5;
63         mul6d <= mul6;
64         mul7d <= mul7;
65         mul8d <= mul8;
66         mul9d <= mul9;
67         add1d <= add1;
68         add2d <= add2;
69         out <= mul;
70     end
71 end
72
73 endmodule
```

✓ 라인 41 - 71 : reg를 구현합니다. (1-clock delay를 구현합니다)

다음 장에서는 dsp를 사용할 때와 사용하지 않을 때의 Implementation 결과를 비교해 보겠습니다.

4.4.5 10bits * 10bits 연산 비교

이번장에서는 10bits * 10bits 연산을 구현하는 2가지 방법에 대해서 Implementation 결과를 비교해 보겠습니다.

아래는 dsp를 사용하는 코드를 보여줍니다.

```

22 module ex_code05f_1(
23     reset      ,
24     clock      ,
25     in1        ,
26     in2        ,
27     out1       ,
28 );
29 input      reset      ;
30 input      clock      ;
31 input [9:0] in1       ;
32 input [9:0] in2       ;
33 output [19:0] out1    ;
34
35 wire [19:0] mul_1 = in1 * in2;
36
37 reg [19:0] out1 ;
38 always @(posedge clock or negedge reset)
39 begin
40     if(~reset)      out1 <= 20'b0 ;
41     else            out1 <= mul_1 ;
42 end
43
44 endmodule

```

아래는 dsp를 사용하지 않고 앞에서 설명했던 mul_10x10_3d 모듈을 사용한 코드를 보여줍니다.

```

22 module ex_code05f_2(
23     reset      ,
24     clock      ,
25     in1        ,
26     in2        ,
27     out1       ,
28 );
29 input      reset      ;
30 input      clock      ;
31 input [9:0] in1       ;
32 input [9:0] in2       ;
33 output [19:0] out1    ;
34
35 wire [19:0] out1 ;
36 mul_10x10_3d mul_10x10_3d(
37     .reset      (reset      ),
38     .clock      (clock      ),
39     .in1        (in1        ),
40     .in2        (in2        ),
41     .out        (out1       )
42 );
43
44 endmodule

```

아래는 xdc 파일을 보여줍니다. in1, in2, out1에는 임의의 핀들을 할당하였습니다. (ex_code05f.xdc)

```

1 ## Clock signal
2 set_property -dict { PACKAGE_PIN E3      IOSTANDARD LVCMOS33 } [get_ports { clock }];
3 create_clock -add -name sys_clk_pin -period 10.00 -waveform {0 5} [get_ports { clock }];
4
5 ## Misc. ChipKit Ports
6 set_property -dict { PACKAGE_PIN C2      IOSTANDARD LVCMOS33 } [get_ports { reset }];
7
8 ##
9 set_property -dict { PACKAGE_PIN R10     IOSTANDARD LVCMOS33 } [get_ports {in1[0]  }];
10 set_property -dict { PACKAGE_PIN G13    IOSTANDARD LVCMOS33 } [get_ports {in1[1]  }];
11 set_property -dict { PACKAGE_PIN D14    IOSTANDARD LVCMOS33 } [get_ports {in1[2]  }];
12 set_property -dict { PACKAGE_PIN C14    IOSTANDARD LVCMOS33 } [get_ports {in1[3]  }];
13 set_property -dict { PACKAGE_PIN B13    IOSTANDARD LVCMOS33 } [get_ports {in1[4]  }];
14 set_property -dict { PACKAGE_PIN B14    IOSTANDARD LVCMOS33 } [get_ports {in1[5]  }];
15 set_property -dict { PACKAGE_PIN C12    IOSTANDARD LVCMOS33 } [get_ports {in1[6]  }];
16 set_property -dict { PACKAGE_PIN B12    IOSTANDARD LVCMOS33 } [get_ports {in1[7]  }];
17 set_property -dict { PACKAGE_PIN B11    IOSTANDARD LVCMOS33 } [get_ports {in1[8]  }];
18 set_property -dict { PACKAGE_PIN A11    IOSTANDARD LVCMOS33 } [get_ports {in1[9]  }];
19
20 set_property -dict { PACKAGE_PIN H16     IOSTANDARD LVCMOS33 } [get_ports {in2[0]  }];
21 set_property -dict { PACKAGE_PIN G16     IOSTANDARD LVCMOS33 } [get_ports {in2[1]  }];
22 set_property -dict { PACKAGE_PIN F15     IOSTANDARD LVCMOS33 } [get_ports {in2[2]  }];
23 set_property -dict { PACKAGE_PIN F16     IOSTANDARD LVCMOS33 } [get_ports {in2[3]  }];
24 set_property -dict { PACKAGE_PIN H14     IOSTANDARD LVCMOS33 } [get_ports {in2[4]  }];
25 set_property -dict { PACKAGE_PIN G14     IOSTANDARD LVCMOS33 } [get_ports {in2[5]  }];
26 set_property -dict { PACKAGE_PIN E17     IOSTANDARD LVCMOS33 } [get_ports {in2[6]  }];
27 set_property -dict { PACKAGE_PIN D17     IOSTANDARD LVCMOS33 } [get_ports {in2[7]  }];
28 set_property -dict { PACKAGE_PIN K13     IOSTANDARD LVCMOS33 } [get_ports {in2[8]  }];
29 set_property -dict { PACKAGE_PIN J13     IOSTANDARD LVCMOS33 } [get_ports {in2[9]  }];
30
31 set_property -dict { PACKAGE_PIN U7      IOSTANDARD LVCMOS33 } [get_ports {out1[0]  }];
32 set_property -dict { PACKAGE_PIN U6      IOSTANDARD LVCMOS33 } [get_ports {out1[1]  }];
33 set_property -dict { PACKAGE_PIN R7      IOSTANDARD LVCMOS33 } [get_ports {out1[2]  }];
34 set_property -dict { PACKAGE_PIN T6      IOSTANDARD LVCMOS33 } [get_ports {out1[3]  }];
35 set_property -dict { PACKAGE_PIN R8      IOSTANDARD LVCMOS33 } [get_ports {out1[4]  }];
36 set_property -dict { PACKAGE_PIN T8      IOSTANDARD LVCMOS33 } [get_ports {out1[5]  }];
37 set_property -dict { PACKAGE_PIN U8      IOSTANDARD LVCMOS33 } [get_ports {out1[6]  }];
38 set_property -dict { PACKAGE_PIN F5      IOSTANDARD LVCMOS33 } [get_ports {out1[7]  }];
39 set_property -dict { PACKAGE_PIN C6      IOSTANDARD LVCMOS33 } [get_ports {out1[8]  }];
40 set_property -dict { PACKAGE_PIN C5      IOSTANDARD LVCMOS33 } [get_ports {out1[9]  }];
41 set_property -dict { PACKAGE_PIN B7      IOSTANDARD LVCMOS33 } [get_ports {out1[10] }];
42 set_property -dict { PACKAGE_PIN B6      IOSTANDARD LVCMOS33 } [get_ports {out1[11] }];
43 set_property -dict { PACKAGE_PIN A6      IOSTANDARD LVCMOS33 } [get_ports {out1[12] }];
44 set_property -dict { PACKAGE_PIN A5      IOSTANDARD LVCMOS33 } [get_ports {out1[13] }];
45 set_property -dict { PACKAGE_PIN D8      IOSTANDARD LVCMOS33 } [get_ports {out1[14] }];
46 set_property -dict { PACKAGE_PIN C7      IOSTANDARD LVCMOS33 } [get_ports {out1[15] }];
47 set_property -dict { PACKAGE_PIN E6      IOSTANDARD LVCMOS33 } [get_ports {out1[16] }];
48 set_property -dict { PACKAGE_PIN E5      IOSTANDARD LVCMOS33 } [get_ports {out1[17] }];
49 set_property -dict { PACKAGE_PIN E7      IOSTANDARD LVCMOS33 } [get_ports {out1[18] }];
50 set_property -dict { PACKAGE_PIN D7      IOSTANDARD LVCMOS33 } [get_ports {out1[19] }];
51
52
53 #####
54 # MISC
55 #####
56 set_operating_conditions -ambient_temp 80.0
57
58 set_property CONFIG_MODE SPIx1 [current_design]
59 set_property CFGBVS VCCO [current_design]
60 set_property CONFIG_VOLTAGE 3.3 [current_design]
61
62 set_property BITSTREAM.CONFIG.SPI_BUSWIDTH 1 [current_design]
63 set_property BITSTREAM.CONFIG.PERSIST NO [current_design]
64 set_property BITSTREAM.CONFIG.SPI_FALL_EDGE NO [current_design]
65 set_property BITSTREAM.CONFIG.CONFIGRATE 22 [current_design]

```

먼저 ex_code05f_1 모듈에 대한 결과입니다. Design Sources 에 “ex_code05f_1.v” 를 추가하고 Top 모듈로 지정합니다. Constraints 에 “ex_code05f.xdc”파일을 추가합니다. Generate Bitstream을 클릭합니다. (또는 Run Implementation을 클릭합니다)

아래는 Implementation 후에 Project Summary의 사용된 로직량을 보여줍니다.

Utilization			
Post-Synthesis Post-Implementation			
Graph Table			
Resource	Utilization	Available	Utilization %
LUT	11	20800	0.05
FF	1	41600	0.01
DSP	1	90	1.11
IO	42	210	20.00
BUFG	1	32	3.13

dsp가 90개 중에 1개가 사용되었습니다. LUT, FF는 각각 11, 1개 사용되었습니다.

이번에는 ex_code05f_2 모듈에 대한 결과입니다. (ex_code05f_2.v를 추가하고 Top 모듈로 지정하고 Run Implementation을 클릭합니다.)

Utilization			
Post-Synthesis Post-Implementation			
Graph Table			
Resource	Utilization	Available	Utilization %
LUT	114	20800	0.55
FF	114	41600	0.27
IO	42	210	20.00
BUFG	1	32	3.13

dsp는 사용되지 않고, LUT, FF는 각각 114, 114개가 사용되었습니다.

dsp 사용에 여유가 있으면, dsp를 사용하는 것이 유리한 것 같습니다. 1-clock안에서 처리할 수 있고 코드도 매우 간단합니다. dsp 사용이 초과되었을 때에는 두번째 방법을 사용하면 됩니다.

** 잠시 쉬어가세요

저자는 약 20년전부터 verilog 코딩을 하였습니다. 제가 verilog를 배웠던 분은 저보다 15년정도 나이가 많으신 분이었습니다. 그래서 제가 코딩하는 방법(?)은 최소 20~30년 전의 스타일입니다. fpga, verilog는 무척 HW 특성에 가깝기 때문에 편하다고 쉽게 바뀌버리면 문제가 발생하기 쉽습니다. 그래서 코딩 스타일을 잘 바꾸지 않습니다. 검증된 코드, 검증된 스타일만을 고집하는 경우가 많습니다. 저자도 그런 것 같습니다. 또한 단순히 FPGA에서 개발이 완료되고 끝나는 것이면 그래도 괜찮은데, 그 당시 주로 FPGA 개발은 ASIC을 개발하기 위한 과정이었습니다. ASIC을 개발하기 위하여 FPGA 보드에서 설계, 검증하고, 검증이 완료되면 ASIC을 개발하였습니다. 그렇기 때문에 FPGA에서 제공하는 IP들을 사용하는 것에 많은 제약이 있었습니다. 곱셈기도 그 중에 하나입니다. fpga에서 dsp를 제공한다고 그냥 사용하면, 나중에 asic으로 코드를 변환할 때 다시 구현해야 하거나, asic용 곱셈기 IP를 구매해야 합니다. 그래서 그 당시에는 dsp를 사용하는 코드를 사용하기 보다는 shift, adder 연산자를 활용하는 방법으로 곱셈기를 구현하였습니다.

그런데 이게 에러를 많이 발생하는 원인이 됩니다. 문제가 있어서 디버깅 하다 보면 주로 곱셈기를 구현하는 부분에서 에러가 빈번히 발생합니다. verilog 하시는 분들은 이전에 구현했던 코드를 잘 사용하지 않습니다. 그 때 그 때마다 새로 코딩하는 경우가 많습니다. 본인이 구현한 것도 잘 사용하지 않고, 새로 구현하는 경우가 많습니다. 이전의 코드, 복잡한 내용 다시 확인해서 사용하기 보다는 차라리 현재 스펙에 맞게 다시 구현하는 경우가 많습니다. 아무튼 프로젝트를 할 때마다, 곱셈기가 많은 오류를 일으키다 보니 짜증(?)이 많이 났습니다.

저는 처음부터 verilog 개발자는 아니었습니다. 저는 처음에는 fw 개발자로 시작해서, 뒤에 verilog를 알게 되고, 많은 분들이 그렇듯이 꼭 할 수 밖에 없는 상황이 되어서 verilog 개발자가 되었습니다.

어느 날 C의 함수처럼 곱셈기를 미리 만들어 놓고 사용하면 어떨까 하는 생각을 하였습니다. 그래서 곱셈기 함수화를 진행하였습니다. 1bit x 1bit 부터 17bits x 20bits 까지의 곱셈기를 모듈로 만들어서 simulation으로 검증하였습니다. 오류가 없음을 확인하고, 앞으로 모든 곱셈기는 이 모듈만을 사용하기로 하였습니다. 그 결과는 아주 만족스러웠습니다. 더이상 곱셈기에서 오류가 발생할 일이 없었습니다. 또한 곱셈기의 delay를 모듈 이름에 추가함으로 해서 몇번 delay가 발생하는지를 알게 되니, 코딩이 매우 수월했습니다.

곱셈기 모듈은 카페에 공유되어 있습니다. 다운받아서 코드를 보시면 bit 연산에 많은 도움을 받을 것으로 기대합니다.

링크 : <https://cafe.naver.com/worshippt/117>

5. SM (State Machine) 구현

이번 장에서는 SM에 대해서 설명합니다. SM은 알고리즘을 구현하는 여러 기법 중에 하나입니다. 반드시 SM을 사용해서 알고리즘을 구현해야 한다는 것을 아닙니다. 본인의 코딩 스타일에 맞게 구현하면 됩니다.

그러나 본서에서 SM을 다루는 이유는, 제가 오랜 동안 verilog 코딩을 하면서 SM을 사용하면 많은 장점들이 있다는 것을 알게 되었습니다. SM을 사용하지 않으면, if - else - if - else 등 반복되는 조건문들이 이어지고 코드가 한눈에 보이지 않아서 디버깅도 어렵고 알고리즘 구현도 어려움이 많았습니다. 그러나 SM을 사용하면 코드도 정리가 되고, 조건문들이 복잡하게 얽히지 않아서 매우 효율적으로 코딩할 수 있었음을 경험적으로 알게 되었습니다.

5.1 SM 구성

SM은 총 4가지로 구성되어 있습니다.

- ✓ 상태 정의
- ✓ 상태 플래그 생성
- ✓ 상태별 코드 구현
- ✓ 상태 전이 구현

이번장에서는 간단한 내용을 SM을 이용하여 구현합니다. (ex_code06a.v)

- ✓ 10bits의 in1, in2, 2개의 입력을 받아 순차적으로 더하기, 빼기, 곱하기를 수행합니다.
- ✓ in1, in2의 연산결과를 out 으로 출력합니다.
- ✓ 3가지의 연산이 완료되면, done 신호를 “H”로 출력합니다.
- ✓ start 신호가 발생하면 시작합니다. STATE1으로 이동합니다.
- ✓ STATE1에서는 $in1 + in2$ 를 출력합니다. 잠시후에 STATE2로 이동합니다.
- ✓ STATE2에서는 $in1 - in2$ 를 출력합니다. 잠시후에 STATE3로 이동합니다.
- ✓ STATE3에서는 $in1 * in2$ 를 출력합니다. DONE 상태로 이동합니다.

5.2 상태 정의

상태는 기본적으로 IDLE, STATE1, STATE2, ~ DONE 으로 구성됩니다. IDLE, DONE 상태는 기본적인 상태이고, 중간에 필요한 상태들을 추가합니다. 상태가 3개 이므로 총 5개의 상태를 정의합니다.

```

38 // -----
39 // 1) define state
40 reg      [2:0]    m_state ;
41 parameter      M_IDLE    = 3'd0 ;
42 parameter      M_STATE1  = 3'd1 ;
43 parameter      M_STATE2  = 3'd2 ;
44 parameter      M_STATE3  = 3'd3 ;
45 parameter      M_DONE    = 3'd4 ;

```

5.3 상태 플래그 생성

5개의 상태에 대한 Flag를 생성합니다.

```

48 // -----
49 // 2) state flag
50 wire      s_idle    = (m_state == M_IDLE    ) ? 1'b1 : 1'b0 ;
51 wire      s_statel  = (m_state == M_STATE1  ) ? 1'b1 : 1'b0 ;
52 wire      s_state2  = (m_state == M_STATE2  ) ? 1'b1 : 1'b0 ;
53 wire      s_state3  = (m_state == M_STATE3  ) ? 1'b1 : 1'b0 ;
54 wire      s_done    = (m_state == M_DONE    ) ? 1'b1 : 1'b0 ;

```

5.4 코드 구현

각 상태에 따른 코드를 구현합니다. 각 상태에서 사용할 counter를 생성하면 상태 전이 등 여러 용도로 사용 가능합니다. 각 상태에 따른 Counter를 생성하여 사용하겠습니다.

```

59 // -----
60 // 3) code implementation
61
62 // 3.1) idle state
63 reg      start_ld, start_2d;
64 wire      start_pedge = start_ld & ~start_2d;
65 always @(posedge clock or negedge reset)
66 begin
67     if(~reset) begin
68         start_ld <= 1'b0;
69         start_2d <= 1'b0;
70     end
71     else begin
72         start_ld <= start;
73         start_2d <= start_ld;
74     end
75 end

```

- ✓ 라인 63 - 75 : idle state 에서는 start 신호의 Positive Edge를 생성합니다. start 신호가 Active 되면 연산을 시작하기 위하여 STATE1으로 이동합니다.

```

78 // 3.2) statel state
79 reg    [2:0]    statel_cnt;
80 always @(posedge clock or negedge reset)
81 begin
82     if(~reset)    statel_cnt <= 3'b0;
83     else          statel_cnt <= ~s_statel ? 3'b0 : statel_cnt+1'b1;
84 end
85
86 wire    [10:0]   add = {1'b0, in1} + {1'b0, in2};

```

✓ 라인 79 - 84 : state1에서 사용할 counter를 생성합니다.

✓ 라인 86 : 더하기를 구현합니다.

```

89 // 3.3) state2 state
90 reg    [2:0]    state2_cnt;
91 always @(posedge clock or negedge reset)
92 begin
93     if(~reset)    state2_cnt <= 3'b0;
94     else          state2_cnt <= ~s_state2 ? 3'b0 : state2_cnt+1'b1;
95 end
96
97 wire    [10:0]   sub = {1'b0, in1} - {1'b0, in2};

```

✓ 라인 90 - 95 : state2에서 사용할 counter를 생성합니다.

✓ 라인 97 : 빼기를 구현합니다.

```

100 // 3.4) state3 state
101 reg    [2:0]    state3_cnt;
102 always @(posedge clock or negedge reset)
103 begin
104     if(~reset)    state3_cnt <= 3'b0;
105     else          state3_cnt <= ~s_state3 ? 3'b0 : state3_cnt+1'b1;
106 end
107
108 wire    [19:0]   mul = in1 * in2;

```

✓ 라인 101 - 106 : state3에서 사용할 counter를 생성합니다.

✓ 라인 108 : 곱하기를 구현합니다.

```

119 // output
120 reg    [19:0]   out;
121 always @(posedge clock or negedge reset)
122 begin
123     if(~reset)    out <= 20'b0;
124     else          out <= (s_statel & (statel_cnt==3'd3)) ? {9'b0, add} :
125                          (s_state2 & (state2_cnt==3'd3)) ? (sub[10] ? 20'b0 : {9'b0, sub}) :
126                          (s_state3 & (state3_cnt==3'd3)) ? mul :
127                          out ;
128 end

```

✓ 라인 120 - 128 : 출력을 구합니다. 각 상태에 따라서 출력을 구합니다. 빼기의 경우, 음수가 나오면 0를 출력합니다.


```

130 // done
131 reg done;
132 always @(posedge clock or negedge reset)
133 begin
134     if(~reset) done <= 1'b0;
135     else done <= (s_idle & start_pedge) ? 1'b0 :
136                 (s_done & (done_cnt==3'd7)) ? 1'b1 : done ;
137 end

```

- ✓ 라인 131 - 137 : done 신호를 생성합니다. start 신호가 Active 되면 0가 되고, 연산을 모두 마치면 1이 됩니다.

5.5 상태 전이 구현

마지막으로 상태 전이를 구현합니다.

```

140 // -----
141 // 4) state transition
142 always @(posedge clock or negedge reset)
143 begin
144     if(~reset) m_state <= 3'b0;
145     else m_state <= (s_idle & start_pedge) ? M_STATE1 :
146                     (s_statel & (statel_cnt==3'd7)) ? M_STATE2 :
147                     (s_state2 & (state2_cnt==3'd7)) ? M_STATE3 :
148                     (s_state3 & (state3_cnt==3'd7)) ? M_DONE :
149                     (s_done & (done_cnt ==3'd7)) ? M_IDLE :
150                     m_state ;
151 end
152
153 endmodule

```

- ✓ 라인 142 - 151 : idle 상태에서 start 신호가 Active 되면 state1으로 이동하고, state1에서는 연산이 수행되고 잠시 기다렸다가 state2로 이동합니다. state2, state3, done 모두 같은 방식으로 동작합니다.

5.6 simulation 결과 확인

Test bench를 이용하여 simulation으로 결과를 확인합니다. (tb_ex_code06a.v)

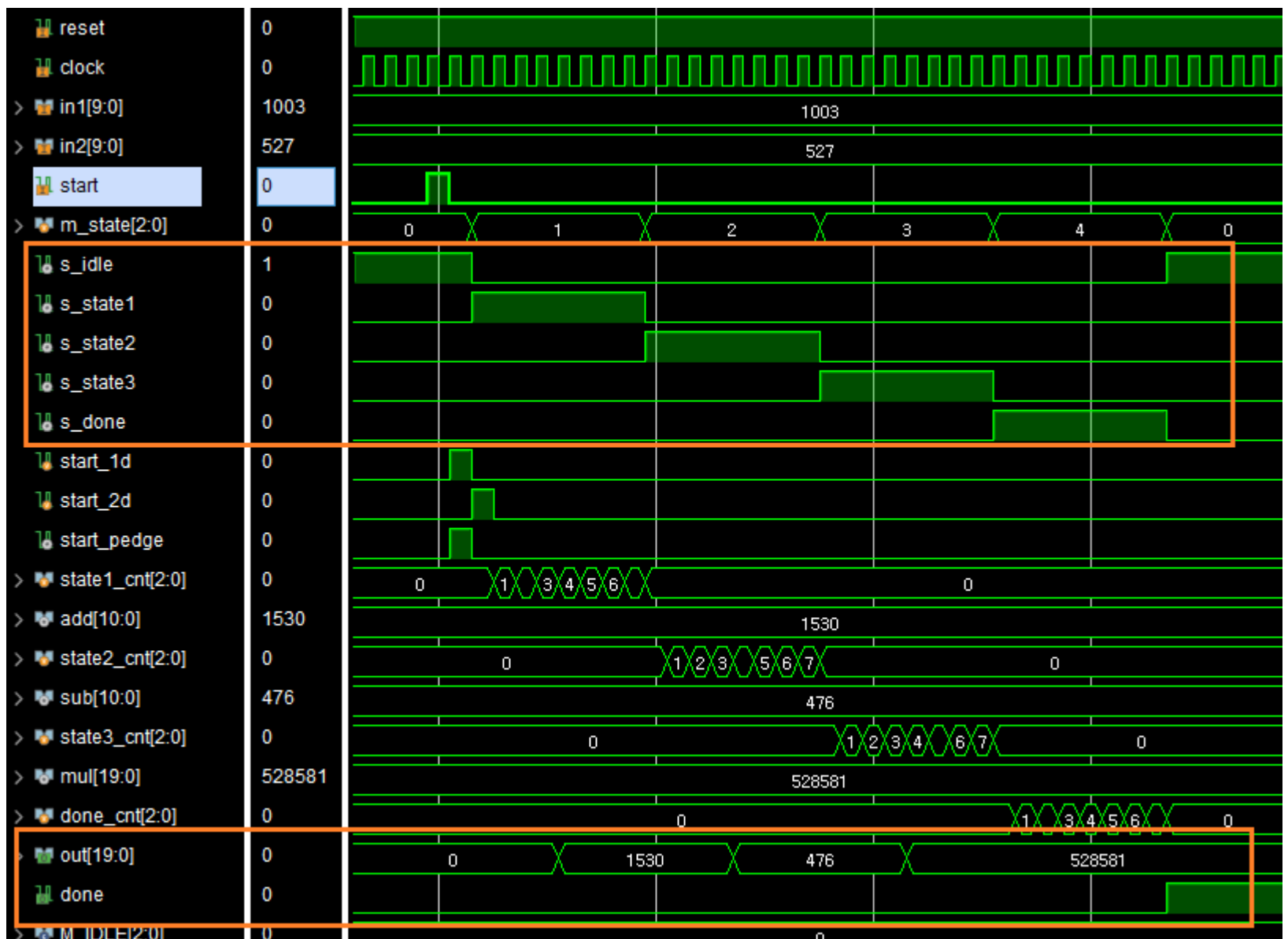
```

23 module tb_ex_code06a();
24
25     reg            reset, clock;
26 initial          begin
27         reset = 1'b0;
28         clock = 1'b0;
29
30     #5000    reset = 1'b1;
31
32 end
33
34 always #5    clock = ~clock;           // 100 Mhz
35
36
37 reg    [5:0]    cnt;
38 always @(posedge clock or negedge reset)
39 begin
40     if(~reset)    cnt <= 6'b0;
41     else          cnt <= (cnt==6'd63) ? 6'd63 : cnt + 1'b1;
42 end
43
44 wire            start = (cnt==6'd30) ? 1'b1 : 1'b0;
45 wire    [9:0]    in1  = 10'd1003;
46 wire    [9:0]    in2  = 10'd527;
47
48 // 1003 + 527 = 1530
49 // 1003 - 527 = 476
50 // 1003 * 527 = 528,581
51 wire    [19:0]    out  ;
52 wire            done  ;
53 ex_code06a ex_code06a(
54     .reset      (reset   ),
55     .clock       (clock  ),
56     .start       (start  ),
57     .in1         (in1    ),
58     .in2         (in2    ),
59     .out         (out     ),
60     .done        (done   )
61 );
62
63 endmodule

```

in1 = 1003, in2 = 527 을 입력하고, start신호를 Active 합니다. out, done 값을 확인합니다.

아래는 simulation 결과를 보여줍니다.



start 신호가 Active 되면 연산을 시작합니다. state가 순차적으로 이루어지는 것을 확인할 수 있습니다. out 값이 예상했던 대로 1530, 476, 528581로 출력되고, 연산이 완료되면 done 신호가 Active 되는 것을 알 수 있습니다.

본 예제는 SM을 사용하지 않고서도 더 간단히 구현할 수 있을 것입니다. 여기에서는 주어진 알고리즘을 SM을 이용하여 어떻게 구현하는지를 보여주고 있습니다. 저자의 경험에 의하면, 아주 간단한 것을 제외하고는 거의 모든 경우에 SM을 이용하여 알고리즘을 구현하는 것이 훨씬 효과적이라고 생각합니다.

본장에서 설명하고 있는 내용대로 SM을 어떻게 구현하는지를 이해하고 본인의 것으로 습득한다면 많은 도움이 될 것으로 생각합니다.

SM을 구현하는 방법/스타일은 많이 있습니다. 그 중에서 자신에게 맞는 한가지 방법을 찾고 그 방법대로 구현하는 습관을 가지는 것이 중요합니다.

6. for 루프 구현

이번 장에서는 for 루프를 구현하는 것을 알아보도록 하겠습니다. 상위 언어 (본 장에서는 C 를 사용합니다)에서 구현하는 방법과 verilog에서 구현하는 방법에 대해서 알아보고 그 차이점이 무엇인지 알아보겠습니다.

for 루프를 사용하는 예로 1 ~ 255 까지의 합을 구하는 것을 설명합니다.

6.1 C 로 구현

C로 구현하는 것은 매우 간단합니다.

```

2  int c_sum(int *data, int len)
3  {
4      int sum = 0;
5      int i;
6
7      for (i=0; i<len; i++)
8      {
9          sum += data[i];
10     }
11
12     return sum;
13 }
14
15 int main()
16 {
17     int i;
18     int data[255];
19     int sum;
20
21     for(i=0; i<255; i++)
22     {
23         data[i] = i+1;
24     }
25
26     sum = c_sum(data, 255);
27     printf("sum : %d \r\n", sum);
28
29     return 0;
30 }
```

- ✓ 라인 2 - 13 : len 으로 설정된 갯수 만큼 data 배열의 값을 더해서 합을 리턴하는 함수
- ✓ 라인 21 - 24 : data 생성
- ✓ 라인 26 - 27 : c_sum 함수를 호출하여 결과를 생성하고, 그 결과를 출력함.

6.2 verilog로 구현

verilog 로 구현하기 위해서는 많은 것을 고려해야 합니다. 먼저 verilog는 data type이 없습니다. 사용자가 직접 bit 수를 정해야 합니다. 1 ~ 255까지의 합을 구하기 위해서는 다음과 같이 bits 수를 정합니다.

- ✓ 입력되는 data : 1 ~ 255 이므로 unsigned 8bits 로 설정합니다.
- ✓ 출력되는 sum : 8bits x 255(개수, 8bits), unsigned 16bits 로 설정합니다.

모듈을 다음과 같이 구현합니다.

```

23 module ex_code07a(
24     reset    ,
25     clock    ,
26     en       ,
27     in1      ,
28     in2      ,
29     out1
30 );
31 input      reset    ;
32 input      clock    ;
33 input      en       ;
34 input [7:0] in1      ;
35 input [15:0] in2     ;
36 output [15:0] out1   ;
37
38 reg [15:0] out1 ;
39 always @(posedge clock or negedge reset)
40 begin
41     if(~reset)      out1 <= 16'b0 ;
42     else            out1 <= ~en ? in2 : {8'b0, in1} + in2 ;
43 end
44
45 endmodule

```

- ✓ 라인 31 - 32 : reset, clock
- ✓ 라인 33 : enable flag, en=1 이면 연산을 수행하고, en=0 이면 이전 값을 유지함. verilog 에서는 매우 중요합니다. 대부분 enable flag를 사용합니다.
- ✓ 라인 34 : 입력 데이터, 1 ~ 255
- ✓ 라인 35 : 이전의 연산 결과를 입력으로 사용합니다.
- ✓ 라인 36 : 출력 데이터, 연산 결과를 출력합니다.
- ✓ 라인 38 - 43 : en=0 이면, 이전 연산결과를 출력(값이 변하지 않음)하고, en=1 이면 “in1 + in2”를 출력합니다. 이 때 unsigned 연산이 수행됩니다.

Test Bench는 tb_ex_code07a.v 입니다.

```

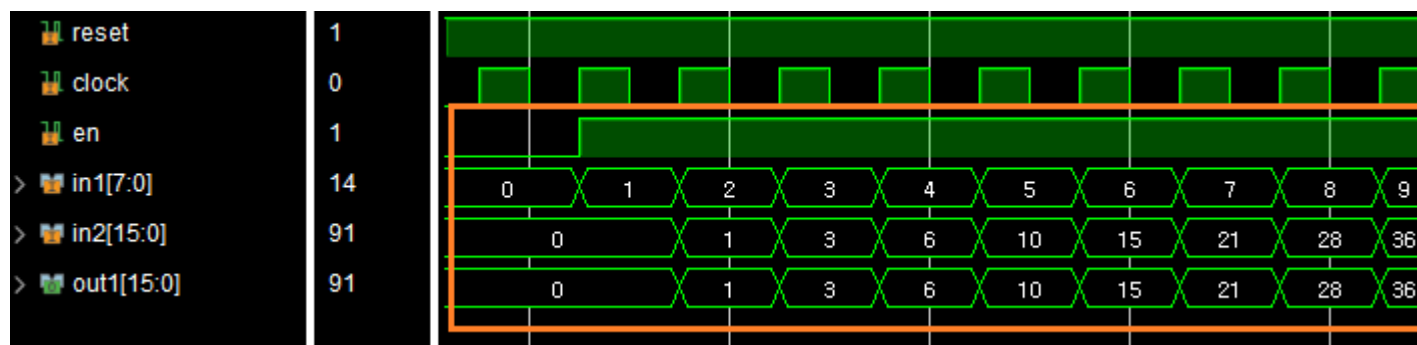
23 module tb_ex_code07a();
24
25     reg            reset, clock;
26     initial        begin
27         reset = 1'b0;
28         clock = 1'b0;
29
30         #5000 reset = 1'b1;
31     end
32
33     always #5      clock = ~clock;          // 100 Mhz
34
35     reg [9:0] cnt;
36     always @(posedge clock or negedge reset)
37     begin
38         if(~reset) cnt <= 10'b0;
39         else cnt <= (cnt==10'd1023) ? 10'd1023 : cnt + 1'b1;
40     end
41
42     reg en;
43     always @(posedge clock or negedge reset)
44     begin
45         if(~reset) en <= 1'b0;
46         else en <= (cnt==10'd100) ? 1'b1 :
47                     (cnt==10'd355) ? 1'b0 : en ;
48     end
49
50     reg [7:0] in1;
51     always @(posedge clock or negedge reset)
52     begin
53         if(~reset) in1 <= 8'b0;
54         else in1 <= ((cnt>=10'd100) && (cnt< 10'd355)) ? in1+1'b1 : in1 ;
55     end
56
57     wire [15:0] sum;
58     ex_code07a ex_code07a(
59         .reset (reset ),
60         .clock (clock ),
61         .en (en ),
62         .in1 (in1 ),
63         .in2 (sum ),
64         .out1 (sum )
65     );
66
67 endmodule

```

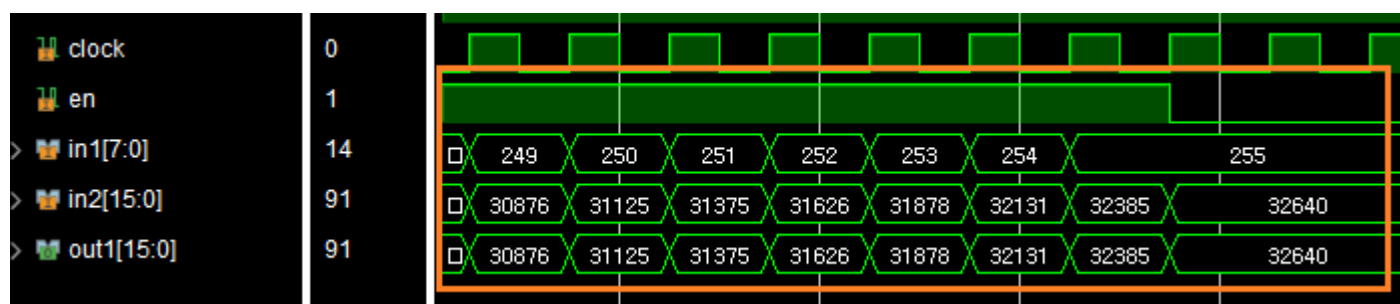
- ✓ 라인 25 - 33 : reset, clock 생성
- ✓ 라인 35 - 40 : en, in1 신호 생성을 위한 카운터 생성
- ✓ 라인 42 - 48 : en 신호 생성, 255 -clock 동안 enable
- ✓ 라인 50 - 55 : in1 신호 생성, en=1 일 때, 1 ~ 255 까지 입력됨.
- ✓ 라인 57 - 65 : ex_code07a 모듈 생성, in2, out1 모두에 sum 값이 사용됨에 주의.

simulation 결과를 확인합니다. tb_ex_code07a.v, ex_code07a.v 파일을 Simulaton Sources에 추가합니다. tb_ex_code07a 모듈을 Top 모듈로 지정하고, Run Simulation - Run Behavioral Simulation 을 실행합니다. ex_code07a의 신호들을 Wave 윈도우에 추가하고, “run 0.1ms”를 실행합니다. 모든 신호를 unsigned Decimal로 바꾸어 줍니다.

en 신호가 Active(1) 일 때, in1, in2 값이 입력되고, out1은 1-clock delay 후에 나타납니다.



마지막 부분을 보여줍니다.



in1값이 255까지 입력되고, en 신호는 255 이후에는 0가 됩니다. 최종 합은 out1 = 32640 이 됩니다.

verilog로 코드를 구현할 때에는 매 clock 마다 데이터 흐름을 주의해야 합니다.

여기서 3항 연산자에 대해서 잠깐 살펴보도록 하겠습니다. 아래 코드는 모두 동일한 코드입니다. 코딩하는 스타일에 따라 선택하면 됩니다. 저는 첫번째 코드를 선호하는 편입니다. 통상적으로 등호(==)가 부등호(>, <) 보다 처리속도가 빠릅니다. 요즘에는 워낙 기술이 발달해서 크게 차이가 나지는 않습니다.

```

39 reg                en;
40 always @(posedge clock or negedge reset)
41 begin
42     if(~reset)      en <= 1'b0;
43     else            en <= (cnt==10'd100) ? 1'b1 :
44                        (cnt==10'd355) ? 1'b0 : en ;
45 end
46
47 reg                en;
48 always @(posedge clock or negedge reset)
49 begin
50     if(~reset)      en <= 1'b0;
51     else            en <= ((cnt>=10'd100) && (cnt<10'd355)) ? 1'b1 : 1'b0 ;
52 end

```

라인 43 - 44

- ✓ if (cnt==100) en = 1;
- ✓ else if(cnt==355) en = 0;
- ✓ else en = en;

라인 50 - 51

- ✓ if((cnt>=100) && (cnt<355)) en = 1;
- ✓ else en = 0;

7. 그 외의 내용

이번장에서는 verilog 가 상위 Language(C, C++, JAVE, python, 기타)와 어떻게 다른 지 간단히 살펴보도록 하겠습니다. 프로젝트를 진행하다 보면, verilog로 처리해야할 부분과 MCU를 사용하여 fw로 처리하는 부분들을 나누어서 진행할 필요성이 많이 있습니다. verilog를 마치 fw를 구현하듯 그렇게 코딩하면 많은 어려움과 오류가 발생할 가능성이 많습니다.

그러면 verilog와 fw로 구현할 부분들을 어떻게 나눌까요?

간단히 예를 들어 설명해 보겠습니다.

i2c controller (master, slave) 자체를 구현하는 것은 verilog로 구현하는 게 더 좋습니다. 그러나 i2c에 데이터를 쓰고, 데이터를 읽어서 어떤 일들을 처리하는 것은 fw로 구현합니다.

mcu안에는 많은 peripheral 들이 있습니다. spi, uart, i2c, ethernet, 등 이러한 것들은 verilog 로 구현되었습니다. (물론 analog IP들도 많이 있습니다. analog IP는 verilog로 구현하지 않습니다) 이러한 peripheral 들을 사용하여 상위단에서 알고리즘을 구현하는 것은 fw로 구현하게 됩니다.

처음 verilog를 접하게 되면, fw나 기타 상위 프로그램 언어와 같이 코딩하려고 합니다. 전연변수를 사용하거나 배열을 사용하거나 반복문(for, while 루프)을 사용하려고 합니다.

많은 상위 프로그램 언어들은 서로 비슷한 면들이 많이 있습니다. 객체지향, 클래스, 메소드, 등등

그러나 verilog는 완전 다른 세계입니다. verilog는 HW를 설계하는 것입니다. 고급 기술이 잘 통하지 않습니다. (물론 system verilog라고 해서 고급 언어와 같이 프로그램할 수 있는 것이 있지만, 아직까지는 기술적 한계가 있다고 생각합니다. 본서에서는 다루지 않고 배제하고 설명합니다)

저자의 경험에 의하면, 몇가지 기본기술을 습득하고 기본기술만을 사용하여 코딩하는 것이 좋은 것 같습니다. 만약 새로운 고급기술을 사용하려면 반드시, simulation등을 거쳐서 오류 없이 동작하는지 확인후 사용해야 합니다.

제가 처음 verilog를 배우면서 잘못 사용했던 오류에 대해서 몇가지 예를 들어 설명하겠습니다.

1) register의 출력은 한곳에서만 해야 합니다.

c에서는 변수의 출력(대입문)을 여러곳에서 사용합니다. 예를 들면 전역변수를 설정하고, 인터럽트 서비스 루틴에서 어떤 값을 설정하고, main문에서 그 값을 사용하거나 다른 값으로 변경합니다. verilog에서는 어떤 register의 출력은 한곳에서만 허용됩니다. 즉 해당 register의 출력에 영향을 주는 모든 경우의 수를 한곳에 모아서 코딩해야 합니다.

아래 코드는 ex_code06a.v의 상태 전이를 구현하는 부분을 보여줍니다.

```

140 // -----
141 // 4) state transition
142 always @(posedge clock or negedge reset)
143 begin
144     if(~reset)        m_state <= 3'b0;
145     else              m_state <= (s_idle   & start_pedge   ) ? M_STATE1 :
146                               (s_state1 & (state1_cnt==3'd7)) ? M_STATE2 :
147                               (s_state2 & (state2_cnt==3'd7)) ? M_STATE3 :
148                               (s_state3 & (state3_cnt==3'd7)) ? M_DONE   :
149                               (s_done   & (done_cnt  ==3'd7)) ? M_IDLE   :
150                               m_state ;
151 end

```

m_state의 결과 값에 영향을 주는 모든 경우가 always 한 문장에 포함되어 있습니다. 라인 145-149의 내용들이 여기저기 흩어져 있으면 안됩니다. m_state의 결과는 오직 이곳에서만 출력되게 코딩해야 합니다.

2) verilog 는 크게 2가지로 나누어집니다. 합성 가능한 코드, 합성 불가능한 코드 입니다.

합성 가능한 코드는 HW로 구현되는 코드입니다. 합성 불가능한 코드는 HW로 구현될 수 없는 코드이지만, simulation 용으로는 사용가능한 코드입니다. 기술이 발달하면서 점점 합성 불가능한 코드(혹은 문법)들이 합성 가능한 코드들로 바뀌고 있습니다. 그래서 그 경계도 애매모호합니다. 저의 경험으로는 신기술은 최대한 늦게 받아들이고, 예전 기술을 익히고 예전 기술을 사용하여 코딩하는 것이 좋은 것 같습니다. 저의 코드들은 매우 단순합니다. 사용하는 기술들이 몇개 없습니다. reg, wire, 3항 연산자, 결합 연산자, always, assign, bit 연산, 이게 전부인 것 같네요~

3) verilog 에 데이터 타입이 없습니다.

verilog에는 reg, wire 밖에 없습니다. 또한 reg, wire로 선언된 데이터의 타입이 없습니다. (simulation을 위한 코드에서는 데이터 타입을 사용할 수 있습니다) 데이터 타입은 프로그래머가 직접 생각해서 정해야 합니다. 예를 들어 unsigned integer로 사용하기 위해서는 값의 범위를 정해서 몇 비트로 설정할 것인지를 정해야 합니다. float형으로 사용하기 위해서는 정밀도를 얼마까지 해야 할지를 정해서 소수점이하 몇자리까지 사용할지를 정해야 합니다. 아래는 몇가지 예입니다.

```

reg      [7:0]  data1;      // unsigned 로 사용할 때 : 0 ~ 255

                                // signed 로 사용할 때 : -128 ~ +127

reg      [15:0] data2;      // unsigned 로 사용할 때 : 0 ~ 65535

                                // signed 로 사용할 때 : -32768 ~ +32767

                                // unsigned float 로 사용할 때 : [15:6] : 정수부, [5:0] : 소수부

                                // unsigned float 로 사용할 때 : [15:10] : 정수부, [9:0] : 소수부

                                // signed float 로 사용할 때 : [15] : sign bit, [14:6] : 정수부, [5:0] : 소수부

                                // signed float 로 사용할 때 : [15] : sign bit, [14:10] : 정수부, [9:0] : 소수부

```

reg [15:0] data2; 로 선언했을 때, unsigned int, signed int, unsigned float, signed float 등 다양하게 사용할 수 있습니다. 프로그래머가 data2를 어떤 데이터 타입으로 사용할지를 정해서 사용합니다. 따라서 데이터 Overflow 에러나 데이터 형변환 이런 것들이 적용되지 않습니다. 오로지 프로그래머가 알아서 다 해야 합니다.

4) 모듈과 함수의 차이

verilog 에서의 모듈은 C 에서의 함수와 비슷하다고 생각할 수 있지만, 사실은 전혀 다른 개념입니다. 모듈은 큰 Block을 나누어서 설계하기 위한 것입니다. 함수도 마찬가지로 자주 사용하는 기능을 함수로 만들어서 호출해서 사용합니다.

동일한 모듈을 10번 사용하는 것과, 동일한 함수를 10번 사용하는 것의 차이가 무엇일까요? (본장에서는 모듈은 verilog에서 사용하는 모듈을 의미하고, 함수는 C (또는 기타 다른 SW)에서 사용하는 함수를 의미합니다)

verilog는 기본적으로 HW를 설계하는 것입니다. HDL (Hardware Description Language)의 태생도 어떻게 하면 HW 설계를 SW와 같이 할 수 있을까 하는 곳에서 시작되었습니다. (미국놈들은 대단한 것 같습니다. 어떻게 그 미개한 시대에 그러한 생각을 할 수 있었을까요?) 당연히 verilog는 HW를 설계하는 툴(or Language)입니다.

함수는 호출의 개념을 사용합니다. 함수 코드가 특정 메모리에 들어 있고, 함수를 사용할 때마다 그 함수가 있는 메모리에서 함수를 실행하고 다시 돌아옵니다. 10번 함수를 호출한다고 해서 프로그램의 양이 10배 증가하지는 않습니다. 단지 호출하는 라인만큼만 증가합니다.

반면에 verilog에서 모듈은 10번 사용하면 사이즈가 10배로 증가합니다. verilog는 호출의 개념이 아니라 그 모듈이 그 자리에 들어가는 것입니다. 회로 설계를 할 때, 같은 블록을 여러번 사용해도 같은 블록을 동일하게 추가하는 것과 같습니다.

verilog 에서의 모듈은 C에서의 매크로 정의나 인라인 함수와 같다고 생각할 수 있습니다.

지금까지 verilog를 배울 때 알아야 하는 기본적인이고 중요한 내용을 설명하였습니다. 추후에 필요한 부분이 있으면 자료를 업데이트할 예정입니다. 본서에 설명된 소스는 자료실을 통해 다운받으시길 바랍니다.

궁금한 사항은 카페(<https://cafe.naver.com/worshippt>) 나 인프런에 질문을 남겨주시면 답변해 드리도록 하겠습니다.

감사합니다.

IHIL

8. Revision History

Date	Version	Description of Revisions
2023. 03. 02	1.0	Initial Release
2023. 03. 21	1.1	for 루프 구현 추가됨.
2023. 06. 10	1.2	오타 수정



[저작권 관련 내용]

이 자료는 대한민국 저작권법의 보호를 받습니다. 작성된 모든 내용의 권리는 작성자에게 있으며, 작성자의 동의 없는 사용이 금지됩니다.

본 자료의 일부 혹은 전체 내용을 무단으로 복제/배포하거나 2차적 저작물로 재편집하는 경우, 5년 이하의 징역 또는 5천만원 이하의 벌금과 민사상 손해배상을 청구합니다.

※ 저작권법 제 30조(사적이용을 위한 복제)

공표된 저작물을 영리를 목적으로 하지 아니하고, 개인적으로 이용하거나 가정 및 이에 준하는 한정된 범위 안에서 이용하는 경우에는 그 이용자는 이를 복제할 수 있다. 다만, 공중의 사용에 제공하기 위하여 설치된 복사기기에 의한 복제는 그러지 아니하다.

※ 저작권법 제 136조(벌칙)의 ① 다음 각 호의 어느 하나에 해당하는 자는 5년 이하의 징역 또는 5천만원 이하의 벌금에 처하거나 이를 병과할 수 있다.

지적재산권 및 이 법에 따라 보호되는 재산적 권리(제93조에 따른 권리는 제외한다)를 복제, 공연, 공중 송신, 전시, 배포, 대여, 2차적 저작물 작성의 방법으로 침해한 자

※ 민법 제750조(불법행위의 내용)

고의 또는 과실로 인한 위법행위로 타인에게 손해를 가한 자는 그 손해를 배상할 책임이 있다.